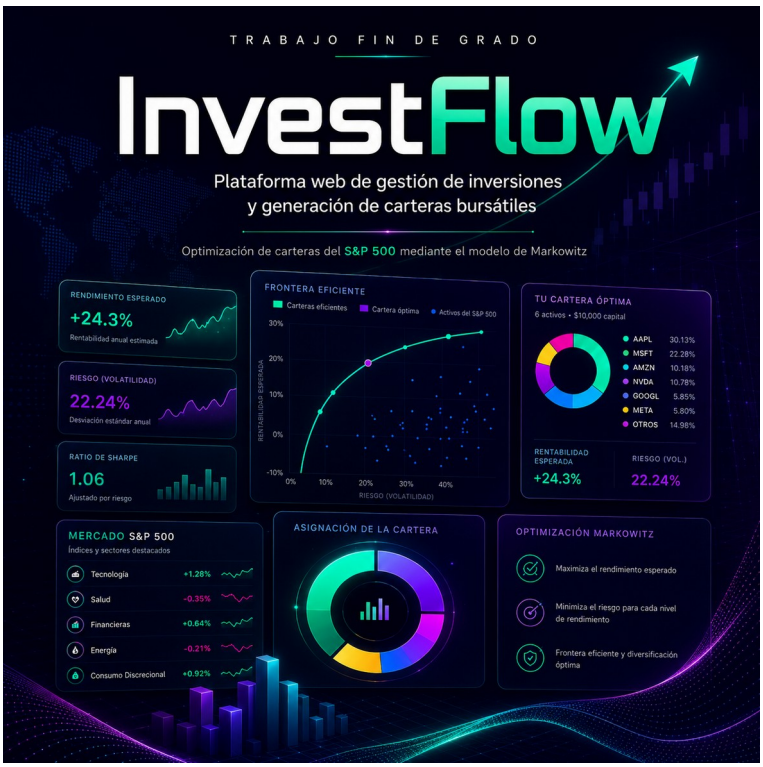
BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

GRADO EN Ingeniería Informática de Gestión y Sistemas de Información

TRABAJO FIN DE GRADO

Aplicación web para la generación y análisis de carteras de inversión del S&P 500 mediante modelos matemáticos de optimización













Estudiante: Uresandi Lamas, Eneko

Director/Directora: Bengoetxea Kortazar, Kepa Xabier

Curso: 2025-2026

Fecha: Bilbao, 15 de Junio de 2026

Resumen Laburpena Abstract

Este proyecto presenta una plataforma web de gestión de inversiones que facilita la creación de carteras bursátiles. Con acceso a datos históricos reales del S&P 500, permite generar automáticamente carteras diversificadas adaptadas en función del perfil de riesgo del usuario, obteniendo una distribución del capital entre activos y sectores.

Especialmente está orientada a inversores particulares sin conocimientos avanzados de finanzas, que quieren acceder a estrategias de inversión de forma sencilla. Para ello, la aplicación tiene un optimizador basado en la Teoría Moderna de Carteras de Markowitz, a partir de la tolerancia al riesgo del usuario y un horizonte temporal, selecciona automáticamente los mejores activos del mercado y calcula los pesos óptimos para cada posición.

La aplicación ofrece dos modalidades: una cartera por sector individual, que optimiza cada uno de los once sectores del S&P 500 de forma independiente, y una multisector, que combina los mejores activos de todos los sectores en una única cartera diversificada.

Palabras Clave: Gestión de inversiones, Carteras bursátiles, Diversificación, Perfil de riesgo, Datos financieros, Aplicación web.

Proiektu honek inbertsioak kudeatzeko web-plataforma bat aurkezten du, burtsa-zorroak sortzea errazten duena. S&P 500-eko datu historiko errealetara sartzearekin, erabiltzailearen arrisku-profilaren arabera egokitutako diru-zorro dibertsifikatuak automatikoki sortzeko aukera ematen du, kapitala aktiboen eta sektoreen artean banatuz.

Bereziki, finantza-ezagutza aurreraturik ez duten inbertitzaile partikularrei zuzenduta dago, inbertsio-estrategiak modu errazean eskuratu nahi baitituzte. Horretarako, aplikazioak Markowitzen Zorroen Teoria Modernoan oinarritutako optimizatzaile bat du, erabiltzailearen arriskuarekiko tolerantzia eta denbora-horizontea oinarri hartuta, merkatuko aktibo onenak automatikoki hautatzen ditu eta posizio bakoitzerako pisu optimoak kalkulatzen ditu.

Aplikazioak bi modalitate eskaintzen ditu: banako sektore bakoitzeko zorro bat, S&P 500 sektoreko hamaika sektoreak modu independentean optimizatzen dituena, eta sektore guztietako aktibo onenak kartera dibertsifikatu bakar batean konbinatzen dituen sektore anitzeko zorro bat.

Gako Hitzak: Inbertsioen kudeaketa, Burtsa-zorroak, Dibertsifikazioa, Arrisku-profila, Finantza-datuak, Web-aplikazioa.

This project presents a web-based investment management platform that facilitates the creation of stock portfolios. With access to real historical S&P 500 data, it allows the automatic generation of diversified portfolios tailored to the user's risk profile, obtaining a capital allocation across assets and sectors.

It is especially aimed at individual investors without advanced financial knowledge who want to access investment strategies in a simple way. To achieve this, the application includes an optimizer based on Markowitz's Modern Portfolio Theory, which, using the user's risk tolerance and investment horizon, automatically selects the best market assets and calculates the optimal weights for each position.

The application offers two modes: a single-sector portfolio, which optimizes each of the eleven S&P 500 sectors independently, and a multi-sector portfolio, which combines the best assets from all sectors into a single diversified portfolio.

Keywords: Investment management, Stock portfolios, Diversification, Risk profile, Financial data, Web application.

Índice

Resumen Laburpena Abstract 1

Lista de figuras 7

Lista de tablas 9

Lista de acrónimos 11

Glosario 13

1. Empleo de Inteligencia Artificial 15

2. Introducción 16

2.1. Descripción del proyecto . . . 17

2.2. Conceptos . . . 18

2.3. Razones de la elección del TFG . . . 24

3. Enfoque Inicial 26

3.1. Conexión con los Objetivos de Desarrollo Sostenible y los principios y valores democráticos (ODS) . . . 26

3.2. Objetivos . . . 27

3.3. Alcance . . . 29

3.4. Herramientas . . . 38

3.5. Análisis de Riesgos . . . 41

3.6. Descripción del presupuesto . . . 43

4. Estado del Arte 47

4.1. Fuentes de datos financieros . . . 47

4.2. Aplicaciones similares . . . 48

4.3. Conclusiones 53

5. Captura de Requisitos **55**

5.1. Documento de Requisitos 55

5.2. Casos de Uso 57

5.3. Modelo de Dominio 60

6. Análisis y diseño **63**

6.1. Diagrama relacional de la Base de Datos 63

6.2. Arquitectura 66

6.3. Diseño de Interfaces 67

7. Desarrollo **85**

7.1. Librerías y Dependencias 85

7.2. Instalación y Preparación del Entorno 88

7.3. Implementación 89

7.4. Aspectos de Seguridad 96

8. Pruebas **98**

8.1. Pruebas de Flujo de Trabajo 98

8.2. Pruebas de funcionalidad 98

8.3. Pruebas con Usuarios Reales 103

8.4. Pruebas de Seguridad 103

8.5. Pruebas de indicadores 104

8.6. Pruebas del módulo de backtesting 105

9. Conclusiones **106**

9.1. Objetivos 106

9.2. Reflexión Personal 107

10.Trabajo Futuro **108**

10.1. Envío Real de Correos y Autenticación 108

10.2. Rebalanceo Automático de Carteras 108

10.3. Ampliación de activos 109

10.4. Conexión con Broker Real 109

10.5. Comunidad de inversores 109
10.6. Aplicación Móvil 109

Bibliografía **110**

Lista de figuras

1. Evolución del agregado monetario M2 de la Reserva Federal de Estados Unidos (1960–2025) [5] 16
2. Conocimientos financieros generales por países [10] 17
3. Imagen de la frontera eficiente 23
4. Diagrama de Estructura de Descomposición del Trabajo del proyecto InvestFlow 29
5. Planificación temporal del proyecto InvestFlow mediante diagrama de Gantt 38
6. Resultado del test de perfil inversor de Indexa Capital 48
7. Cartera modelo propuesta para el perfil 4/10 49
8. Simulador de carteras de Finizens 50
9. Cuestionario de tolerancia al riesgo de eToro 51
10. Visualización de activo individual en eToro 51
11. Smart Portfolios disponibles en eToro 52
12. Balance de posiciones en un Smart Portfolio de eToro 52
13. Actores que participan en el sistema 58
14. Diagrama de Casos de Uso 58
15. Modelo Entidad-Relación 60
16. Diagrama de Base de Datos 64
17. Arquitectura cliente-servidor de la plataforma 66
18. Interfaz de registro 68
19. Mensaje de error en el registro 69
20. Mensaje de bienvenida tras registrarse 69
21. Interfaz de inicio de sesión 70
22. Mensaje de error al iniciar sesión 70
23. Mensaje de bienvenida al iniciar sesión 71
24. Interfaz recuperar contraseña 71

25. Mensaje de confirmación de envío del enlace 72
26. Enlace de recuperación 72
27. Interfaz para introducir nueva contraseña 73
28. Interfaz contraseña actualizada correctamente 73
29. Interfaz modificar datos personales 74
30. Mensaje de error al modificar datos 75
31. Interfaz de acción requerida para eliminar cuenta 76
32. Interfaz de cuenta eliminada correctamente 76
33. Interfaz de sección inicial página 1 77
34. Interfaz de sección inicial página 2 77
35. Interfaz de sección inicial página 3 77
36. Interfaz de sección inicial página 4 78
37. Sección aprende página principal 78
38. Interfaz de explicación del algoritmo 78
39. Interfaz de la sección mercado 79
40. Interfaz para filtrar los activos 79
41. Información sobre activos 80
42. Interfaz de generación de carteras 80
43. Carteras generadas por el algoritmo 81
44. Interfaz de la cartera de inversión generada 81
45. Interfaz de los portafolios generados 82
46. Rendimiento de la cartera 82
47. Balance de los activos 83
48. Balance de las posiciones 83
49. Rendimiento de la posición 84
50. Interfaz de backtesting 84
51. Resultados del backtesting 84
52. Historial de commits en GitHub 99

Lista de tablas

1. Reuniones con el tutor 30
2. Definición de objetivos 30
3. Definición de tareas (EDT) 30
4. Selección de herramientas 31
5. Investigación académica 31
6. Arquitectura Flask-Python 31
7. MySQL y gestión de base de datos 32
8. Búsqueda de APIs para el proyecto 32
9. Herramientas para el front-end 32
10. Definición de requisitos funcionales 32
11. Casos de uso para el proyecto 33
12. Diagrama Entidad-Relación 33
13. Diseño de la interfaz de usuario 33
14. Configuración del entorno de desarrollo 33
15. Implementación del algoritmo de Markowitz 34
16. Integración con APIs financieras 34
17. Gestión de usuarios y sesiones 34
18. Endpoints y comunicación Front-End/Back-End 35
19. Extensiones del algoritmo 35
20. Página de inicio 35
21. Registro e inicio de sesión 36
22. Listado de activos y gráficos 36
23. Generación de carteras 36
24. Mi portafolio y sección Aprende 36
25. Pruebas del algoritmo de la aplicación 37

26. Pruebas de la conexión con el front-end 37
27. Corrección de errores 37
28. Redacción de la memoria 38
29. Preparación de la defensa 38
30. Niveles de probabilidad de riesgos 41
31. Niveles de impacto de riesgos 41
32. Riesgo: Incumplimiento en la planificación temporal 41
33. Riesgo: Fallo del equipo personal 42
34. Riesgo: API de datos financieros no viable 42
35. Riesgo: Datos financieros incorrectos o desactualizados 42
36. Riesgo: Cambio de requisitos del proyecto 43
37. Riesgo: Vulnerabilidades de seguridad en la aplicación 43
38. Costes de personal del proyecto 44
39. Costes de hardware amortizados del proyecto 45
40. Costes de software del proyecto 45
41. Resumen del coste total del proyecto 46
42. Pruebas de registro para un usuario sin identificar 100
43. Pruebas del generador de carteras del S&P 500 101
44. Pruebas de gestión de portafolios y cuenta de usuario 102
45. Resultados de las pruebas de indicadores. 105
46. Resultados de las pruebas del módulo de backtesting. 105

Lista de acrónimos

TFG Trabajo Fin de Grado.

IA Inteligencia Artificial.

PIB Producto Interior Bruto.

FMI Fondo Monetario Internacional.

BCE Banco Central Europeo.

IPC Índice de Precios de Consumo.

CNMV Comisión Nacional del Mercado de Valores.

ODS Objetivos de Desarrollo Sostenible.

EDT Estructura de Descomposición del Trabajo.

RF Risk-Free Rate.

API Application Programming Interface.

FMP Financial Modeling Prep.

AOS Animate On Scroll.

HTTP Hypertext Transfer Protocol.

HTTPS Hypertext Transfer Protocol Secure.

HTML HyperText Markup Language.

CSS Cascading Style Sheets.

JSON JavaScript Object Notation.

AJAX Asynchronous JavaScript and XML.

CORS Cross-Origin Resource Sharing.

SMTP Simple Mail Transfer Protocol.

PBKDF2 Password-Based Key Derivation Function 2.

BD Base de Datos.

ER Entidad-Relación.

DOP Diagramas de Operaciones del Proceso.

OLS Ordinary Least Squares.

SLSQP Sequential Least Squares Programming.

GPL General Public License.

MIT Massachusetts Institute of Technology License.

BSD Berkeley Software Distribution License.

Glosario de Términos

A lo largo de esta memoria se utilizan términos tanto del ámbito financiero como del ámbito informático. Para aclarar las dudas que puedan tener los lectores no especializados, a continuación se definen distintos conceptos.

Quantitative easing Política monetaria expansiva basada en la compra de activos por parte de un banco central para aumentar la liquidez del sistema financiero.

Capitalización bursátil Se llama al valor total de mercado de una empresa, se calcula multiplicando el precio de sus acciones por el número de acciones que posee.

Fondo indexado Fondo de inversión que replica el comportamiento de un índice bursátil determinado.

Agregado monetario M2 es una medida de la cantidad de dinero disponible en la economía. M2 representa el dinero que puede convertirse rápidamente en efectivo o utilizarse para gastar.

Back-End Parte de una aplicación encargada de la lógica interna, la gestión de datos, la autenticación y la comunicación con la base de datos.

Front-End Parte visible de una aplicación con la que interactúa el usuario.

API Protocolos que permiten a las aplicaciones comunicarse entre sí.

Wireframe Esquema visual que representa la estructura de una interfaz antes del diseño final.

Outlier Desviación atípica, valores extremos que se alejan del conjunto total de datos.

Endpoint Es una ruta específica dentro de una API (Interfaz de Programación de Aplicaciones). Los endpoints permiten a las aplicaciones comunicarse entre sí.

Framework Es un marco de trabajo que proporciona herramientas y bibliotecas para desarrollar una aplicación.

Librería Conjunto de funciones reutilizables que son programadas previamente por programadores y facilitan el desarrollo de código.

Robo-advisor Plataforma digital que ofrece servicios automatizados de asesoramiento o gestión de inversiones.

Smart Portfolios Conjunto de activos que siguen una temática. Carteras de inversión específicas.

Web scraping Es una técnica de programación que recopila y extrae datos de una página web.

Algoritmo de *hash* Algoritmo matemático que transforma un conjunto de valores en nuevos datos con una longitud determinada.

Token Conjunto de caracteres que permiten al usuario acceder a distintas aplicaciones o sitios web.

Pop-up Ventana emergente que se superpone en la interfaz del usuario de una aplicación o sitio web.

Backtesting Proceso que se utiliza para evaluar una inversión aplicándola a datos históricos.

Blueprint Herramienta que sirve para definir los procesos en un servicio, desde los procesos visibles para el usuario final como los procesos internos.

Momentum Indicador que determina las tendencias del mercado, si el precio de un activo continuará subiendo o bajando.

1. Empleo de Inteligencia Artificial

A lo largo de la elaboración de este Trabajo de Fin de Grado se ha hecho uso de herramientas de Inteligencia Artificial (IA) como apoyo en algunas de las secciones del proyecto.

En primer lugar, se ha utilizado **GitHub Copilot** durante el desarrollo de la aplicación, con el objetivo de detectar y corregir errores en la implementación, y obtener sugerencias puntuales en partes del código [1]. A pesar del uso de la Inteligencia Artificial, el código se ha revisado, comprendido y adaptado antes de implementarse dentro de la plataforma realizada [2].

Por otro lado, se ha empleado **Claude** como apoyo para la búsqueda avanzada de información para el modelo de optimización, revisión y mejora de la redacción de determinados apartados técnicos de la memoria, y apoyo en la corrección de errores [3].

2. Introducción

En las últimas décadas, el sistema financiero mundial ha atravesado un cambio que ha alterado la capacidad de ahorro del ciudadano. La deuda pública global alcanzó en 2024 la cifra récord de 102 billones de dólares, representando el 92,3 % del Producto Interno Bruto (PIB) mundial según el Fondo Monetario Internacional (FMI) [4], además, se prevé que se acercará al 100 % del PIB global antes de que termine la década, niveles no vistos desde 1948.

Este escenario macroeconómico, determinado por una expansión monetaria que se acelera a medida que ocurren las crisis mundiales y el nivel de endeudamiento, hace que el ahorro tradicional deje de ser una estrategia viable para mantener el patrimonio. Tras la crisis financiera global de 2008, los principales bancos centrales pusieron en marcha programas masivos de compra de activos conocidos como *Quantitative easing* (QE) y mantuvieron los tipos de interés a cero. Como resultado de estas políticas el agregado monetario M2 de la Reserva Federal pasó de 8,2 billones de dólares en diciembre de 2008 a 21,5 billones en diciembre de 2024. El Banco Central Europeo (BCE), realizó un programa de compra de activos entre 2015 y 2018 inyectando 1,14 billones de euros equivalentes al 11,3 % del PIB nominal, con efectos al alza sobre la inflación [6]. La oferta o agregado monetario M2 se refiere a la cantidad total de dinero que circula en una economía en un momento dado. Esto incluye el efectivo (billetes y monedas) y los depósitos bancarios que pueden utilizarse fácilmente para realizar transacciones. Los bancos centrales son los principales responsables de gestionar la oferta monetaria a través de diversas herramientas de política monetaria. Por ejemplo, cuando el banco central compra bonos a los bancos comerciales, inyecta dinero en el sistema bancario, aumentando las reservas de los bancos y, por lo tanto, la oferta monetaria.

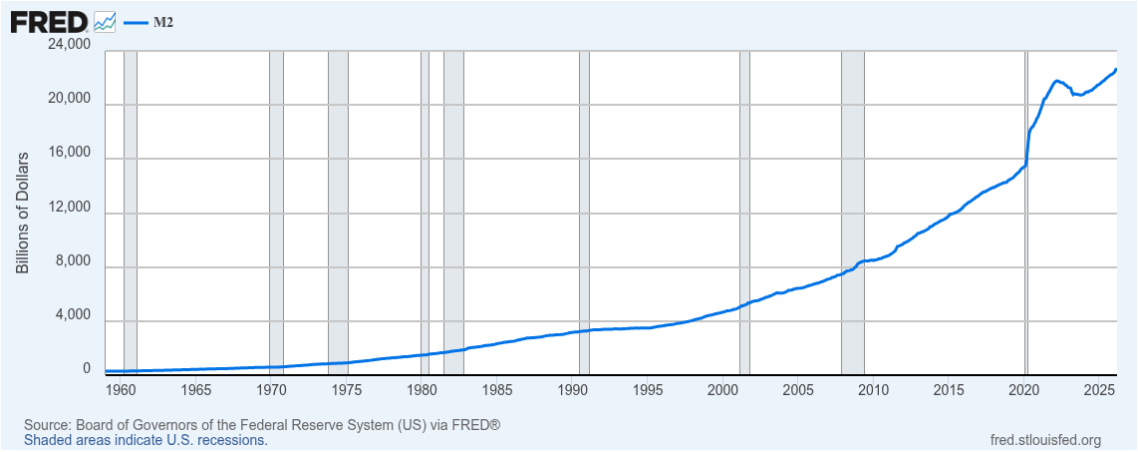


Figura 1: Evolución del agregado monetario M2 de la Reserva Federal de Estados Unidos (1960–2025) [5]

Las consecuencias de este exceso de liquidez se caracterizaron con mayor intensidad a partir de 2021. En España, el Índice de Precios de Consumo (IPC) alcanzó su máximo histórico desde 1985 en julio de 2022, con una variación del 10,8 % [7], mientras que los depósitos bancarios apenas remuneraban un 0,01 % anual [8]. Por cada 10.000 euros depositados en un banco, el ahorrador perdía 1.000 euros de poder adquisitivo en un solo año. El sistema financiero se había convertido en una penalización para quienes no invertían, por lo que la única alternativa para protegerse es destinar parte del capital a activos refugio, como el oro, el bitcoin o activos que históricamente han superado a la inflación, como las acciones en los principales índices bursátiles.

Sin embargo, acceder a esa alternativa no resulta sencillo para el inversor particular. La encuesta sobre Cultura Financiera de Funcas [9] revela que el 36 % de los españoles de entre 18 y 64 años reconoce no disponer de los conocimientos necesarios para tomar decisiones financieras adecuadas. Esta situación se confirma en el ámbito internacional: según el informe del Banco Santander [10], solo el 54 % de los españoles afirma manejar conceptos financieros generales, frente al 81 % en Estados Unidos o el 71 % en Reino Unido, situando a España en los últimos puestos del ranking mundial. La gran mayoría desconoce conceptos básicos como la diversificación, la relación rentabilidad-riesgo o el horizonte temporal de cada inversión.

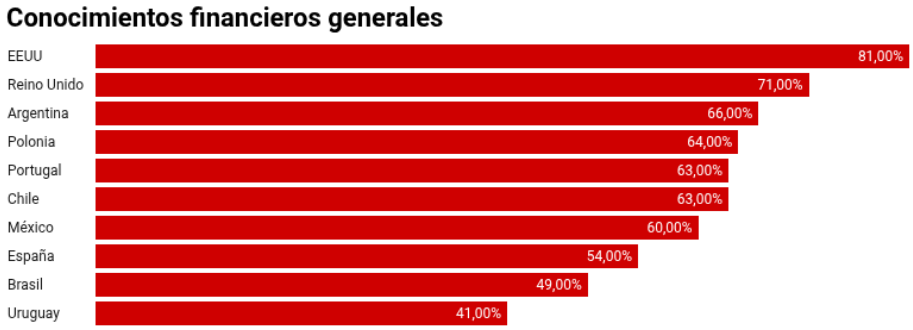


Figura 2: Conocimientos financieros generales por países [10]

2.1. Descripción del proyecto

Con ese problema de cara, este Trabajo de Fin de Grado (TFG) presenta el diseño e implementación de una aplicación web que facilita la creación de carteras bursátiles diversificadas, sin requerir conocimientos financieros avanzados. La aplicación dispone de una base de datos con los registros de los activos que componen el índice S&P 500, compuesto por las 500 empresas más representativas de la economía estadounidense, y emplea el modelo de optimización de Markowitz [11] para calcular, dado un perfil de riesgo y horizonte temporal, una distribución del capital entre dichos activos.

La plataforma ofrece dos opciones para poder generar las carteras. La primera optimiza cada uno de los sectores que compone el S&P 500 de forma independiente —tecnología, salud, energía, finanzas, consumo básico, consumo discrecional, industriales, materiales, servicios públicos, inmobiliario y comunicaciones—, permitiendo al usuario concentrarse en un sector concreto. La segunda modalidad, combina los activos con mejor relación rentabilidad-riesgo de todos los sectores en una única cartera multisector, lo que reduce la exposición a riesgos sectoriales específicos. En ambos casos, la interfaz muestra de forma visual el peso recomendado para cada posición, con el objetivo de que cualquier usuario, independientemente de su nivel de conocimiento, pueda comprender

y aplicar la estrategia que la aplicación propone.

2.2. Conceptos

Para calcular y estructurar una cartera bajo el modelo de Markowitz (también conocido como la Teoría Moderna de Carteras o análisis de varianza-media), es indispensable manejar una serie de conceptos estadísticos y financieros fundamentales.

2.2.1. El índice S&P 500

El índice S&P 500 es un índice en el mercado de valores donde se recogen las 500 empresas más grandes de Estados Unidos, las empresas con mayor capitalización de mercado. Es uno de los índices más seguidos por los inversores y por las instituciones financieras. [13]

Este índice es muy importante en el mercado de valores, ya que, debido a su gran capitalización bursátil, permite representar las tendencias de la economía mundial.

2.2.2. Sectores del S&P 500

Este índice abarca once sectores principales [14]:

- **1. Tecnologías de la información**: Empresas que distribuyen productos de tecnología.
- **2. Finanzas**: Empresas que se dedican a las finanzas y la inversión.
- **3. Atención médica**: Empresas que desarrollan productos farmacéuticos.
- **4. Consumo discrecional**: Empresas que desarrollan productos considerados de lujo.
- **5. Servicios de comunicación**: Empresas que se dedican a los servicios de comunicación, como proveedores de internet, medios de comunicación o empresas dedicadas al entretenimiento.
- **6. Industria**: Empresas como aerolíneas y constructoras.
- **7. Bienes de consumo básico**: Empresas dedicadas al desarrollo de productos para la vida diaria.
- **8. Energía**: Empresas petrolíferas o de gas.
- **9. Bienes raíces**: Empresas inmobiliarias o constructoras de viviendas.
- **10. Materiales**: Empresas que suministran materias primas como cobre, plata u oro.
- **11. Servicios públicos**: Empresas que suministran agua, electricidad o algún componente necesario para la vida diaria a personas o a otras empresas.

2.2.3. James-Stein

El modelo de Markowitz requiere estimar la rentabilidad esperada de cada activo. El método más directo es utilizar la media histórica de sus rendimientos, pero esta estimación contiene mucho ruido: un activo que subió mucho en el pasado no tiene por qué seguir haciéndolo, y Markowitz tiende a sobreinvertir en activos que parecen muy rentables solo por ese ruido estadístico.

Para corregirlo, el estimador de James-Stein contrae cada rentabilidad esperada hacia la media del grupo, moderando los valores extremos [15]:

$$\hat{\mu}_i = (1 - \delta)\,\mu_i + \delta\,\bar{\mu} \tag{2.1}$$

donde μ_i es la rentabilidad histórica del activo i, $\bar{\mu}$ es la media de todos los activos del sector, y $\delta \in (0{,}1)$ es la intensidad de contracción.

En este proyecto se aplica con $\delta = 0{,}30$, es decir, los retornos esperados se desplazan un 30 % hacia la media sectorial antes de ejecutar la optimización. Esto reduce la sensibilidad del optimizador al ruido histórico y genera carteras más diversificadas y estables.

2.2.4. Retorno esperado de la cartera

El retorno esperado de la cartera es el beneficio futuro que se estima obtener combinando los distintos retornos de cada activo y ponderándolos por su peso en la cartera. Para estimar el beneficio futuro de la cartera, se van a realizar los siguientes pasos:

Paso 1: Se calcula el Retorno logarítmico diario por activo: De todos los precios históricos pasados obtenidos, se calcula el Retorno logarítmico diario de cada activo i:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) \tag{2.2}$$

donde P_t es el precio de cierre en el día t y P_{t-1} es el precio del día anterior.

Se calcula la media diaria (promedio de todos los días) donde N son los precios históricos obtenidos

$$\bar{r} = \frac{1}{N}\sum_{t=1}^{N} r_t \tag{2.3}$$

Por ejemplo, si la cartera contiene tres activos A, B y C, sus medias diarias podrían ser: $\bar{r}_A = 0{,}00050$, $\bar{r}_B = 0{,}00030$ y $\bar{r}_C = 0{,}00070$.

Paso 2: Se obtiene la media diaria a anual, multiplicando por 252 (media de días activos en bolsa al año):

$$\mu_i^{\text{hist}} = \bar{r}_i \times 252 \tag{2.4}$$

Aplicando esta fórmula:

- $\mu_A^{\text{hist}} = 0{,}1260$ (12,60 %)
- $\mu_B^{\text{hist}} = 0{,}0756$ (7,56 %)
- $\mu_C^{\text{hist}} = 0{,}1764$ (17,64 %)

Paso 3: Se obtiene la media transversal:

Se calcula la media de los retornos actualizados de todos los activos del sector, es decir, la media global μ^{grand}:

$$\mu^{\text{grand}} = \frac{1}{n}\sum_{i=1}^{n} \mu_i^{\text{hist}} = \frac{0{,}1260 + 0{,}0756 + 0{,}1764}{3} = 0{,}1260 \tag{2.5}$$

Paso 4: Se obtiene *shrinkage* de *James-Stein*.

El estimador de James-Stein contrae cada retorno individual hacia la media global con $\alpha = 0{,}30$:

$$\hat{\mu}_i = (1-\alpha)\,\mu_i^{\text{hist}} + \alpha\,\mu^{\text{grand}} \tag{2.6}$$

Aplicado a los tres activos del ejemplo:

$$\hat{\mu}_A = 0{,}1260, \quad \hat{\mu}_B = 0{,}09072, \quad \hat{\mu}_C = 0{,}16128$$

Paso 5: Retorno esperado de la cartera:

Una vez aplicado el *shrinkage*, el retorno esperado de la cartera se obtiene como la suma ponderada de los retornos esperados de cada activo:

$$R_p = \sum_i w_i\,\hat{\mu}_i = \mathbf{w}^\top \hat{\boldsymbol{\mu}} \tag{2.7}$$

Si el optimizador asigna los siguientes pesos:

$w_A = 0{,}50$, $w_B = 0{,}30$ y $w_C = 0{,}20$

El retorno esperado de la cartera es:

$$R_p = 0{,}50 \times 0{,}1260 + 0{,}30 \times 0{,}09072 + 0{,}20 \times 0{,}16128 = 0{,}1225 \tag{2.8}$$

2.2.5. El ratio de Sharpe

Un ratio de Sharpe indica el nivel de rentabilidad de una inversión respecto a cuánto riesgo conlleva. Cuanto más alto sea, más atractiva se convierte la acción, ya que está generando más rentabilidad respecto al riesgo total. Cuando el Ratio de Sharpe es negativo, indica que el riesgo asumido no está siendo recompensado.

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p} \tag{2.9}$$

- R_p **(Rentabilidad de la cartera)**: es la rentabilidad media que se espera obtener con la combinación de activos elegida.
- R_f **(Tasa libre de riesgo)**: es la rentabilidad de una inversión prácticamente sin riesgo (como la deuda pública), y se utiliza como referencia para comparar.
- σ_p **(Volatilidad de la cartera)**: es la desviación típica de los rendimientos de la cartera, y representa el riesgo que se asume.

Por lo tanto, el numerador ($R_p - R_f$) representa el exceso de rentabilidad: lo que se gana por encima de una inversión sin riesgo.

La tasa libre de riesgo `risk_free_rate` se obtiene del factor *Risk-Free Rate* (RF) de Fama-French (Sección 2.2.9).

2.2.6. La matriz de covarianza muestral

La covarianza mide cómo varían conjuntamente dos variables. Si la covarianza es positiva, cuando una variable aumenta, la otra tiende a aumentar también. Si la covarianza es negativa, cuando una variable aumenta, la otra tiende a disminuir. Si la covarianza es cercana a cero, no hay una relación lineal clara entre ambas variables. La matriz de covarianza es un componente central del modelo de Markowitz, ya que mide cómo varían entre sí los rendimientos de los activos. Es el concepto central para lograr la diversificación, ya que evalúa la interacción matemática entre los activos para minimizar la volatilidad global de la inversión. Sin embargo, cuando el número de activos es elevado respecto al número de días de datos disponibles, la matriz de covarianza muestral no tiene suficiente información para estimar bien todas las relaciones entre activos, acumulando un alto nivel de error. Este error tiende a concentrarse en los coeficientes más extremos de la matriz, precisamente los que más influyen en la optimización [17, 18].

2.2.6.1. Ledoit-Wolf

Para corregir este problema, Ledoit y Wolf propusieron una técnica denominada *shrinkage*, que combina la matriz de covarianza muestral S con una matriz objetivo más estable F:

$$\hat{\Sigma} = \delta F + (1 - \delta)S \tag{2.10}$$

donde $\delta \in (0,1)$ es el coeficiente de contracción, calculado de forma óptima para minimizar el error de estimación. El resultado es una matriz mejor condicionada y más robusta para su uso en la optimización de carteras [17].

En el proyecto, Ledoit-Wolf se aplica para calcular la matriz de covarianza de los activos seleccionados antes de ejecutar la optimización de Markowitz. El coeficiente δ se determina de forma automática. El resultado es una matriz de covarianza más estable que la muestral, lo que permite que el optimizador genere carteras más robustas y menos sensibles al ruido de los datos históricos.

2.2.7. Riesgo (Volatilidad, Varianza o Desviación Estándar)

En el modelo de Markowitz, el riesgo se identifica directamente con la volatilidad de los rendimientos. Para medirlo, se emplean la varianza o la desviación estándar anualizada, las cuales miden el grado de dispersión matemática de los retornos de un activo. El cálculo no consiste simplemente en sumar o promediar los riesgos individuales de cada activo, sino que evalúa cómo interactúan esos activos entre sí matemáticamente.

Como se ha explicado en la sección anterior, el cálculo del riesgo parte de la matriz de covarianzas. En este proyecto se utiliza Ledoit-Wolf *shrinkage* para obtener una matriz más estable antes de calcular la volatilidad de la cartera [17, 18].

El cálculo tiene dos fases separadas: primero construir la matriz de covarianzas y luego calcular la volatilidad de cartera con esa matriz.

Fase 1 — Matriz de covarianzas (Ledoit-Wolf).

La matriz utilizada por el algoritmo se obtiene aproximando la matriz de covarianza muestral hacia una estructura más simple. De esta forma, se evita que valores extremos de los datos históricos tengan demasiado peso en el resultado final:

$$\hat{\boldsymbol{\Sigma}}^{LW} = (1-\delta)\,\hat{\boldsymbol{\Sigma}}^{\text{muestral}} + \delta\,\mu\mathbf{I} \tag{2.11}$$

$\hat{\boldsymbol{\Sigma}}^{\text{muestral}}$ representa la matriz de covarianza calculada directamente a partir de los retornos históricos, $\mathbf{I}$ es la matriz identidad, μ es un valor que mantiene la escala de las varianzas y δ indica cuánto se desplaza la matriz muestral hacia esa estructura más simple. Este valor δ, conocido como *shrinkage intensity*, se calcula automáticamente.

Fase 2 — Volatilidad de la cartera.

Una vez obtenida la matriz de covarianzas anualizada $\boldsymbol{\Sigma}$, se calcula el riesgo total de la cartera teniendo en cuenta tanto el peso de cada activo como la relación entre ellos. Con los pesos óptimos $\mathbf{w}$, la volatilidad anual de la cartera se calcula como:

$$\sigma_p = \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w}} \tag{2.12}$$

Esta fórmula permite obtener una única medida de riesgo para toda la cartera. Si los activos se mueven de forma parecida, la covarianza aumenta el riesgo conjunto; en cambio, si los activos se comportan de forma diferente, la diversificación puede reducir la volatilidad total.

2.2.8. El modelo de optimización de Markowitz

El modelo de optimización de Markowitz se basa en la diversificación de activos en una cartera para obtener la máxima rentabilidad y reducir el riesgo de la inversión. Harry Markowitz, el creador del algoritmo, destacó que la diversificación no es suficiente para optimizar una cartera, sino que también se debe tener en cuenta la varianza media.

Para ello, el algoritmo se centra en la correlación entre activos: dos activos poco correlacionados entre sí reducen el riesgo total de la cartera de inversión.

El algoritmo se basa en distintos pasos:

- **Conjunto de posibilidades de la inversión**: Se analizan n activos y las posibles carteras. Cada cartera se representa como un vector de pesos $\mathbf{w} \in \mathbb{R}^n$, donde w_i indica la fracción del capital invertida en el activo i, sujeto a:

$$\sum_{i=1}^{n} w_i = 1, \quad w_i \geq 0 \tag{2.13}$$

  A partir de los retornos esperados $\boldsymbol{\mu}$ y la matriz de covarianza $\boldsymbol{\Sigma}$, el retorno y el riesgo de la cartera se estiman como:

$$R_p = \boldsymbol{\mu}^\top \mathbf{w}, \qquad \sigma_p = \sqrt{\mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w}} \tag{2.14}$$

  El objetivo es minimizar la varianza para un nivel de retorno dado [11, 12]:

$$\min_{\mathbf{w}} \ \mathbf{w}^\top \boldsymbol{\Sigma}\, \mathbf{w} \quad \text{sujeto a} \quad \boldsymbol{\mu}^\top \mathbf{w} \geq R^*, \quad \sum_{i=1}^{n} w_i = 1, \quad w_i \geq 0 \tag{2.15}$$

- **Frontera eficiente** Una cartera será eficiente, si para su nivel de rendimiento, no existe otra cartera con menor riesgo, y con riesgo que esa cartera conlleva, ofrece la mayor rentabilidad esperada.

  La frontera eficiente es el conjunto de carteras que cumplen ambas condiciones. Se representa en un gráfico donde el eje Y es rentabilidad y el riesgo en el eje X:

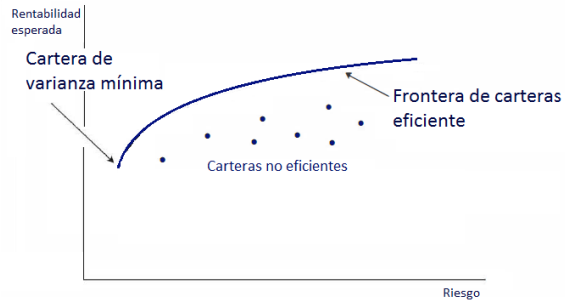


Figura 3: Imagen de la frontera eficiente

  Un inversor que no asume mucho riesgo elegiría una cartera en la zona inferior izquierda de la frontera. Un inversor que busca mayor rentabilidad elegiría una cartera en la zona superior derecha.

- **Cartera óptima:** La cartera óptima es el punto de tangencia entre la frontera eficiente y las preferencias del inversor. Es única para cada inversor, ya que depende del nivel de riesgo que está dispuesto a asumir. En la aplicación, la cartera óptima es que ofrece la mayor rentabilidad por unidad de riesgo asumido, es decir, mayor ratio de Sharpe. [11, 12]

2.2.9. El modelo Fama-French

En este proyecto, el modelo de Fama-French no se usa para decidir qué activos entran en la cartera, sino para ajustar los pesos finales ya calculados: sobreponderar acciones que están a favor de la tendencia del mercado, y subponderar a las que están en contra.

El modelo dice que el retorno de una acción no depende de si el mercado sube o baja, sino más características, que son los cinco factores:

- **Mercado (MktRF)**: si el mercado en general sube o baja.
- **Tamaño (SMB)**: si las empresas pequeñas lo están haciendo mejor que las grandes.
- **Valor (HML)**: si las empresas con menos capitalización lo están haciendo mejor que las de mayor capitalización.
- **Rentabilidad (RMW)**: si las empresas muy rentables lo están haciendo mejor que las poco rentables.
- **Inversión (CMA)**: si las empresas que invierten poco lo están haciendo mejor que las que invierten mucho.

En el proyecto se usa de la siguiente manera:

1. **Mirar la tendencia de cada factor**: para cada uno de los 5 factores, se comprueba si en el último mes ha ido bien (+1) o mal (-1).
2. **Mirar cuánto le afecta cada factor a cada acción**: para cada acción de la cartera, se calcula cuánto depende su precio históricamente de cada uno de esos 5 factores (algunas acciones dependen mucho del factor "tamaño", otras del factor "valor", etc.).
3. **Recorrer las acciones y calcular un *score***: para cada acción, se recorren sus 5 factores y se suma: si un factor va bien (+1) y la acción depende de él, suma; si un factor va mal (-1) y la acción depende de él, resta. El resultado es un único número por acción: cuanto más alto, más a favor de la tendencia está esa acción ahora mismo.
4. **Convertir el *score* en un porcentaje**: se recorren las acciones otra vez y se transforma cada *score* en un porcentaje entre $-8\,\%$ y $+8\,\%$. La acción con el mejor *score* se lleva el $+8\,\%$, la del peor *score* el $-8\,\%$, y el resto algo intermedio según lo cerca o lejos que esté de esos extremos. El límite de $8\,\%$ es un parámetro de diseño ajustable: valores más altos dan más peso a la señal de factores sobre la optimización de Markowitz, y valores más bajos la reducen.
5. **Aplicar ese porcentaje al peso**: se recorren los pesos que ya había calculado el algoritmo, y a cada uno se le aplica su porcentaje correspondiente: si a una acción le tocó $+8\,\%$, su peso sube un 8 % (por ejemplo, de 10 % pasa a 10,8 %); si le tocó $-8\,\%$, baja un 8 % (de 10 % a 9,2 %).
6. **Reajustar para que todo siga sumando 100 %**: como al subir unos pesos y bajar otros ya no suman 100 %, se reparten de nuevo de forma proporcional.

2.3. Razones de la elección del TFG

A la hora de escoger un TFG u otro, era indispensable para mí que fuese útil y que pudiese ser una herramienta de ayuda para la gente. Desde hace años, el mundo de la inversión ha despertado mi interés de forma autodidacta, al entender que el dinero parado pierde valor año tras año y que el ciudadano de a pie debe protegerse de los gobernantes del sistema. Comprender esto me llevó a querer aprender cómo poder gestionar patrimonio mediante la inversión en activos.

Por otro lado, también me atrajo del proyecto es la matemática que hay detrás. El modelo de Markowitz es un problema de optimización cuadrática con grandes fundamentos matemáticos. Además, el proceso de automatización para que el sistema ejecute de forma repetida y consistente me apasiona dentro de la rama de la ingeniería informática.

Por último, quería construir algo que fuese útil para aquellas personas sin conocimientos técnicos, la idea de que cualquier usuario pueda obtener una cartera rentable sin saber nada de finanzas es una de las razones más importantes. A nivel profesional, este TFG también indica una orientación clara hacia el sector financiero. Me gustaría enfocar mi carrera en áreas como el análisis de datos aplicado a la banca, la ingeniería del negocio o cualquier rol que combine la tecnología con el mundo de las inversiones.

3. Enfoque Inicial

Antes de comenzar el proyecto, es necesaria una planificación inicial que recoja todos los objetivos a alcanzar, así como una planificación tanto de tareas como de tiempo. Esto es imprescindible para:

- Obtener una visión clara de los aspectos a realizar.
- Poder alcanzar los requisitos necesarios en el plazo de tiempo establecido.
- Estructurar correctamente todos los apartados del proyecto, de forma que cualquier persona que se incorpore al equipo sea capaz de comprenderlo.
- Asegurar que las funcionalidades implementadas corresponden con las necesidades del usuario.

3.1. Conexión con los Objetivos de Desarrollo Sostenible y los principios y valores democráticos (ODS)

Este apartado se enfoca en cómo el trabajo de fin de grado puede impactar con los objetivos de la Agenda 2030 de Naciones Unidas.

3.1.1. ODS 4 - Educación de calidad

La aplicación incluye un apartado donde indica el algoritmo de Markowitz y los conceptos básicos de inversión, principalmente este apartado está dirigido a un público sin conocimientos previos. El objetivo es que el usuario sepa cómo y por qué debe invertir en dichos activos, y una vez ejecutado el algoritmo por qué esa cartera tendría sentido. Esto está vinculado con el objetivo 4.4 de la Agenda 2030, fomentar el conocimiento de la ciudadanía para tomar decisiones financieras [21].

3.1.2. ODS - 8 Trabajo decente y crecimiento económico

La aplicación también está relacionada con la meta 8.10, que busca ampliar el acceso de las personas a servicios financieros [21]. La optimización de carteras ha sido utilizada durante años por gestoras profesionales o a personas con posibilidad de permitirse pagar un asesor financiero. InvestFlow pone ese mismo desarrollo al alcance de todos los públicos de manera gratuita. Un objetivo claro es facilitar la inversión para protegerse de la inflación.

3.1.3. ODS 9 - Industria, innovación e infraestructura

InvestFlow utiliza técnicas de computación matemática -optimización cuadrática, integración con APIs- sobre una página web de fácil uso. La aplicación está vinculada con las metas 9.5 que fomentan el uso de la tecnología para poder hacer frente a problemas específicos [21]. En este caso, la aplicación convierte un algoritmo matemático complejo en algo usable para todo público.

3.1.4. ODS 10 - Reducción de las desigualdades

Según los datos del Banco de España y la Comisión Nacional del Mercado de Valores (CNMV) [22], España se sitúa entre los países con menor nivel de cultura financiera, y esa desigualdad afecta especialmente a personas con ingresos más bajos, que son precisamente las que necesitan protegerse frente a la poca capacidad de ahorrar que tienen. Esta aplicación gratuita y accesible contribuye a reducir esa brecha.

3.1.5. ODS 17 - Alianzas para lograr los objetivos

Por último, el proyecto al ser de código abierto y con datos públicos, se vincula con la colaboración entre las personas y transferencia de concomientos técnicos y financieros. Este proyecto está en línea con la meta 17.8 sobre el uso de la tecnología para el desarrollo [21].

3.2. Objetivos

El objetivo principal de este trabajo es el desarrollo de una aplicación web que permita llevar a cabo la generación de carteras de inversión bursátiles del índice S&P 500. Dicho índice ha demostrado su rentabilidad a lo largo del tiempo, y es el elemento perfecto para protegerse de la degradación monetaria. En este caso, la aplicación dispone de un análisis exhaustivo de los activos que componen este índice, con el objetivo de transmitir la mayor información para el usuario. La aplicación genera carteras de inversión en tan solo dos clics. Para ello, se quiere ofrecer un diseño intuitivo, limpio y de fácil uso.

Para cumplir los requisitos del proyecto será necesario cumplir los siguientes objetivos:

- Desarrollar una aplicación web que:
    - Permita el seguimiento exhaustivo de los activos del S&P 500, ofreciendo información financiera y con gráficas para ver el balance del precio.
    - Que permita mejorar el conocimiento promedio del usuario.

La aplicación deberá contar con características como:

- **Inicio:** Se podrá ver información relevante acerca de la app y guiar al usuario al uso de esta.
- **Inicio sesión/Registro:** Cada usuario tendrá su propia información. Es importante que los correos electrónicos y los nombres de usuario no se dupliquen.

- **Mercado:** Una vez registrado se podrán ver los activos que monitoriza la aplicación. Además, se podrá ver el progreso de los precios de cada activo a lo largo del tiempo, y su correspondiente información empresarial.
- **Aprende:** El usuario podrá informarse del algoritmo utilizado en la aplicación con el objetivo de ayudar a entender el funcionamiento.
- **S&P500:** La sección donde el usuario genera la cartera de inversión. Donde el algoritmo se ejecuta y genera las posiciones que correspondan en función de las opciones que haya elegido el usuario.
- **Mi portafolio:** En esta sección se podrá ver las posiciones que ha realizado el usuario en la aplicación. Además, dispone de gráficos donde proporciona el balance de pérdidas y ganancias.
- **Usuario:** El usuario podrá modificar su información personal, incluyendo el nombre de usuario, nombre, apellidos y correo electrónico.

3.2.1. Objetivos técnicos

- Implementar el algoritmo de optimización de Markowitz, con el objetivo de resolver el problema de minimizar la varianza para una rentabilidad esperada, mediante optimización cuadrática con `scipy.optimize`
- Desarrollar el Back-End completo de la aplicación Flask, gestionando todos los endpoints y la comunicación con el Front-End y la base de datos.
- Diseñar e implementar una base de datos en MySQL que almacene los datos necesarios para el correcto funcionamiento.
- Integrar APIs externas que permitan obtener los datos financieros - Yahoo Finance mediante `yfinance`.
- Desarrollar un Front-End intuitivo que permita visualizar gráficos y datos de forma clara para el usuario.
- Ayudarse de aplicaciones de control de versiones como Git, para estructurar bien las fases del desarrollo del proyecto.

3.2.2. Objetivos sociales

- Dar acceso a cualquier persona para que pueda ayudarse a gestionar su dinero, permitiendo al público enseñar técnicas para realizar inversiones.
- Fomentar el conocimiento sobre el dinero y las finanzas.
- Ofrecer una aplicación gratuita que apoye a los usuarios a proteger su dinero.

3.2.3. Objetivos personales

- Como objetivo personal se espera profundizar en el modelo matemático de Markowitz y en los fundamentos de la gestión de carteras.
- Obtener experiencia real en el desarrollo de una aplicación web, integrando Back-End, base de datos y Front-End.
- Enfocar la formación hacia el mundo *fintech* y el análisis de datos financieros, dos áreas en las que se quiere orientar la carrera profesional.

3.3. Alcance

Esta sección se va a desglosar las tareas del proyecto.

3.3.1. Estructura de Descomposición del Trabajo (EDT)

El proyecto se ha dividido en 7 fases de trabajo definidas en el EDT, cada una con sus correspondientes apartados:



Figura 4: Diagrama de Estructura de Descomposición del Trabajo del proyecto InvestFlow

1. **Gestión y planificación:** En esta sección se incluyen todas las tareas principales para organizar el proyecto: reuniones periódicas con el tutor, definición de objetivos y selección de herramientas a utilizar.
2. **Formación y aprendizaje:** Este módulo es necesario para conocer el tiempo que supone adquirir los conceptos técnicos para la elaboración del proyecto. Además, se deberá conocer el tiempo que se deberá dedicar para obtener la información correspondiente.
3. **Análisis y diseño:** Para este apartado se definirán correctamente los requisitos funcionales de la aplicación, los casos de uso, el diseño de la base de datos y la interfaz de usuario del sistema.
4. **Desarrollo Back-End:** Es el módulo con más peso en el proyecto. En esta sección es donde se implementan todos los conceptos adquiridos para que la aplicación funcione. Incluye la implementación del algoritmo de optimización, la integración con APIs, la gestión de los usuarios y los datos, la implementación de los *endpoints* de comunicación con el Front-End y las extensiones del algoritmo.

5. **Desarrollo Front-End:** Este apartado sirve para desarrollar todas las vistas con las que el usuario interactuará.

6. **Pruebas:** Recoge todas las pruebas realizadas sobre el algoritmo, la base de datos. Además, recoge también las pruebas de integración entre el Front-End y Back-End y la corrección de los errores detectados durante el proceso.

7. **Documentación:** En esta tarea se incluye la redacción del proyecto completo y la preparación de la defensa final.

3.3.1.1. Módulo 1: Gestión y planificación

1.1 Reuniones con el tutor	
Descripción	Reunirse periódicamente con el tutor del proyecto para resolver dudas y recibir orientación sobre el contenido y decisiones técnicas.
Entregables	Avances mostrados al tutor y decisiones tomadas en cada reunión.
Herramientas	—
Precedencias	—
Horas estimadas	12 horas
Horas reales	12 horas

Tabla 1: Reuniones con el tutor

1.2 Definición de objetivos	
Descripción	Definir los objetivos principales del proyecto, así como las funcionalidades necesarias de la aplicación. Cerciorarse de un resultado exitoso.
Entregables	Objetivos del proyecto.
Herramientas	Navegador web y Overleaf.
Precedencias	—
Horas estimadas	2 horas
Horas reales	2 horas

Tabla 2: Definición de objetivos

1.3 Definición de tareas (EDT)	
Descripción	Identificar y estructurar todas las tareas del proyecto.
Entregables	Diagrama EDT y listado de subtareas.
Herramientas	Gloomaps, Overleaf.
Precedencias	1.2
Horas estimadas	2 horas
Horas reales	2.5 horas

Tabla 3: Definición de tareas (EDT)

1.4 Selección de herramientas	
Descripción	Identificar las mejores herramientas para poder llevar a cabo el proyecto, búsqueda a través de navegador web y en base a las reuniones del profesor.
Entregables	Listado de herramientas seleccionadas.
Herramientas	Navegador web.
Precedencias	—
Horas estimadas	1 hora
Horas reales	1 hora

Tabla 4: Selección de herramientas

3.3.1.2. Módulo 2: Formación y aprendizaje

2.1 Investigación académica del modelo	
Descripción	Ayudarse de documentación recibida por el tutor de las prácticas. Estudiar y obtener los conocimientos técnicos necesarios para la computación científica que requiere a través de repositorios en GitHub y artículos académicos.
Entregables	Conocimiento del modelo y sus implicaciones para el desarrollo.
Herramientas	Navegador web, artículos académicos, repositorios de GitHub.
Precedencias	—
Horas estimadas	14 horas
Horas reales	16 horas

Tabla 5: Investigación académica

2.2 Arquitectura Flask-Python	
Descripción	Recordar el funcionamiento del *framework* Flask a través de documentos de la universidad y trabajos realizados durante el grado.
Entregables	Conocimientos de Flask aplicados al proyecto.
Herramientas	Navegador web, documentación eGela.
Precedencias	—
Horas estimadas	6 horas
Horas reales	5 horas

Tabla 6: Arquitectura Flask-Python

2.3 MySQL y gestión de base de datos	
Descripción	Recordar la arquitectura Entidad-Relación (ER) de la base de datos con MySQL a través de documentos proporcionados durante el grado. Aprender a gestionar la base de datos incluyendo las consultas y tablas.
Entregables	Conocimientos de MySQL.
Herramientas	MySQL Workbench, navegador web.
Precedencias	—
Horas estimadas	5 horas
Horas reales	4.5 horas

Tabla 7: MySQL y gestión de base de datos

2.4 Búsqueda de APIs para el proyecto	
Descripción	Investigar las APIs financieras disponibles para obtener datos históricos de precios.
Entregables	Selección de APIs: yfinance.
Herramientas	Navegador web.
Precedencias	2.1
Horas estimadas	2 horas
Horas reales	2 horas

Tabla 8: Búsqueda de APIs para el proyecto

2.5 Herramientas para el front-end	
Descripción	Aprender el uso de Bootstrap 5, Chart.js, SweetAlert2 y Animate On Scroll (AOS) para el desarrollo de la interfaz de usuario.
Entregables	Conocimientos de las librerías de front-end.
Herramientas	Navegador web.
Precedencias	—
Horas estimadas	4 horas
Horas reales	5 horas

Tabla 9: Herramientas para el front-end

3.3.1.3. Módulo 3: Análisis y diseño

3.1 Definición de requisitos funcionales	
Descripción	Identificar qué debe hacer el sistema, qué secciones tendrá y qué necesidades debe cubrir para el usuario final.
Entregables	Listado de requisitos funcionales.
Herramientas	Overleaf, navegador web.
Precedencias	1.2
Horas estimadas	5 horas
Horas reales	6 horas

Tabla 10: Definición de requisitos funcionales

3.2 Casos de uso para el proyecto	
Descripción	Representar mediante el diagrama de casos de uso a todos los actores que requiere el sistema. Definir los casos de uso principales
Entregables	Diagrama de casos de uso.
Herramientas	Visual Paradigm, Overleaf.
Precedencias	3.1
Horas estimadas	4 horas
Horas reales	4 horas

Tabla 11: Casos de uso para el proyecto

3.3 Diagrama Entidad-Relación y arquitectura web	
Descripción	Diseñar el modelo de la base de datos, qué relaciones y atributos tendrá cada tabla de la base de datos. Además, diseñar la arquitectura web de la aplicación, mediante modelo cliente-servidor.
Entregables	Diagrama Entidad-Relación de la base de datos y diagrama de arquitectura web.
Herramientas	MySQL Workbench, draw.io.
Precedencias	3.1
Horas estimadas	5 horas
Horas reales	5 horas

Tabla 12: Diagrama Entidad-Relación

3.4 Diseño de la interfaz de usuario	
Descripción	Elaborar los *wireframes* de las principales vistas de la aplicación.
Entregables	*Wireframes* de las vistas principales.
Herramientas	Navegador web.
Precedencias	3.1
Horas estimadas	7 horas
Horas reales	9 horas

Tabla 13: Diseño de la interfaz de usuario

3.3.1.4. Módulo 4: Desarrollo Back-End

4.1 Configuración del entorno de desarrollo	
Descripción	Configurar el entorno de Python, instalar las dependencias necesarias y conectar la aplicación con la base de datos MySQL.
Entregables	Entorno de desarrollo funcional y estructura base del proyecto.
Herramientas	VS Code, Ubuntu 24.04, MySQL.
Precedencias	2.2, 2.3
Horas estimadas	2 horas
Horas reales	2 horas

Tabla 14: Configuración del entorno de desarrollo

4.2 Implementación del algoritmo de Markowitz	
Descripción	Implementar el algoritmo de optimización de Markowitz con estimación de la matriz de covarianza, incluyendo el cálculo de la frontera eficiente, la cartera de mínima varianza y la cartera de máximo *Sharpe ratio*. Añadir el método Ledoit-Wolf para quitar el ruido en datos atípicos u *outliers*.
Entregables	Módulo Python del algoritmo de optimización.
Herramientas	VS Code, NumPy,scikit-learn, SciPy, pandas, Bash.
Precedencias	2.1, 4.1
Horas estimadas	25 horas
Horas reales	28 horas

Tabla 15: Implementación del algoritmo de Markowitz

4.3 Integración con APIs financieras	
Descripción	Implementar la descarga y almacenamiento de precios históricos e información empresarial de los activos del S&P 500 mediante `yfinance` y `requests`. Para los *tickers* de la base de datos, registrarse en Financial Modeling Prep (FMP) para obtener *API Key* y añadirlo al proyecto.
Entregables	Módulo con APIs y base de datos poblada con datos históricos.
Herramientas	VS Code, yfinance, requests, Bash.
Precedencias	2.4, 4.1
Horas estimadas	5 horas
Horas reales	4 horas

Tabla 16: Integración con APIs financieras

4.4 Gestión de usuarios y sesiones	
Descripción	Implementar el sistema de registro, inicio de sesión y cierre de sesión de usuarios, incluyendo validaciones de correo y nombre de usuario, *hash* de contraseñas y gestión de sesiones con Flask.
Entregables	Sistema de autenticación funcional.
Herramientas	VS Code, Flask, MySQL, Bash.
Precedencias	4.1
Horas estimadas	10 horas
Horas reales	10 horas

Tabla 17: Gestión de usuarios y sesiones

4.5 Endpoints y comunicación Front-End/Back-End	
Descripción	Definir e implementar todos los endpoints de la aplicación Flask que sirven datos al Front-End: precios de activos, resultados del algoritmo, portafolios del usuario y datos de empresa.
Entregables	Conjunto de *endpoints* REST funcionales.
Herramientas	VS Code, Flask, curl, navegador web, Bash.
Precedencias	4.2, 4.3, 4.4
Horas estimadas	14 horas
Horas reales	16 horas

Tabla 18: Endpoints y comunicación Front-End/Back-End

4.6 Extensiones del algoritmo	
Descripción	Implementar las extensiones para un mejor funcionamiento del algoritmo: señal de selección por máximo de 52 semanas (*52-week High*), restricción de beta de mercado para controlar el riesgo y factor *Momentum Overlay* para filtrar activos con tendencia negativa. Mejorar el filtro del algoritmo para escoger los activos que más se adecuen a las circunstancias del mercado en el momento de realizar carteras de inversión.
Entregables	Módulo de extensiones integrado en el algoritmo principal.
Herramientas	VS Code, NumPy, pandas, SciPy, Bash.
Precedencias	4.2
Horas estimadas	16 horas
Horas reales	18 horas

Tabla 19: Extensiones del algoritmo

3.3.1.5. Módulo 5: Desarrollo Front-End

5.1 Página de inicio	
Descripción	Desarrollar la página de inicio con información sobre la aplicación, añadir animaciones de entrada con AOS y enlaces hacia los distintos endpoints que la aplicación tendrá.
Entregables	Página de inicio funcional.
Herramientas	VS Code, Bootstrap 5, SweetAlert2, Font Awesome.
Precedencias	2.5, 4.5
Horas estimadas	5 horas
Horas reales	6 horas

Tabla 20: Página de inicio

5.2 Registro e inicio de sesión	
Descripción	Desarrollar los formularios de registro e inicio de sesión con validaciones, adición de *pop-ups* mediante SweetAlert2 y redirección tras el formulario realizado.
Entregables	Vistas de registro e inicio de sesión funcionales.
Herramientas	VS Code, Bootstrap 5, SweetAlert2, Font Awesome.
Precedencias	4.4, 5.1
Horas estimadas	4 horas
Horas reales	4 horas

Tabla 21: Registro e inicio de sesión

5.3 Listado de activos y gráficos	
Descripción	Desarrollar la sección Mercado, con el listado de activos del S&P 500, gráficas de evolución de precios con Chart.js e información empresarial de cada compañía.
Entregables	Sección Mercado funcional con gráficas interactivas.
Herramientas	VS Code, Chart.js, Bootstrap 5, SweetAlert2, Font Awesome.
Precedencias	4.3, 4.5
Horas estimadas	9 horas
Horas reales	9 horas

Tabla 22: Listado de activos y gráficos

5.4 Generación de carteras	
Descripción	Desarrollar la sección S&P 500, donde el usuario selecciona el perfil de riesgo y el horizonte temporal, ejecuta el algoritmo y visualiza los pesos recomendados para cada activo mediante gráfico donut y tabla de resultados.
Entregables	Sección de generación de carteras funcional.
Herramientas	VS Code, Chart.js, Bootstrap 5, SweetAlert2, Font Awesome.
Precedencias	4.2, 4.6, 4.5
Horas estimadas	8 horas
Horas reales	9 horas

Tabla 23: Generación de carteras

5.5 Mi portafolio y sección Aprende	
Descripción	Desarrollar la sección Mi portafolio, con el historial de carteras generadas y gráficos sobre el balance de las posiciones, y la sección Aprende, con contenido explicativo sobre el algoritmo y los conceptos básicos de inversión.
Entregables	Secciones Mi portafolio y Aprende funcionales.
Herramientas	VS Code, Chart.js, Bootstrap 5, SweetAlert2, Font Awesome, AOS.
Precedencias	4.4, 4.5
Horas estimadas	10 horas
Horas reales	10 horas

Tabla 24: Mi portafolio y sección Aprende

6.1 Pruebas del algoritmo de la aplicación	
Descripción	Verificar el correcto funcionamiento del algoritmo de Markowitz y sus extensiones, comprobando que resultados son coherentes con distintos perfiles de riesgo y distintos importes.
Entregables	Informe de pruebas del algoritmo.
Herramientas	VS Code, Python, pandas.
Precedencias	4.2, 4.6
Horas estimadas	5 horas
Horas reales	6 horas

Tabla 25: Pruebas del algoritmo de la aplicación

6.2 Pruebas de la conexión con el front-end	
Descripción	Comprobar que todos los *endpoints* responden correctamente, que los datos se muestran bien en las vistas y que el flujo de usuario funciona sin errores desde el registro hasta la generación de carteras.
Entregables	Aplicación validada de extremo a extremo.
Herramientas	Navegador web, curl.
Precedencias	4.5, 5.1, 5.2, 5.3, 5.4, 5.5
Horas estimadas	4 horas
Horas reales	4 horas

Tabla 26: Pruebas de la conexión con el front-end

6.3 Corrección de errores	
Descripción	Identificar y corregir los errores detectados durante las pruebas, tanto en el algoritmo como en la interfaz, hasta obtener una versión estable de la aplicación.
Entregables	Código depurado y estable.
Herramientas	VS Code, navegador web.
Precedencias	6.1, 6.2
Horas estimadas	8 horas
Horas reales	8 horas

Tabla 27: Corrección de errores

Módulo 7: Documentación

7.1 Redacción de la memoria	
Descripción	Redactar la memoria del TFG en Overleaf a lo largo de todo el proyecto, documentando cada apartado de manera estructurada.
Entregables	Memoria del TFG completa.
Herramientas	Overleaf, navegador web.
Precedencias	—
Horas estimadas	80 horas
Horas reales	90 horas

Tabla 28: Redacción de la memoria

7.2 Preparación de la defensa	
Descripción	Preparar la presentación oral del TFG y ensayar para la defensa ante el tribunal.
Entregables	Presentación de la defensa.
Herramientas	PowerPoint, navegador web.
Precedencias	7.1
Horas estimadas	13 horas
Horas reales	—

Tabla 29: Preparación de la defensa

3.3.2. Planificación temporal

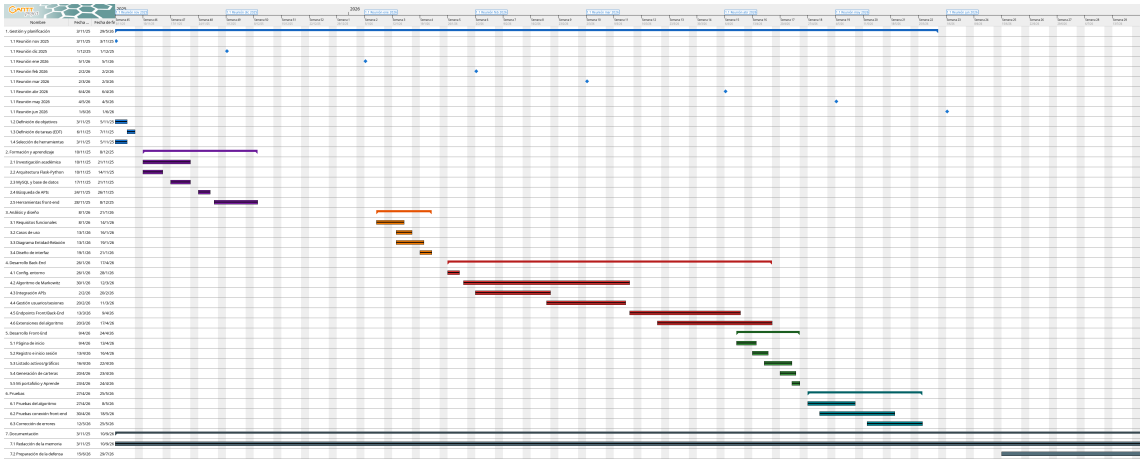


Figura 5: Planificación temporal del proyecto InvestFlow mediante diagrama de Gantt

3.4. Herramientas

En esta sección se detallan las tecnologías, lenguajes de programación y herramientas empleadas para el desarrollo del proyecto.

Lenguajes de programación

- **Python 3.10** Lenguaje de programación orientado a objetos. Es un lenguaje de fácil uso y potente en análisis de datos. Este lenguaje tiene un gran ecosistema de librerías que permiten analizar datos de manera consistente, lo cual es necesario para la cantidad de información que dispone la aplicación. Se ha utilizado para construir el Back-End completo.
- **JavaScript** Lenguaje de programación que se ejecuta en el navegador del usuario, permite a las aplicaciones ejecutar comportamiento dinámico e interactivo sin recargar la página. Se ha utilizado para definir la estructura de todas las vistas de la aplicación: página de inicio, mercado, portafolios, etc.
- **HTML5** Lenguaje de marcado que define la estructura y el contenido de las páginas web como texto, vídeos, imágenes y enlaces. Es el lenguaje utilizado por los navegadores para interpretar el contenido.
- **CSS3** Lenguaje de estilos que se emplea para el diseño visual de las páginas web. Trabaja junto a HTML, definiendo colores, tamaños y animaciones de los elementos. Se ha manejado para diseñar la interfaz de usuario, con un diseño moderno para la aplicación.
- **SQL** Lenguaje de consulta para interactuar con bases de datos relacionales. Permite crear tablas, insertar datos y realizar consultas de forma eficiente. Se ha utilizado para definir el esquema de la base de datos MySQL del proyecto y para las operaciones de lectura y escritura: usuarios, portafolios, activos y precios históricos del S&P 500.

Back-End: Librerías de Python principales

- **Flask** Es un *framework* para Python. Permite construir aplicaciones web [40]. Se ha utilizado para gestionar todas las rutas de la aplicación, las sesiones del usuario y gestionar la comunicación entre el Front-End y Back-End.
- **pandas** Es una librería para analizar datos en forma de tablas. Es una de las herramientas más utilizadas en el análisis de datos con Python [45]. Se ha utilizado para procesar los precios históricos de los activos que componen la aplicación, calcular retornos y preparar los datos.
- **NumPy** Librería de cálculo numérico con matrices y vectores [43]. Necesario para todos los cálculos matriciales del algoritmo de Markowitz.
- **SciPy** Librería de computación científica [44]. Necesaria para la optimización matemática de carteras del algoritmo con `scipy.optimize.minimize`.
- **yfinance** Librería que permite descargar datos históricos de bolsa desde Yahoo Finance de forma gratuita [29]. Ideal para obtener los registros históricos de los activos e información de las empresas bursátiles.
- **requests** Librería para realizar llamadas Hypertext Transfer Protocol (HTTP) a APIs externas . Necesaria para realizar peticiones hacia APIs para obtener el listado actualizado de las empresas que componen el S&P 500 [35].

Front-End: Librerías JavaScript y CSS

- **Chart.js** Librería de gráficos interactivos . Es una de las librerías de gráficos más populares por su sencillez y resultado [42]. Se ha utilizado para mostrar gráficos de líneas, diagrama de barras y gráfico donut.
- **Bootstrap 5** *Framework* CSS que facilita la creación de interfaces web modernas adaptándose a cualquier tamaño de pantalla [41]. Ha sido utilizado para estructurar la barra de navegación, botones y formularios de la aplicación.
- **SweetAlert2** Librería JavaScript que reemplaza las alertas por ventanas personalizables [52]. Se utiliza para mostrar mensajes de error, confirmaciones de acciones requeridas y notificaciones al usuario.
- **AOS (Animate On Scroll)** Librería JavaScript que permite animar atributos CSS [53]. Es muy sencilla de integrar ya que no requiere escribir código. Se ha utilizado para animar la aparición de secciones de manera progresiva a medida que desciendes por las páginas de la aplicación.

Base de datos

- **MySQL 8.0** Gestor de base de datos de código abierto. Es muy utilizado en aplicaciones web por su facilidad de uso. El uso de MySQL sirve para organizar la información en tablas y relacionar datos entre sí [46].

Herramientas para el desarrollo

- **Visual Studio Code** Editor de código principal de código abierto. Se ha utilizado para escribir el código fuente de la aplicación [56].
- **Git** Sirve para registrar las nuevas versiones que van desarrollándose a medida que avanza el proyecto, muy útil para registrar los cambios [57].
- **GitHub** Sitio web donde se aloja el repositorio remoto del proyecto [58].
- **Terminal (Bash)** Interfaz de línea de comandos utilizada para ejecutar la aplicación, lanzar los scripts automatizados y actualizar la base de datos.
- **Ubuntu 24.04 LTS** Sistema operativo donde se ha desarrollado el proyecto [59].

Herramientas para documentación y diagramas

- **Visual Paradigm** Herramienta utilizada para diseñar el diagrama de casos de uso, el diagrama de actores y el modelo entidad-relación de la base de datos [36].
- **GlooMaps** Herramienta de mapas visuales utilizada para construir el EDT del proyecto por su sencillez e interfaz intuitiva [37].
- **Draw.io** Herramienta para realizar diagramas, utilizada para elaborar el diagrama cliente-servidor de la arquitectura de la aplicación [38].
- **GanttProject** Aplicación gratuita utilizada para gestionar proyectos. Esta herramienta ha sido utilizada para planificar y visualizar el cronograma del proyecto mediante un diagrama de Gantt [39].
- **Overleaf** Editor de texto online basado en Latex utilizado para redactar la documentación de este Trabajo de Fin de Grado [60].

3.5. Análisis de Riesgos

A lo largo del proyecto pueden surgir problemas que afecten al correcto desarrollo, ya sean averías, fallos en las integraciones externas, errores de seguridad o imprevistos en la planificación. Por este motivo es necesario tener en cuenta los posibles problemas que puedan surgir y crear planes de contingencia para resolverlos de la forma más rápida y eficiente posible. A continuación se exponen los riesgos que podrían surgir durante la realización de este proyecto:

3.5.1. Planificación de gestión de riesgos

Identificación de probabilidad	
Nivel	**Probabilidad (%)**
Baja	0 – 30
Media	30 – 70
Alta	70 – 100

Tabla 30: Niveles de probabilidad de riesgos

Identificación de impacto	
Nivel	**Retraso**
Muy bajo	Menos de 1 día
Bajo	Entre 1 y 3 días
Moderado	Entre 3 y 5 días
Alto	Entre 5 y 10 días
Muy alto	Más de 10 días

Tabla 31: Niveles de impacto de riesgos

3.5.2. Plan de riesgos

Es muy complicado que durante la elaboración de un desarrollo no se dé ningún imprevisto; por lo que es necesario identificar, elaborar un plan de prevención y un plan de contingencia para cuando surjan.

Incumplimiento en la planificación temporal	
Descripción	Retraso en alguna de las tareas previstas, impidiendo cumplir los plazos establecidos en la planificación.
Nivel de probabilidad	Media
Plan de prevención	Ser generoso a la hora de asignar las duraciones de las tareas y revisar el avance semanalmente con el tutor.
Plan de contingencia	Reasignar la duración de la tarea aumentando el tiempo disponible y, si fuera necesario, reducir el alcance de alguna funcionalidad secundaria.
Nivel de impacto	Moderado – Muy alto

Tabla 32: Riesgo: Incumplimiento en la planificación temporal

Fallo del equipo personal	
Descripción	Parte del proyecto se va a realizar con un ordenador personal, por lo que cualquier fallo de hardware o software podría retrasar el desarrollo.
Nivel de probabilidad	Baja
Plan de prevención	Mantener el equipo actualizado y realizar copias de seguridad periódicas del código mediante Git y GitHub.
Plan de contingencia	Al disponer del repositorio en GitHub, el proyecto puede recuperarse y continuarse desde cualquier otro dispositivo.
Nivel de impacto	Bajo – Moderado

Tabla 33: Riesgo: Fallo del equipo personal

API de datos financieros no viable	
Descripción	La API seleccionada para obtener datos financieros puede resultar inviable por restricciones económicas, límites de peticiones o cambios en sus condiciones de uso. Este riesgo se materializó durante el proyecto: Financial Modeling Prep (FMP) fue descartada al requerir un plan de pago para acceder a los tickers necesarios.
Nivel de probabilidad	Alta
Plan de prevención	Investigar en detalle las condiciones de uso y los límites de cada API antes de integrarla en el proyecto, priorizando alternativas gratuitas y de código abierto.
Plan de contingencia	Buscar una alternativa equivalente. En el caso de InvestFlow, se optó por combinar *web scraping* con un *dataset* público de GitHub, lo que permitió continuar el desarrollo sin interrupciones significativas.
Nivel de impacto	Alto

Tabla 34: Riesgo: API de datos financieros no viable

Datos financieros incorrectos o desactualizados	
Descripción	Los datos históricos de precios obtenidos mediante `yfinance` u otras fuentes pueden contener errores, valores nulos o estar desactualizados, afectando a la calidad de los resultados del algoritmo de Markowitz.
Nivel de probabilidad	Media
Plan de prevención	Implementar validaciones y limpieza de datos antes de alimentar el algoritmo, comprobando la ausencia de valores nulos y la coherencia temporal de las series de precios.
Plan de contingencia	Descartar los activos con datos insuficientes o corruptos del proceso de optimización y registrar los errores para su revisión posterior.
Nivel de impacto	Moderado – Alto

Tabla 35: Riesgo: Datos financieros incorrectos o desactualizados

Cambio de requisitos del proyecto	
Descripción	Los requisitos del proyecto pueden cambiar a lo largo del desarrollo, ya sea por decisión propia, por indicación del tutor o por limitaciones técnicas descubiertas durante la implementación.
Nivel de probabilidad	Media
Plan de prevención	Definir con claridad los requisitos funcionales desde el inicio y mantener reuniones periódicas con el tutor para validar el alcance del proyecto.
Plan de contingencia	Replantear las funcionalidades afectadas, ajustar la planificación y documentar el motivo del cambio en la memoria.
Nivel de impacto	Moderado – Alto

Tabla 36: Riesgo: Cambio de requisitos del proyecto

Vulnerabilidades de seguridad en la aplicación	
Descripción	Una implementación incorrecta de la autenticación puede exponer datos de los usuarios: almacenamiento de contraseñas sin cifrado, consultas SQL vulnerables a inyección o una gestión inadecuada de las sesiones de Flask.
Nivel de probabilidad	Baja
Plan de prevención	Aplicar buenas prácticas de seguridad desde el inicio: uso de `werkzeug.security` para el *hash* de contraseñas, consultas parametrizadas en MySQL y configuración correcta de las sesiones de Flask.
Plan de contingencia	Identificar y corregir la vulnerabilidad de forma inmediata, y si fuera necesario, notificar a los usuarios afectados y restablecer las credenciales comprometidas.
Nivel de impacto	Alto – Muy alto

Tabla 37: Riesgo: Vulnerabilidades de seguridad en la aplicación

3.6. Descripción del presupuesto

Todo proyecto conlleva una serie de costes asociados: de personal, hardware, software y gastos indirectos. En este apartado se realiza una evaluación económica sobre los costes de este proyecto, con el objetivo de estimar el valor real del trabajo realizado.

3.6.1. Costes de personal

Los costes de personal representan el gasto principal de cualquier proyecto de desarrollo software, ya que reflejan el valor del tiempo invertido por las personas implicadas. En este proyecto han participado dos roles: el estudiante como desarrollador principal y el tutor como director del proyecto.

Para calcular el coste de personal se utiliza la siguiente fórmula:

$$\text{Sueldo/hora} = \frac{\text{Salario anual}}{160\,\text{h/mes} \times 12\,\text{meses}}$$

$$\text{Coste Personal} = \text{N}^\circ \text{ horas} \times \text{Sueldo/hora}$$

Las horas del estudiante se han obtenido directamente de las horas reales registradas en cada tarea de la EDT, que cuenta un total de 320,5h horas. El coste por hora del estudiante corresponde al salario medio de un desarrollador junior en España [23], equivalente a 12,67 €/h, mientras que el del tutor refleja el salario medio de un docente universitario en España [24], que son 21,88 €/h.

$$\text{Sueldo/hora estudiante} = \frac{24{,}336\,€}{160 \times 12} = \mathbf{12{,}67}\,€/\text{h}$$

$$\text{Sueldo/hora tutor} = \frac{42{,}000\,€}{160 \times 12} = \mathbf{21{,}88}\,€/\text{h}$$

Costes de personal			
Rol	**Horas**	**Coste/hora**	**Total**
Estudiante (Desarrollador Junior)	320,5 h	12,67 €/h	4.058,74 €
Tutor / Director del proyecto	15 h	21,88 €/h	328,20 €
Total	**335,5 h**	—	**4.386,94 €**

Tabla 38: Costes de personal del proyecto

3.6.2. Costes de hardware

En este apartado se recogen los costes asociados a los dispositivos físicos utilizados durante el desarrollo del proyecto. Todo el trabajo se ha realizado sobre un único equipo personal. El coste no se calcula mediante el precio total del equipo, sino su amortización durante el tiempo de uso. Se considera una vida útil de 5 años y una jornada de trabajo habitual de 160 horas mensuales.

Ordenador portátil (HP Laptop 15s-fq2xxx): Equipo principal utilizado para el desarrollo de la aplicación, la redacción de la memoria y todas las tareas del proyecto. Funciona con Ubuntu 24.04 LTS como sistema operativo.

- Procesador: Intel Core i3-1115G4 (11.ª Gen) @ 3,00 GHz (boost hasta 4,1 GHz) — 4 núcleos
- RAM: 3,5 GB (con 4,2 GB de swap activo)
- GPU: Intel UHD Graphics G4 (integrada, Tiger Lake)
- Sistema operativo: Ubuntu 24.04.4 LTS (Noble Numbat)

Disco duro SSD NVMe (1 TB): Disco de almacenamiento adicional instalado en el equipo para ampliar la capacidad de almacenamiento del proyecto, base de datos y entorno de desarrollo.

La duración del proyecto en meses se obtiene de:

$$\text{Meses} = \frac{320{,}5\,\text{h}}{160\,\text{h/mes}} = 2{,}003\,\text{meses}$$

La amortización mensual de cada dispositivo se calcula como:

$$\text{Amortización mensual} = \frac{\text{Coste de adquisición}}{5\,\text{años} \times 12\,\text{meses}}$$

Costes de hardware				
Dispositivo	**Coste**	**Amort. mensual**	**Meses**	**Total**
HP Laptop 15s-fq2xxx	425,00 €	7,08 €/mes	2,003	14,18 €
Disco SSD NVMe 1 TB	100,00 €	1,67 €/mes	2,003	3,34 €
Total	525,00 €	—	—	**17,52 €**

Tabla 39: Costes de hardware amortizados del proyecto

3.6.3. Costes de software

Una de las características de este proyecto es que ha sido desarrollado con herramientas gratuitas y de código abierto, lo que supone un coste de software de 0 €. A continuación, se detallan las herramientas utilizadas y su licencia.

Costes de software	
Herramienta	**Licencia**
Python 3.x	Open source (PSF)
Flask, NumPy, Pandas, SciPy	Open source (MIT/BSD)
MySQL Community Edition	Open source (GPL)
Ubuntu 24.04 LTS	Open source (GPL)
Bootstrap 5, Chart.js, SweetAlert2	Open source (MIT)
Git / GitHub	Gratuito
Visual Studio Code	Gratuito
GitHub Copilot	Licencia académica (GitHub Education)
Visual Paradigm	Licencia académica
Overleaf	Versión gratuita
Total	**0,00 €**

Tabla 40: Costes de software del proyecto

Durante la fase de desarrollo Back-End, a la hora de introducir datos financieros se evaluó el uso de la API Financial Modeling Prep (FMP), que requería un plan de pago para acceder a los tickers necesarios. Finalmente se descartó y se optó por alternativas gratuitas.

3.6.4. Gastos indirectos

Los gastos indirectos se calculan como el 2 % de los gastos directos del proyecto:

$$\text{Gastos directos} = 4{,}386{,}94\,€ + 17{,}52\,€ + 0{,}00\,€ = \mathbf{4{,}404{,}46}\,€$$

$$\text{Gastos indirectos} = 0{,}02 \times 4{,}404{,}46\,€ = \mathbf{88{,}09}\,€$$

3.6.5. Coste total del proyecto

El coste total del proyecto se obtiene sumando los gastos directos e indirectos:

$$\text{Gastos totales} = 4{,}404{,}46\,€ + 88{,}09\,€ = \mathbf{4{,}492{,}55}\,€$$

Resumen del coste total	
Concepto	**Coste**
Costes de personal	4.386,94 €
Costes de hardware	17,52 €
Costes de software	0,00 €
Gastos indirectos (2 %)	88,09 €
TOTAL	**4.492,55 €**

Tabla 41: Resumen del coste total del proyecto

4. Estado del Arte

En los últimos años, la industria financiera ha experimentado una transformación gracias a las nuevas tecnologías. Han surgido numerosas aplicaciones y herramientas que facilitan las inversiones de manera automatizada. Sin embargo, la mayoría de las plataformas están orientadas a un público con cierto capital disponible para invertir. Además, no ofrecen transparencia sobre el algoritmo que utilizan dentro del sistema.

En este apartado se analizan las plataformas comerciales y herramientas de código abierto más relevantes para poder orientar el proyecto. El objetivo es identificar las necesidades que InvestFlow cubre respecto a otras alternativas: una aplicación web gratuita, accesible y transparente que aplica el algoritmo de optimización de carteras de Markowitz sobre los activos del índice S&P 500.

4.1. Fuentes de datos financieros

Para el correcto funcionamiento de la aplicación, es necesario disponer de datos históricos de precios de los activos que componen el S&P 500, así como información empresarial de cada compañía. En esta sección se analizan distintas fuentes de datos, diferenciando entre APIs y datos estáticos.

4.1.1. APIs de datos financieros

- **yfinance**[1]**:** Librería Python de código abierto que permite descargar datos históricos de precios directamente desde Yahoo Finance. Ofrece metadatos empresariales e información esencial de distintos activos. Su principal ventaja es que es completamente gratuita, no requiere registro ni API key, y permite obtener datos históricos con una sola llamada.
- **Financial Modeling Prep (FMP)**[2]**:** [32] API financiera que ofrece datos fundamentales, precios históricos, balances y listados de tickers de índices bursátiles. Inicialmente fue evaluada para obtener el listado de tickers del S&P 500 junto con su información sectorial. Sin embargo, el acceso a los endpoints necesarios requería un plan de pago, por lo que fue descartada.
- **Alpha Vantage**[3]**:** [33] API gratuita con clave de acceso que proporciona datos históricos de precios de acciones, divisas y criptomonedas.

[1] `https://pypi.org/project/yfinance/`
[2] `https://financialmodelingprep.com/`
[3] `https://www.alphavantage.co/`

- **Polygon.io**[4]**:** [34] API financiera con datos en tiempo real e históricos de mercados de Estados Unidos.

4.1.2. Datos estáticos

- **Dataset público de GitHub**[5]**:** [31] Repositorio de código abierto que mantiene actualizado el listado de empresas que componen el S&P 500, incluyendo *ticker*, nombre, sector, subsector y fecha de incorporación al índice.

Para el desarrollo del proyecto se utilizó la combinación de fuentes estáticas, junto con `yfinance` para los precios históricos. Además, como fuente de verificación y complementos se realizó un proceso de extracción de datos automatizados mediante la librería `requests` y `BeautifulSoup` [55], lo que permitió obtener y contrastar información de los sectores con el dataset de GitHub.

4.2. Aplicaciones similares

Para poder buscar diferenciación respecto a otras alternativas es importante analizar otras aplicaciones ya existentes en el mercado. En esta sección se se evalúan plataformas del sector de la gestión de carteras de inversión, identificando funcionalidades, similitudes y limitaciones respecto a InvestFlow.

4.2.1. Indexa Capital

Indexa Capital[6] [26] es el robo-advisor líder en España por volumen de activos gestionados. Su propuesta es la gestión automatizada de carteras de fondos indexados de bajo coste, adaptadas al perfil inversor de cada usuario.

El proceso comienza con un cuestionario de evaluación de perfil inversor, a través del cual la plataforma determina automáticamente el nivel de riesgo del usuario. El resultado es un perfil numérico del 1 al 10, sobre el que se asigna una cartera modelo predefinida compuesta por entre 2 y 12 clases de activo, según el importe invertido.

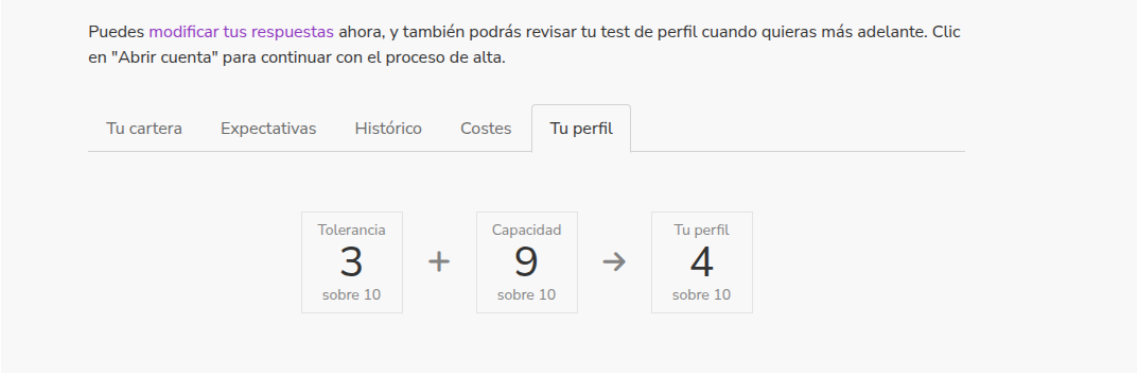


Figura 6: Resultado del test de perfil inversor de Indexa Capital

[4] `https://polygon.io/`
[5] `https://github.com/datasets/s-and-p-500-companies`
[6] `https://indexacapital.com`

Como se puede observar en la figura 6, el perfil inversor se calcula como la combinación ponderada entre la tolerancia (3/10) y la capacidad (9/10), resultando en un perfil final de 4/10. El usuario no puede fijar este valor manualmente: la plataforma lo determina de forma automática en función de sus respuestas.

Una vez calculado el perfil, Indexa Capital propone una cartera modelo con los pesos asignados a cada clase de activo. En el caso del perfil 4/10, la cartera asigna un 40 % a acciones globales mediante el fondo Vanguard Global Stk Idx Eur y un 60 % a bonos globales mediante el Vanguard Global Bnd Idx Eur, tal y como se muestra en la figura **??**.


seleccionado el mejor fondo indexado en términos de liquidez, costes y representatividad de la clase de activo.

Tamaño de la cartera Menos de 10 mil euros

CLASE DE ACTIVO	INVERSIÓN	%	VALOR
Acciones globales	Vanguard Global Stk Idx Eur -Ins Plus	40,0 %	800 €
Bonos globales a largo plazo cubierto a euro	Vanguard Global Bnd Idx Eur -Ins Plus	60,0 %	1.200 €

Aviso legal: esta cartera busca un objetivo de diversificación global y minimización de costes. No tiene en cuenta ninguna política de integración de los riesgos de sostenibilidad ni posibles incidencias adversas.

Figura 7: Cartera modelo propuesta para el perfil 4/10

Sus principales limitaciones son las siguientes:

- Requiere un depósito mínimo de 2.000 € para abrir una cartera de fondos indexados.
- El perfil de riesgo lo asigna automáticamente la plataforma; el usuario no puede ajustarlo de forma continua.
- No permite seleccionar sectores ni activos individuales: la cartera es fija por perfil.
- No ofrece transparencia sobre el algoritmo de optimización utilizado.
- Está orientada a fondos indexados globales, no a la selección de acciones individuales del S&P 500.

4.2.2. Finizens

Finizens[7] [27] es un robo-advisor español que gestiona carteras de fondos indexados. Ofrece 6 perfiles de cartera predefinidos, donde el perfil 1 corresponde al más conservador y el perfil 6 al 100 % invertido en renta variable.

Una de sus características más destacadas es que publica en su web un backtest histórico de cada cartera modelo desde el año 2011, lo que permite al usuario evaluar el comportamiento pasado de su inversión. Finizens describe su metodología como basada en *"reglas y algoritmos sistemáticos"* orientados a maximizar el compromiso rentabilidad-riesgo, aunque sin nombrar explícitamente ningún modelo académico de referencia.

[7] `https://finizens.com`



Figura 8: Simulador de carteras de Finizens

Sus principales limitaciones son las siguientes:

- Depósito mínimo de 1.000 € para el plan de inversión.
- Las 6 carteras modelo comparten los mismos fondos; únicamente varían los pesos según el perfil.
- No permite seleccionar activos individuales ni ajustar la composición de la cartera.
- El algoritmo subyacente no se publica ni es reproducible por el usuario.

4.2.3. eToro

eToro[8] [28] es una plataforma de inversión que permite operar con acciones, ETFs y criptomonedas, y seguir las estrategias de otros inversores mediante su función *CopyTrader*. Adicionalmente, ofrece *Smart Portfolios*: carteras temáticas predefinidas por el equipo de analistas de eToro, orientadas a sectores como tecnología, energías renovables o inteligencia artificial.

Al igual que InvestFlow, eToro incorpora un cuestionario inicial donde el usuario indica su tolerancia al riesgo como se muestra en la figura **??**.

[8] `https://etoro.com`

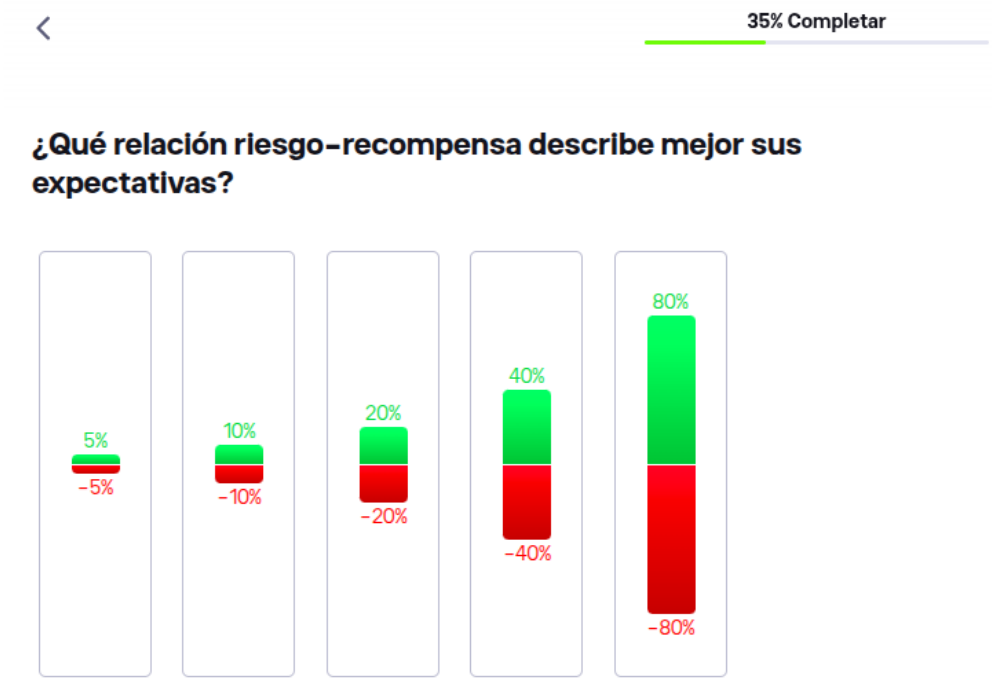


Figura 9: Cuestionario de tolerancia al riesgo de eToro

En cuanto a la visualización de activos, eToro ofrece gráficos de rendimiento histórico para cada acción, de forma similar a la sección *Mercado* de InvestFlow, donde el usuario puede consultar la evolución del precio de cada activo del S&P 500 a lo largo del tiempo.

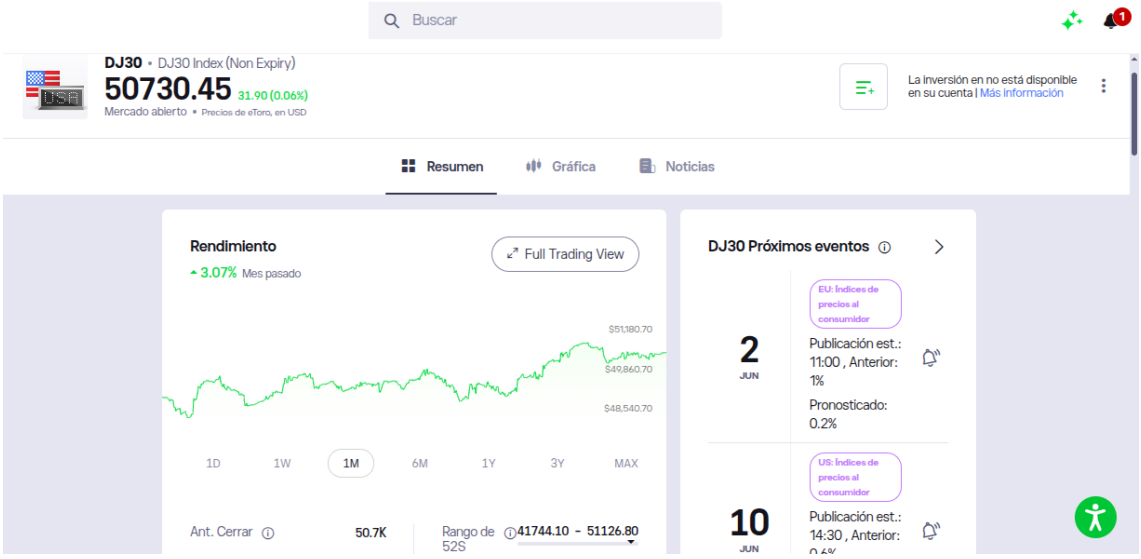


Figura 10: Visualización de activo individual en eToro

La sección *Smart Portfolios* muestra carteras temáticas como *Chip-Tech*, *Quantum-Computing* o *WarrenBuffet-CF*, construidas manualmente por analistas de eToro, tal y como se muestra en la figura **??**.

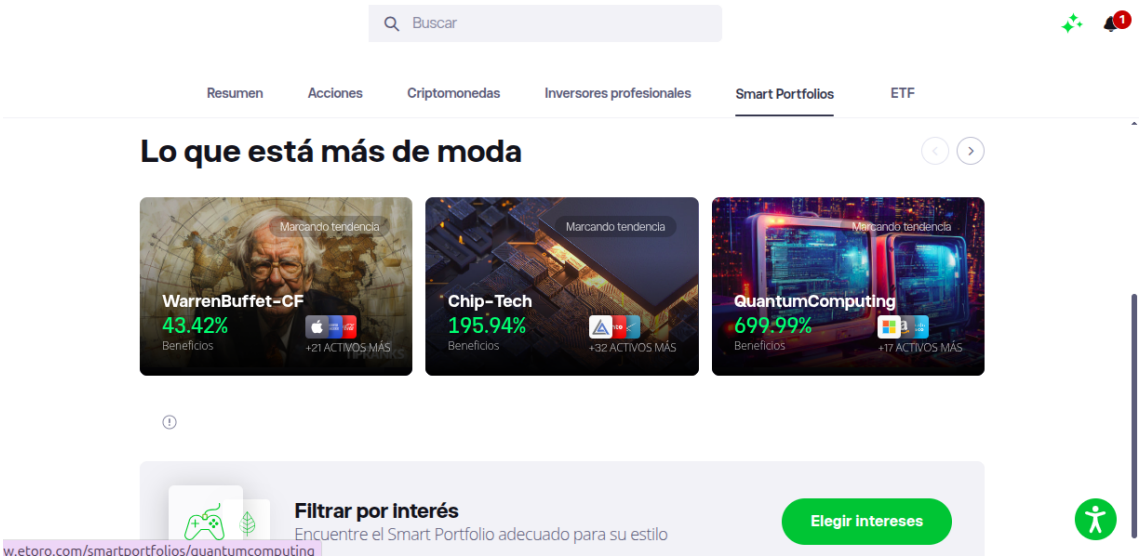


Figura 11: Smart Portfolios disponibles en eToro

En esta sección, el portafolio generado por eToro muestra para cada posición el precio actual, el peso en la cartera y el porcentaje de ganancia o pérdida acumulada. Esta funcionalidad es similar a la sección *Mi portafolio* de InvestFlow, donde el usuario puede consultar el rendimiento de la cartera y el balance de cada posición a lo largo del tiempo.



Figura 12: Balance de posiciones en un Smart Portfolio de eToro

Sus principales limitaciones son las siguientes:

- Los Smart Portfolios son carteras realizadas por analistas, sin optimización algorítmica basada en modelos matemáticos.
- Requiere un depósito mínimo para operar con dinero real.
- No permite construir ni personalizar una cartera optimizada propia.

4.3. Conclusiones

En conclusión, la aplicación a desarrollar puede tener factores de diferenciación y valor añadido respecto al resto de alternativas analizadas, especialmente por su enfoque en la optimización mediante algoritmos matemáticos, la transparencia del funcionamiento de la aplicación, el uso exclusivo sobre el índice S&P 500 de forma gratuita y accesible. A continuación se justifican las fuentes de datos seleccionadas y las funciones que la aplicación ofrece al usuario.

4.3.1. Fuentes de datos elegidas

Para la obtención de los datos necesarios se evaluaron distintas fuentes, tal y como se ha descrito en la sección anterior. Los criterios de selección fueron los siguientes:

- **Disponibilidad gratuita:** La fuente debe ser accesible sin coste económico, descartando cualquier API que requiera un plan de pago para los endpoints necesarios.
- **Completitud:** Debe proporcionar tanto los datos históricos de precios como la información sectorial y empresarial de los activos del S&P 500.
- **Facilidad de integración:** La fuente debe poder integrarse de forma sencilla con el entorno de desarrollo Python y Flask utilizado en el proyecto.

Aplicando estos criterios, Financial Modeling Prep (FMP) fue descartada al requerir un plan de pago para acceder al listado completo de tickers del S&P 500 con su información sectorial. Alpha Vantage y Polygon.io también fueron descartadas por sus limitaciones en el número de peticiones diarias, además de los cambios que pueda sufrir una API, ya sea cambiando el plan gratuito, cambiando el endpoint, o no pudiendo acceder a esos datos en concreto.

Como solución, se optó por combinar dos fuentes complementarias: el *dataset* público de GitHub para obtener el listado de empresas con su información sectorial, verificado mediante *web scraping* sobre Wikipedia [30] con las librerías `requests` y `BeautifulSoup`. Para los precios históricos de cada activo se utilizó `yfinance`, que permite descargar series temporales completas de forma gratuita y sin necesidad de registro.

4.3.2. Funciones a ofrecer

Del análisis de las aplicaciones existentes se ha identificado que todas comparten un conjunto de funcionalidades básicas: visualización de gráficos de rendimiento de activos, seguimiento del balance de posiciones y algún mecanismo de adaptación al perfil de riesgo del usuario. InvestFlow incorpora estas funcionalidades y añade un valor diferencial respecto a las alternativas analizadas. A continuación se detallan las funciones que ofrece la aplicación:

- **Sin barreras de entrada:** A diferencia de los robo-advisors analizados, InvestFlow no requiere ningún depósito mínimo ni compromiso económico. La aplicación actúa como un recomendador de carteras: calcula y muestra la distribución óptima de activos según el perfil del usuario, pero la decisión y la ejecución de la inversión

recae completamente en él. El usuario define libremente tanto su tolerancia al riesgo como el capital que desea invertir.

- **Consultar activos del S&P 500:** El usuario puede explorar los activos que monitoriza la aplicación, consultar su información empresarial y visualizar la evolución histórica del precio mediante gráficos interactivos.
- **Generar una cartera de inversión optimizada:** A partir del perfil de riesgo definido por el usuario mediante un *slider* de tolerancia al riesgo y la selección del sector del S&P 500 deseado (o multisector), el algoritmo de Markowitz calcula la distribución óptima del capital.
- **Consultar mi portafolio:** El usuario puede consultar las carteras generadas, visualizar el balance de cada posición con indicadores de ganancia y pérdida, y seguir la evolución del rendimiento a lo largo del tiempo mediante gráficos.
- **Backtesting de la cartera:** La aplicación permite evaluar el comportamiento histórico de la cartera generada, ofreciendo métricas como la rentabilidad anualizada, la volatilidad y el ratio de Sharpe.
- **Sección Aprende:** El usuario puede acceder a contenido educativo sobre los conceptos básicos de inversión y sobre el funcionamiento del algoritmo de Markowitz, con el objetivo de mejorar la cultura financiera del usuario.

5. Captura de Requisitos

Para esta sección, se describen los requisitos que debe tener la aplicación.

5.1. Documento de Requisitos

La aplicación va dirigida principalmente al usuario interesado en el ámbito de las inversiones bursátiles. El propósito es que cualquier persona, sin conocimientos previos, sea capaz de utilizar la aplicación.

La aplicación contiene información que puede ser útil para posibles compras/ventas de acciones. Además, la sección de aprendizaje proporciona información sobre el funcionamiento del algoritmo utilizado y puntos importantes a la hora de comenzar en el mundo bursátil.

5.1.1. Funcionalidades del sistema

En este apartado, se describen las funcionalidades que debe cumplir el sistema para su correcto funcionamiento.

- **Registro de Usuarios**
  - Los usuarios deberán poder registrarse en la plataforma proporcionando un nombre de usuario, nombre, apellido, correo electrónico y contraseña.
  - El sistema deberá comprobar que no existen nombres de usuario ni correos electrónicos duplicados.
- **Inicio de Sesión**
  - Los usuarios deberán poder iniciar sesión tanto con el correo electrónico como con el nombre de usuario, junto con la contraseña.
  - Los usuarios podrán recuperar la contraseña en caso de no recordarla o tras cinco intentos fallidos de inicio de sesión incorrectos.
- **Gestión inicial**
  - Cualquier usuario no registrado también podrá acceder a la página principal, donde se mostrará información general sobre la aplicación.
  - La página de inicio deberá guiar al usuario de forma clara e intuitiva.
- **Gestión de aprendizaje**

- Los usuarios podrán acceder a una sección donde podrán obtener información acerca del funcionamiento completo de la aplicación.
- El sistema deberá proporcionar información detallada sobre el proceso de generación de carteras.
- La sección también ofrecerá contenido para orientar al usuario sin conocimientos financieros previos.

- **Mercado de activos**
  - Los usuarios podrán consultar datos acerca de los activos que componen la aplicación de manera sencilla.
  - Los datos deberán ser legítimos y estar actualizados en todo momento. Los usuarios podrán ver la evolución de los datos.

- **Gestión de carteras de inversión**
  - Los usuarios podrán generar tantas carteras por sector como sectores hay en el índice S&P 500 o generar una cartera multisector diversificada que contiene las mejores empresas por cada sector.
  - El sistema permitirá ajustar la tolerancia al riesgo mediante un control deslizante, generando carteras conservadoras, moderadas o agresivas según el perfil del inversor.
  - El sistema generará carteras optimizadas aplicando el algoritmo de Markowitz, calculando la frontera eficiente y seleccionando la cartera con la mejor relación rentabilidad-riesgo para el perfil indicado.
  - Los usuarios podrán guardar las carteras generadas en su portafolio personal para su seguimiento posterior.
  - Los usuarios podrán consultar el backtesting de una cartera guardada, visualizando cómo habría evolucionado históricamente su inversión.

- **Gestión del portafolio personal**
  - Los usuarios podrán consultar todas las carteras guardadas en su portafolio personal, visualizando su composición y las métricas principales de cada una.
  - El sistema mostrará el histórico de cada cartera guardada, permitiendo al usuario ver la evolución de su inversión a lo largo del tiempo.
  - Los usuarios podrán eliminar cualquier cartera de su portafolio personal.

- **Gestión del perfil de usuario**
  - Los usuarios podrán consultar y modificar sus datos de perfil, incluyendo nombre, apellidos, correo electrónico y contraseña.
  - Los usuarios podrán cerrar sesión en cualquier momento desde la plataforma.
  - Los usuarios podrán eliminar su cuenta de perfil permanentemente.

5.1.2. Requisitos no Funcionales

Por otro lado, los requisitos no funcionales son aquellos que describen las características de un sistema que no están directamente relacionadas con las funciones específicas que realiza, sino con la forma en que estas funciones se llevan a cabo:

- **Seguridad**
  - Las contraseñas de los usuarios deberán almacenarse cifradas mediante un algoritmo de hash seguro.
  - Todas las rutas que manejen datos privados estarán protegidas y solo serán accesibles por usuarios autenticados.
- **Disponibilidad y mantenimiento de datos**
  - Los datos de mercado deberán mantenerse actualizados de forma automática mediante procesos programados periódicamente.
  - El sistema deberá estar disponible de forma continua durante el período de uso.
- **Mantenibilidad**
  - El código deberá estar estructurado de forma modular, separando claramente la lógica de negocio, los datos y la interfaz de usuario.
- **Usabilidad**
  - La interfaz deberá ser intuitiva y accesible para usuarios sin conocimientos técnicos ni financieros previos.
  - El sistema deberá ser responsivo y adaptarse correctamente a distintos tamaños de pantalla.
- **Rendimiento**
  - El sistema deberá responder a las peticiones del usuario en un tiempo razonable, no superando los 3 segundos en operaciones comunes de consulta.
  - La generación de carteras optimizadas, al ser un proceso computacionalmente costoso, deberá completarse en un tiempo máximo de 10 segundos.

5.2. Casos de Uso

Los diagramas de casos de uso sirven para especificar la manera en que itera un sistema con el usuario y también con otros sistemas externos. Es necesario para conocer qué puede realizar cada uno dentro de la aplicación.

Además, la jerarquía de actores sirve para organizar a los usuarios según el nivel de acceso que tengan, un usuario con mayor privilegio puede heredar casos de uso de otro usuario con menor privilegio.

En esta aplicación, se han definido tres actores: *Usuario no Identificado*, *Usuario Identificado* y *Administrador*. El Usuario Identificado hereda los casos de uso del Usuario no Identificado, ya que, una vez que se identifica, puede acceder a las funcionalidades públicas.



Figura 13: Actores que participan en el sistema

Vamos a realizar el Diagrama de Casos de Uso de los actores que participan en el sistema:

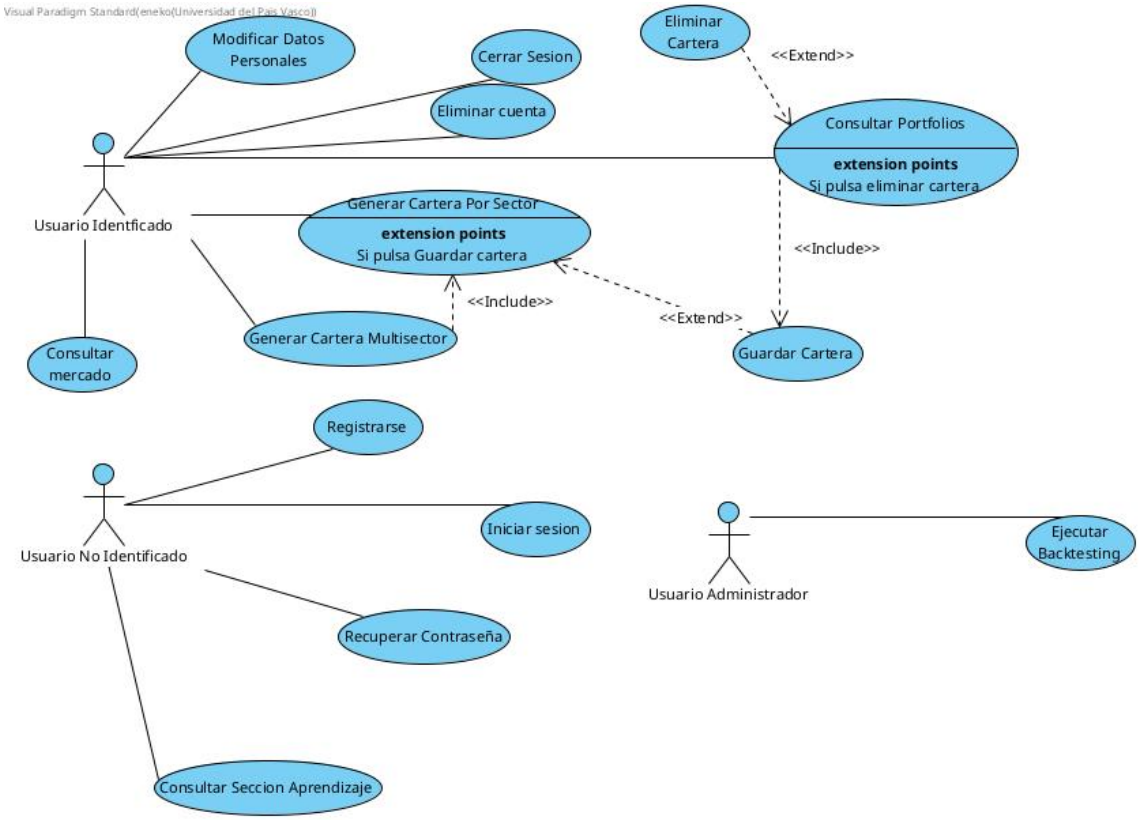


Figura 14: Diagrama de Casos de Uso

5.2.1. Usuario no Identificado

Para el usuario no identificado, es decir, el usuario que no se ha registrado previamente podrá:

- **Registrarse:** Un usuario ajeno al sistema podrá crear una nueva cuenta introduciendo sus datos personales, como el nombre, apellidos, correo electrónico y el nombre de usuario.

- **Iniciar Sesión:** Cualquier usuario puede acceder al sistema mediante sus credenciales. Los datos introducidos deberán ser correctos y estar registrados en el sistema previamente.
- **Recuperar Contraseña:** El usuario puede solicitar un cambio de contraseña introduciendo el correo electrónico asociado a su cuenta. El sistema enviará un enlace de recuperación al correo para que pueda establecer una nueva contraseña.
- **Consultar Sección Aprendizaje:** Cualquier usuario puede acceder a la sección donde se muestra la información del algoritmo de la aplicación y los conceptos básico del mundo bursátil.

5.2.2. Usuario Identificado

Para el usuario identificado, es decir, el que ha introducido sus credenciales válidas y ha accedido a todas las funciones se han identificado los siguientes casos de uso.

- **Cerrar Sesión:** El usuario cierra la sesión de su cuenta.
- **Modificar Datos Personales:** El usuario identificado puede editar tanto el nombre de usuario, el nombre, los apellidos y el correo electrónico.
- **Eliminar Cuenta:** El usuario identificado puede eliminar permanentemente su cuenta del sistema.
- **Consultar Mercado:** Una vez identificado, el usuario puede acceder al listado del mercado del S&P 500, donde podrá filtrar activos por sectores, precios o variaciones de precio a lo largo del tiempo, también dispone de un apartado para buscar un activo en concreto.
- **Generar Cartera Por Sector:** El usuario configura los parámetros necesarios para la optimización y se ejecuta el algoritmo de Markowitz para generar la cartera óptima.
- **Generar Cartera Multisector:** El usuario elige la opción de generar cartera multisector, donde configura los parámetros para generar una cartera diversificada entre activos de distintos sectores del índice.
- **Guardar Cartera:** Una vez generada una cartera, el usuario puede guardarla en el sistema. La cartera queda almacenada junto con las posiciones y el precio de entrada de cada activo en el momento que se guarda.
- **Eliminar Cartera:** Una vez guardada la cartera, el usuario puede eliminarla de su portafolio. La cartera junto con sus posiciones y balance queda eliminada del sistema.
- **Consultar Portfolios:** El usuario puede visualizar todas las carteras guardadas. Al consultar, el portafolio muestra la valoración actual de cada posición respecto al precio de entrada y el histórico de valor de la cartera en distintos rangos temporales: 1 mes, 3 meses, 6 meses, 1 año y desde el inicio.

5.2.3. Usuario Administrador

El administrador es un actor con acceso restringido al panel de administrador. Se identifica mediante un token en la cabecera HTTP. No dispone de login.

- **Ejecutar Backtesting:** El administrador puede simular qué cartera habría generado el algoritmo en una fecha pasada y comparar la rentabilidad obtenida desde esa fecha respecto al mercado, permitiendo así validar y evaluar el comportamiento del algoritmo de optimización.

5.3. Modelo de Dominio

La base de datos de la aplicación es necesaria para almacenar tanto los datos de los activos financieros como la información de los usuarios y las carteras generadas. Un diseño sólido permite realizar las consultas de manera eficiente y que los datos sean coherentes a lo largo del tiempo. En este caso, la base de datos utiliza el motor InnoDB de MySQL para garantizar integridad mediante claves foráneas y soporte para las transacciones.

Este es el modelo entidad-relación de la aplicación:

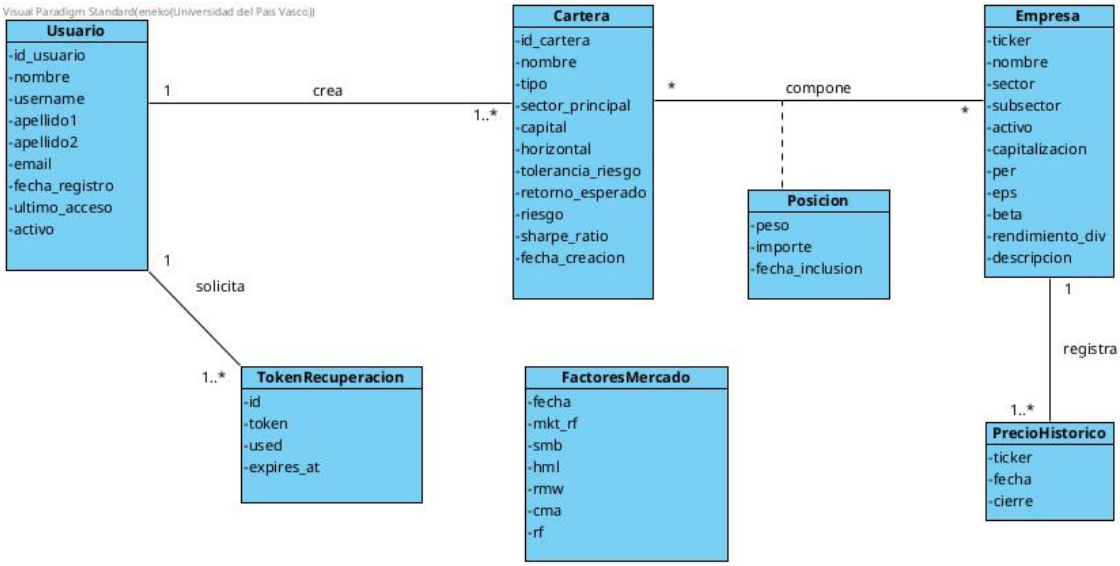


Figura 15: Modelo Entidad-Relación

5.3.1. Entidades

Aquí se muestran las entidades que componen el sistema:

- **Usuario**: Esta entidad representa a los usuarios que se registran dentro de la plataforma. Tienen atributos como el identificador único (`id_usuario`), nombre `nombre`, nombre de usuario (`username`), apellido1 `apellido1`, apellido2 `apellido2`, correo electrónico `email`, contraseña almacenada de forma segura mediante un hash `password_hash`, fecha de registro `fecha_registro`, fecha de último acceso `ultimo_acceso` y un indicador de si la cuenta está activa o no `activo`.

- **TokenRecuperación**: Esta entidad representa las solicitudes que se realizan para recuperar la contraseña, estas solicitudes las realiza el usuario. Se identifica mediante un `id` y almacena el token único que se genera `token`, la fecha de expiración del token (`expires_at`), el atributo `tipo` que clasifica el token y un id de si el token ya ha sido utilizado (`used`).

- **Empresa**: Esta entidad representa las empresas que componen el índice S&P 500. Se identifica por el símbolo bursátil (`ticker`) y contiene atributos como `nombre`, `sector`, `subsector` y si la empresa continúa en el índice `activo`. Además, almacena datos actualizados periódicamente como la capitalización bursátil `capitalizacion`, la relación entre el precio de la acción y el beneficio `per`, el beneficio por acción `eps`, el rendimiento del dividendo `rendimiento_div`, la volatilidad relativa al mercado `beta`, el margen del beneficio `margen_beneficio`, el volumen medio `volumen_medio`, los máximos y mínimos en las últimas 52 semanas `max_52s` y `min_52s` respectivamente, una descripción `descripcion`, el número de empleados `empleados`, el país `pais`, la moneda `moneda`, el sitio web `sitio_web`, la fecha de la última actualización del índice `ultima_act_universo` y la última actualización de la información de la empresa `ultima_act_info`.

- **PrecioHistórico**: Esta entidad representa los precios históricos de cierre diario de cada empresa del S&P 500. Se identifica mediante una clave compuesta, formada por el ticker (`ticker`) y la fecha (`fecha`), y contiene el precio de cierre (`cierre`).

- **FactoresMercado**: Esta entidad representa los factores del modelo Fama-French, obtenidos mediante la Kenneth French Data Library. Se identifica por la fecha y almacena los cinco factores que componen el modelo: prima de mercado(`mkt_rf`), tamaño (`smb`), valor (`hml`), rentabilidad (`rmw`), inversión (`cma`) y tasa libre de riesgo(`rf`). Estos factores se utilizan como referencia en el análisis de carteras.

- **Cartera**: Esta entidad representa las carteras de inversión generadas y guardadas por los usuarios. Se identifica por `id_cartera` y tiene atributos como nombre `nombre`, tipo (`sector` o `multisector`) `tipo`, sector principal `sector_principal`, capital invertido `capital`, el horizonte de tiempo de la inversión `horizonte`, la tolerancia al riesgo `tolerancia_riesgo`, el retorno esperado `retorno_esperado`, el riesgo `riesgo`, el sharpe ratio`sharpe_ratio` y la fecha de creación `fecha_creacion`.

- **Posicion**: Esta entidad intermedia representa cada activo que forma parte de una cartera en concreto. Se identifica a través de la cartera y la empresa, y almacena el peso asignado al activo (`peso`), el importe en euros (`importe`) y la fecha de cuando fue incluida en la cartera (`fecha_inclusion`).

5.3.2. Relaciones

Aquí se muestran las relaciones que componen el modelo de dominio:

- **crea 1..***: Es una relación entre Usuario y Cartera. Representa las carteras que un usuario ha guardado dentro del sistema. Un usuario puede generar múltiples carteras.

- **solicita 1..***: Es una relación entre Usuario y TokenRecuperacion. Representa las solicitudes que el usuario realiza para cambiar la contraseña. Un usuario puede generar múltiples peticiones para recuperar la contraseña.

- **compone *..***: Es una relación entre Cartera y Empresa, mediante la entidad Posicion. Representa los activos que componen cada cartera: una cartera incluye varias empresas y una misma empresa puede aparecer en distintas carteras de inversión.
- **registra 1..***: Es una relación entre Empresa y PrecioHistorico. Representa el histórico de precios de cierre diario de cada empresa del índice. Cada empresa tiene múltiples cierres de precio.

6. Análisis y diseño

En este capítulo se describe el análisis y diseño de la aplicación. Es importante detallar concretamente la base de datos mediante un diagrama relacional, la arquitectura web utilizada en el proyecto, el flujo de la aplicación mediante diagramas de secuencia y los distintos diseños de las interfaces que componen la plataforma.

6.1. Diagrama relacional de la Base de Datos

Una vez definido el modelo de dominio, se debe transformar en la base de datos relacional que permita almacenar y consultar toda la información de forma consistente.

A continuación, se muestra el diagrama de la base de datos, donde se recogen las tablas y sus atributos, junto con las claves primarias y foráneas, y las relaciones mediante claves.

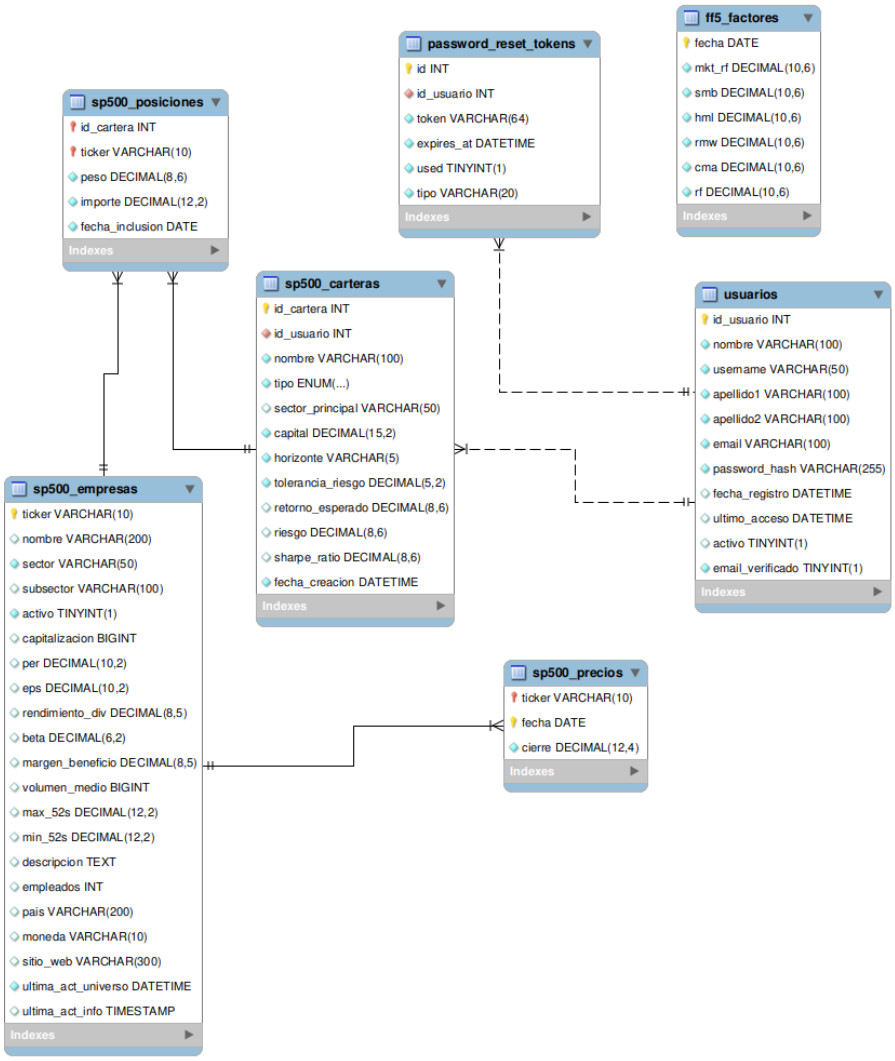


Figura 16: Diagrama de Base de Datos

6.1.1. Descripción de las tablas

A continuación, se describe cada tabla de la base de datos:

6.1.1.1. Tabla: usuarios

Almacena la información de los usuario registrados en la plataforma. Se identifica mediante **id_usuario** como clave primaria. Los campos **email** y **username** tienen restricción `UNIQUE`, ya que no pueden existir dos correos electrónicos o nombres de usuario dentro del sistema. La contraseña se almacena como **password_hash**, en vez de contenerla en texto plano. EL campo **activo** permite deshabilitar cuentas sin eliminarlas.

6.1.1.2. Tabla: password_reset_tokens

Esta tabla contiene los *tokens* de recuperación de contraseña. Cada *token* es único(`UNIQUE`) para garantizar que cada solicitud se pueda identificar de forma individual.

El campo **used** es necesario para que cada enlace de recuperación sea utilizada una única vez, y **expires_at** sirve para establecer el tiempo de validez del *token* (24 horas). El campo **tipo** queda reservado para posibles extensiones futuras del sistema, como la incorporación de distintas formas de autenticarse. La clave extranjera **id_usuario** se utiliza mediante `ON DELETE CASCADE` ya que, si se elimina un usuario, todos sus *tokens* se borran automáticamente, para evitar registros que se queden huérfanos en la base de datos.

6.1.1.3. Tabla: sp500_empresas

Representa todas las empresas del índice. La clave primaria es el **ticker**. El campo **activo** indica si la empresa sigue formando parte del índice; cuando una empresa sale del S&P 500 se marca como inactiva, a pesar de ello los datos siguen manteniéndose ya que puede haber carteras antiguas que contengan el activo. Los datos bursátiles se actualizan cada 24 horas mediante **ultima_act_info**, mientras que los datos de los sectores se sincronizan trimestralmente mediante **ultima_act_universo**, esto se debe a que el índice no se actualiza diariamente, el índice se rebalancea cada cierto tiempo.

6.1.1.4. Tabla: sp500_precios

Esta tabla almacena todos los precios que se cierran diariamente en cada empresa. Utiliza una **clave primaria compuesta** formada por el **ticker** y **fecha**, ya que una empresa solo puede tener un precio de cierre por cada sesión. La clave foránea **ticker** garantiza que no existan precios de empresas no registradas en el sistema.

6.1.1.5. Tablas: ff5_factores

Contiene los factores del modelo Fama-French de cinco factores. La clave primaria es **fecha**, ya que solo existe un conjunto de factores por día de sesión. Se utiliza como fuente de datos bursátiles independiente, utilizada por el algoritmo para realizar una mejor optimización. La tabla se actualiza diariamente mediante un script automatizado.

6.1.1.6. Tablas: sp500_carteras

Esta tabla almacena las carteras de inversión creadas por cada usuario. La clave primaria es **id_cartera**. El atributo **tipo** distingue entre una cartera de un único sector (**sector**) o varios sectores (**multisector**). Se guardan también los parámetros que el usuario introdujo a la hora de realizar la cartera, como son: **capital**, **horizonte**, **tolerancia_riesgo** y los **sharpe_ratio**. La clave foránea **id_usuario** usa `ON DELETE CASCADE`, puesto que si se elimina el usuario, todas su carteras quedan borradas del sistema automáticamente.

6.1.1.7. Tabla: sp500_posiciones

Es la tabla que se crea entre `sp500_carteras` y `sp500_empresas`. Usa una **clave primaria compuesta** de **id_cartera** y **ticker**, ya que cada ticker solo puede aparecer una vez en una cartera. Contiene el atributo **peso** asignado por el algoritmo y el **importe** en euros correspondiente, además almacena la **fecha_inclusion** en la que se añadió

el activo. Tiene dos claves foráneas, si se elimina la cartera, las posiciones se borran en cascada; si se intenta introducir un ticker que no está registrado en **sp500_empresas**, no se ejecuta la operación.

6.2. Arquitectura

La arquitectura de la aplicación sigue el modelo **cliente-servidor**: el usuario, con el navegador, envía peticiones al servidor, el servidor responde con la información o las vistas que el usuario solicite. La aplicación utiliza **Flask** que actúa como servidor y escucha en el puerto 5000, atendiendo a todas las peticiones HTTP.

A continuación, se describen los componentes de la arquitectura web:

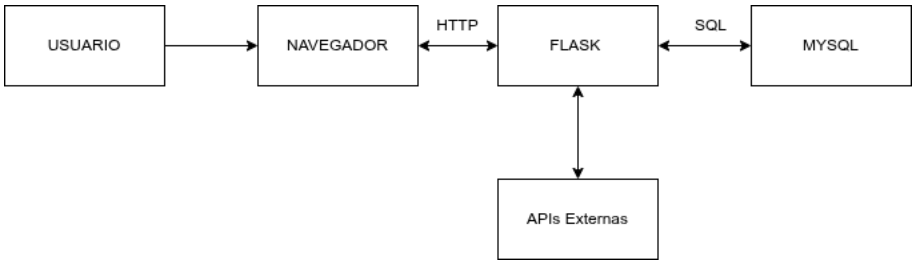


Figura 17: Arquitectura cliente-servidor de la plataforma

6.2.1. Cliente Web (Navegador)

El cliente es el navegador del usuario (Chrome, Firefox, etc.), que se ejecuta en el dispositivo donde este ejecutándose la aplicación. El usuario accede a la plataforma a través de la URL y realiza dos tipos de peticiones:

- **Peticiones de navegación (Peticiones GET)**: al acceder a un endpoint de la aplicación, el navegador solicita la página y el servidor Flask responde con una plantilla html, la cual el navegador interpreta y muestra.
- **Peticiones de datos (Peticiones POST/AJAX**: al realizar acciones como iniciar sesión, registrarse o generar una cartera de inversión, el navegador envía los datos mediante JavaScript. El servidor recoge esos datos y devuelve una respuesta en formato JavaScript Object Notation (JSON), que se utiliza para actualizar la página sin necesidad de recargarla.

6.2.2. Servidor: Aplicación Flask (InvestFlow)

El servidor es el responsable de recibir las peticiones del navegador y devolver la respuesta adecuada, esta desarrollado en Python con el framework **Flask**.

Dentro del servidor se distinguen dos bloques:

- **Rutas (`routes/`)**: actúan como intemediario en cada petición del usuario y el resto de la aplicación. Reciben las solicitudes HTTP, y realizan las consultas correspondientes en función de lo que se quiera obtener, como consultar la base de datos. Por último, devuelven la respuesta al navegador.

- **Servicios (`services/`)**: contienen la lógica para realizar los cálculos y procesos de la aplicación, como la sincronización de datos de mercado diariamente o la ejecución del algoritmo de optimización.

6.2.3. Base de datos (MySQL)

Toda la información de la aplicación se almacena en la base de datos, tanto usuarios, empresas, precios, carteras, etc. La aplicación se conecta a la base de datos a través del módulo `database.py`, que gestiona las conexiones.

6.2.4. Patrón Modelo-Vista-Controlador (MVC)

La aplicación Flask organiza el código mediante el diseño **Modelo-Vista-Controlador (MVC)**, que separa la lógica del negocio, la presentación y el flujo de peticiones en tres capas independientes:

- **Modelo**: formado por los módulos de `services/` junto con `database.py`. Encapsula la lógica del código sin conocer cómo se presentan los datos al usuario.
- **Vista**: se compone de las plantillas HTML de `templates/` que se renderizan mediante el motor *Jinja2* integrado en Flask, junto con los archivos estáticos de `static/` (CSS y JavaScript) para dar estilo y transiciones a la interfaz de usuario.
- **Controlador**: formado por `app.py` y los módulos `routes/`. Recibe la petición HTTP, llama al servicio del modelo correspondiente y decide qué vista debe devolverse como respuesta.

6.2.5. Fuentes de datos externas (APIs)

La aplicación procesa periódicamente varias fuentes de datos a través de Internet para mantener los datos actualizados dentro del sistema:

- **yfinance**: es utilizada para obtener los precios de cierre y datos de las empresas que componen el índice.
- **Kenneth French Data Library**: la fuente oficial de los cinco factores que sirven para mejorar la optimización en el algoritmo, los datos se descargan a través de la librería `pandas-datareader`.

La sincronización con las APIs ocurre a través de *scripts* automatizados con *cron* que se ejecutan diariamente y mensualmente. Se ejecutan en segundo plano para que un usuario que realice la petición tenga los datos previamente actualizados, y no tenga mucho tiempo de espera entre solicitudes.

6.3. Diseño de Interfaces

A continuación, se describen las distintas interfaces de usuario implementadas en el sistema.

6.3.1. Registro de usuario

En esta interfaz, el usuario deberá introducir los datos personales para poder acceder al sistema. El usuario deberá introducir la información que aparece como obligatoria, excepto el segundo apellido, que es opcional. El nombre de usuario no deberá tener formato de dominio web, correo electrónico o un nombre demasiado corto. Además, se comprueba si el nombre de usuario o el correo electrónico ya existen dentro de la base de datos.



Figura 18: Interfaz de registro

Si alguno de los datos personales introducidos es incorrecto, se muestra el siguiente mensaje de error:

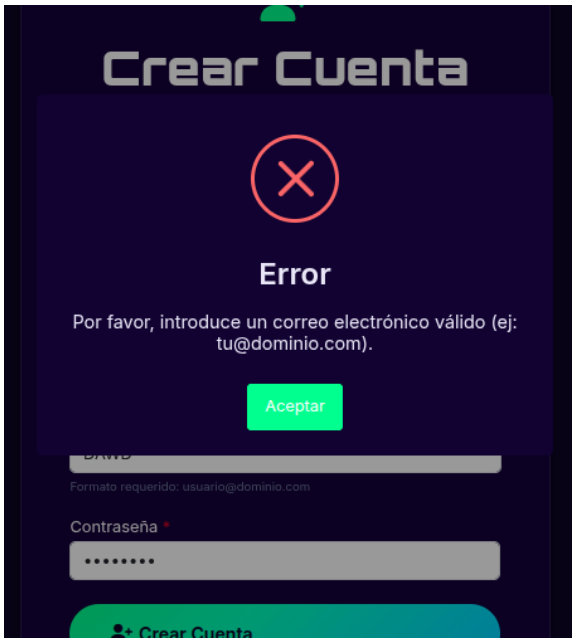


Figura 19: Mensaje de error en el registro

Una vez introducida la información, se muestra un mensaje de bienvenida:

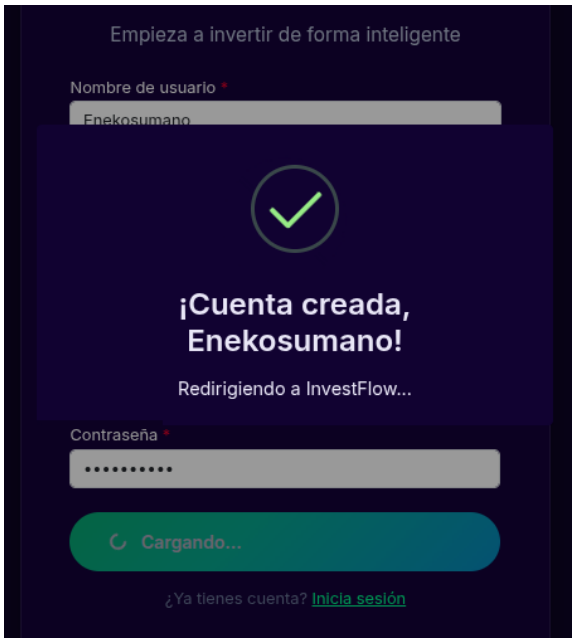


Figura 20: Mensaje de bienvenida tras registrarse

6.3.2. Inicio de Sesión

En este apartado, el usuario deberá introducir los datos personales para acceder al sistema. La plataforma permite acceder a través de correo electrónico o nombre de usuario.

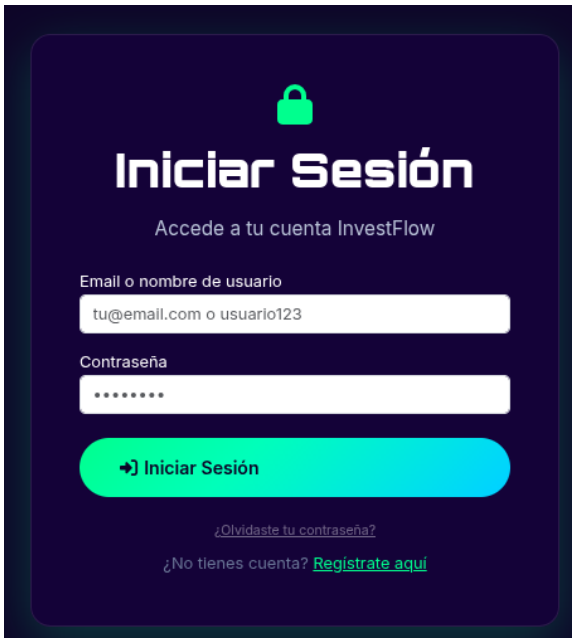


Figura 21: Interfaz de inicio de sesión

En el caso de algún dato introducido incorrectamente se muestra el siguiente pop-up:

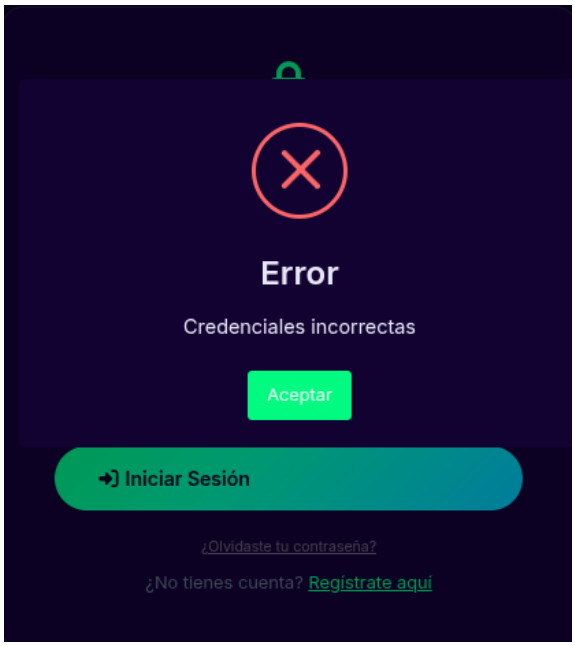


Figura 22: Mensaje de error al iniciar sesión

Una vez introducidos los datos correctamente, se muestra un mensaje de bienvenida al iniciar sesión:

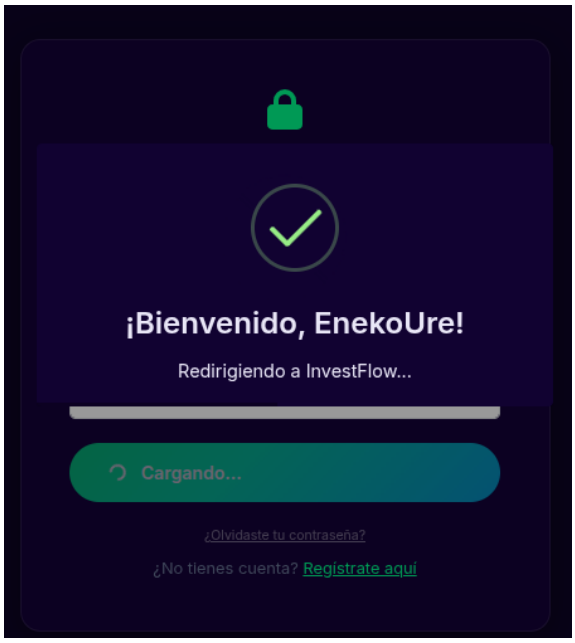


Figura 23: Mensaje de bienvenida al iniciar sesión

6.3.3. Recuperar contraseña

El usuario podrá recuperar la contraseña pulsando en el mensaje «¿Olvidaste tu contraseña?».

Una vez pulsado, se abre una ventana donde se deberá introducir el correo electrónico asociado a la cuenta del usuario:

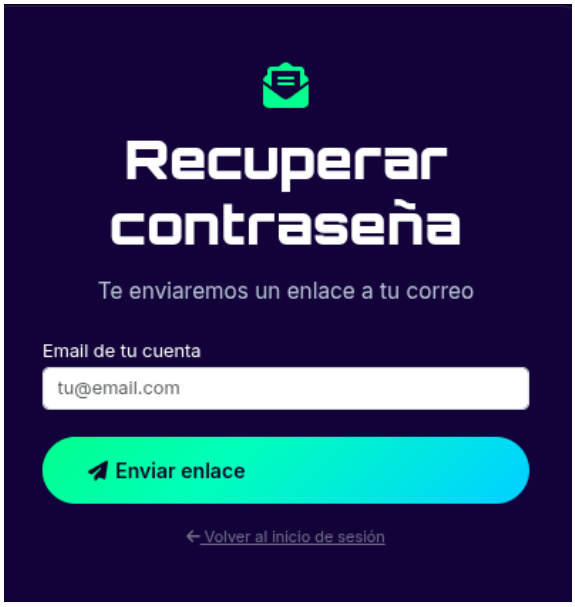


Figura 24: Interfaz recuperar contraseña

Como medida de seguridad, si el usuario introduce un correo aleatorio que no esté dentro de la base de datos, se muestra un mensaje ambiguo con el objetivo de que el atacante no sepa si existe el correo electrónico o no:



Figura 25: Mensaje de confirmación de envío del enlace

Una vez introducido el correo electrónico, se muestra el enlace de recuperación directamente en pantalla.

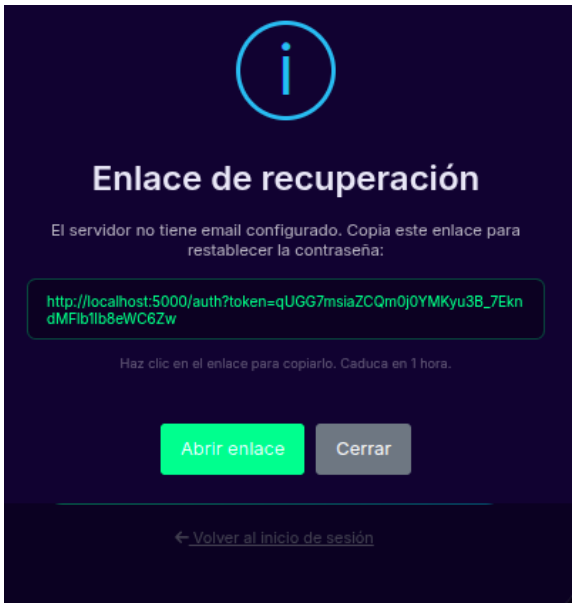


Figura 26: Enlace de recuperación

Al abrir el enlace de recuperación, el usuario podrá introducir una nueva contraseña con un mínimo de 6 caracteres.

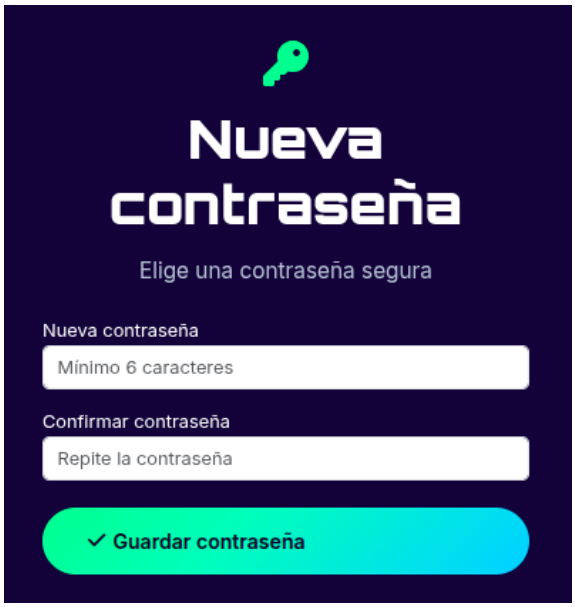


Figura 27: Interfaz para introducir nueva contraseña

Cuando el usuario completa la acción exitosamente aparece la siguiente ventana emergente:



Figura 28: Interfaz contraseña actualizada correctamente

A partir de esta acción, se podrá iniciar sesión de nuevo en la aplicación con los nuevos datos.

6.3.4. Modificar datos personales

El usuario identificado, podrá hacer clic en el nombre de usuario de la aplicación donde podrá modificar datos personales. Las validaciones en los datos siguen siendo las mismas, se deberán dejar rellenados los campos obligatorios, no podrá haber un correo

electrónico con formato inválido, el nombre de usuario tampoco podrá introducirse con formato de correo o dominio y deberá tener un mínimo de caracteres.

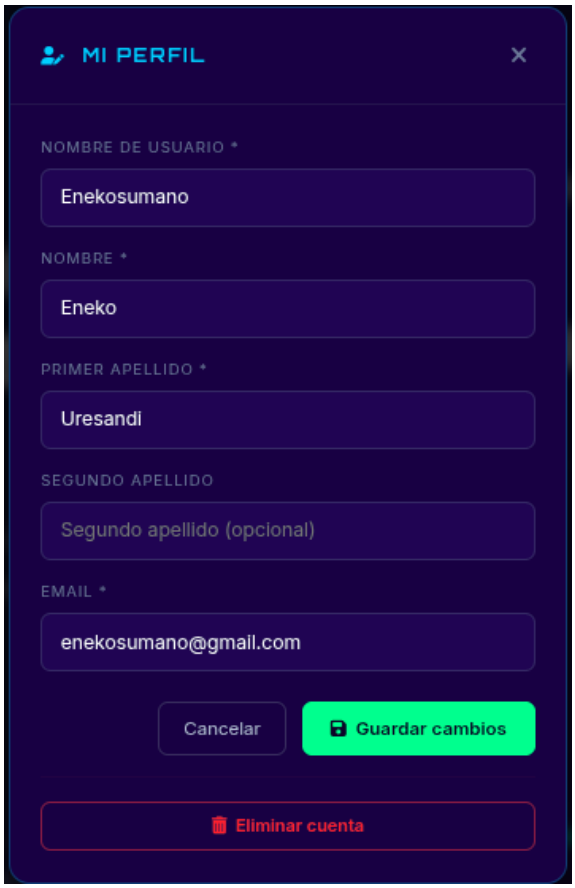


Figura 29: Interfaz modificar datos personales

Si el usuario modifica algún dato de forma incorrecta se muestra el siguiente mensaje:

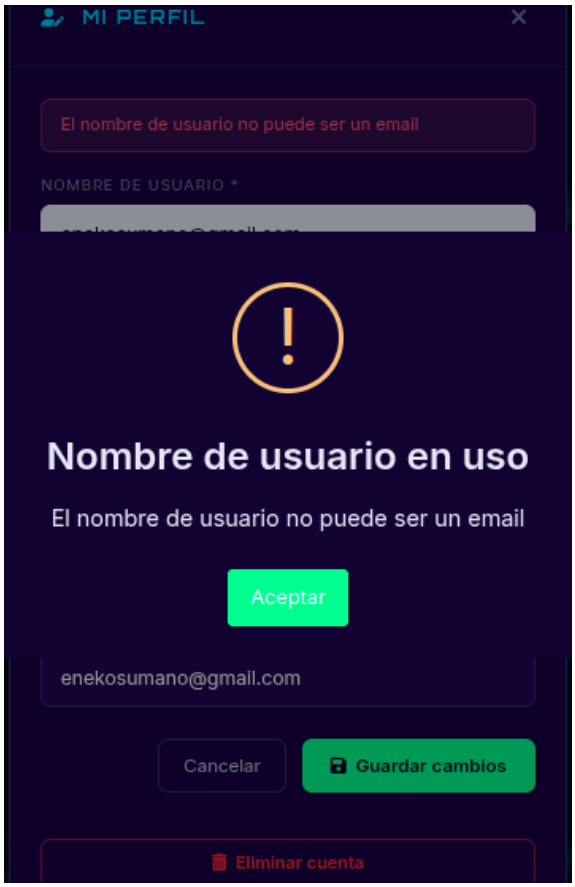


Figura 30: Mensaje de error al modificar datos

6.3.5. Eliminar cuenta

El usuario tendrá la opción de eliminar la cuenta permanentemente. Para ello, deberá hacer clic en el nombre de usuario y se volverá a abrir la ventana emergente de modificación de datos personales. En este caso, si el usuario pulsa eliminar cuenta se muestra una ventana emergente pidiendo la contraseña por motivos de seguridad.

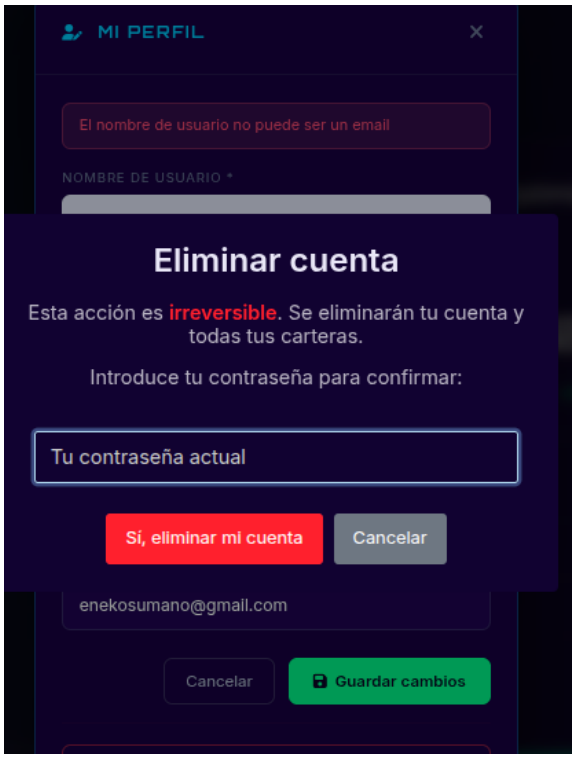


Figura 31: Interfaz de acción requerida para eliminar cuenta

Cuando el usuario introduce correctamente la contraseña, aparecerá el siguiente mensaje indicando que se ha ejecutado con éxito:

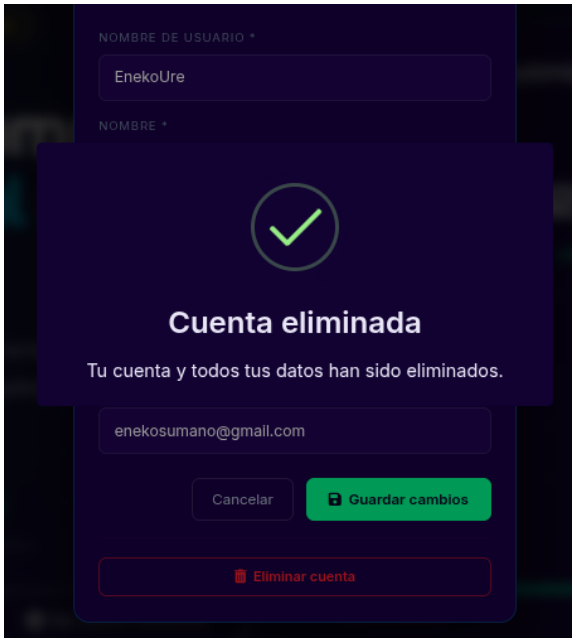


Figura 32: Interfaz de cuenta eliminada correctamente

6.3.6. Sección Inicio

Esta sección página donde se presenta la aplicación, no hay que iniciar sesión para poder acceder a ella. Explica de forma visual y resumida en qué se centra la plataforma.

Además, aparecen los demás enlaces de la aplicación para acceder directamente.



Figura 33: Interfaz de sección inicial página 1

Si realizamos *scroll* hacia abajo, se encuentra otra sección donde explica la aplicación en tres sencillos pasos:



Figura 34: Interfaz de sección inicial página 2

Continuando con el *scroll*, se muestra otra sección donde se describen las ventajas de usar la aplicación InvestFlow.



Figura 35: Interfaz de sección inicial página 3

Por último, aparece una sección donde invita al usuario a realizar su primera cartera de inversión.

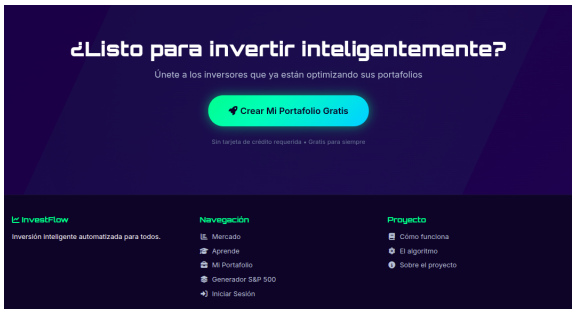


Figura 36: Interfaz de sección inicial página 4

6.3.7. Sección Aprende

Esta sección es un tutorial interactivo para que el usuario entienda el fundamento de la plataforma. Se organiza en varios bloques donde se explica el problema, la solución y la implementación del algoritmo de optimización implementado.

A continuación, se muestra el concepto riesgo-retorno donde se explica la frontera eficiente de manera sencilla:



Figura 37: Sección aprende página principal

A medida que el usuario avanza haciendo *scroll*, se van describiendo las distintas fases del algoritmo de optimización, explicando paso a paso cada apartado:



Figura 38: Interfaz de explicación del algoritmo

6.3.8. Sección Mercado

Esta sección sirve para los usuarios ya identificados, donde se podrá consultar todos los activos que pertenecen al índice S&P 500.

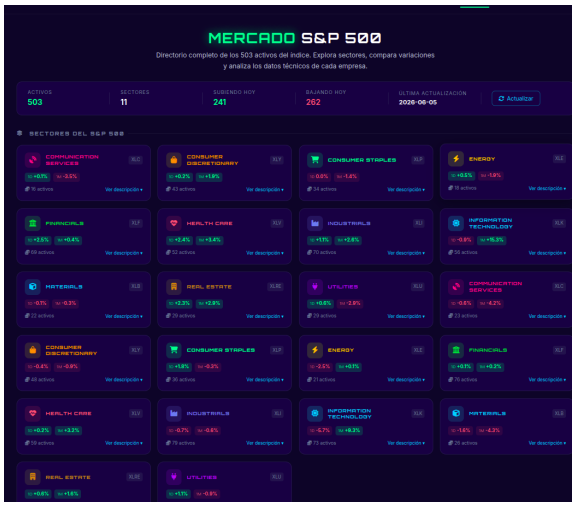


Figura 39: Interfaz de la sección mercado

Esta sección permite filtrar los activos por precios, por sectores y a través de la variación del precio en un día, una semana, un mes, etc.

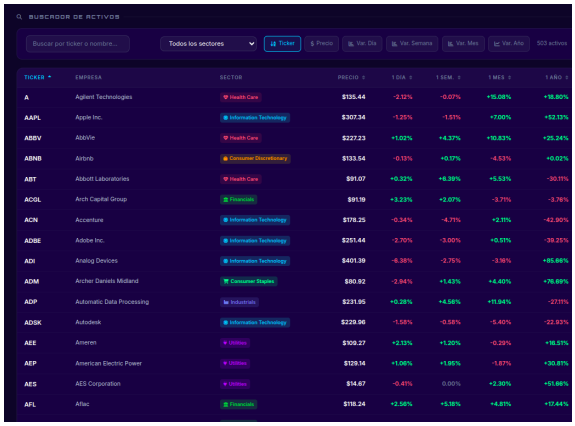


Figura 40: Interfaz para filtrar los activos

Además, haciendo clic en un activo en concreto, se muestra una pantalla donde aparecen datos acerca del activo, como el balance del precio a lo largo del tiempo e información empresarial.

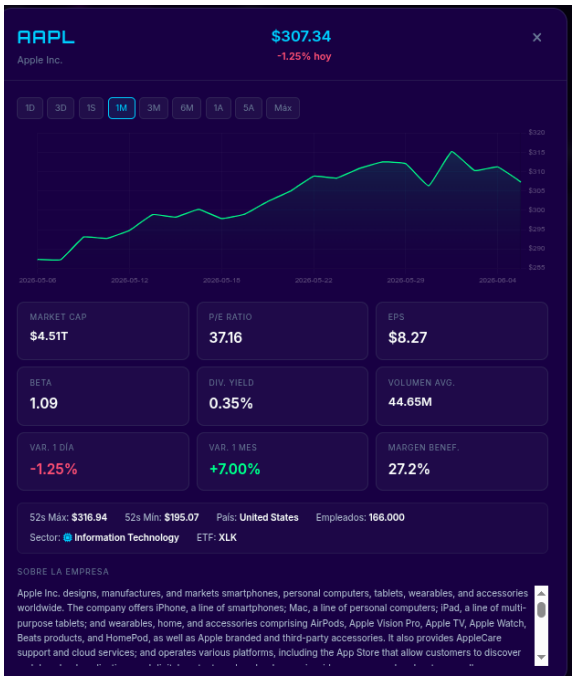


Figura 41: Información sobre activos

6.3.9. Sección S&P 500

Esta página es donde el usuario decide los parámetros para generar la cartera de inversión y también donde se ejecuta el algoritmo de optimización.



Figura 42: Interfaz de generación de carteras

Una vez introducido los parámetros que el usuario indique para su inversión, se generan las distintas carteras con sus respectivas posiciones. Además, el usuario podrá guardar la cartera que más se adapte a sus necesidades.

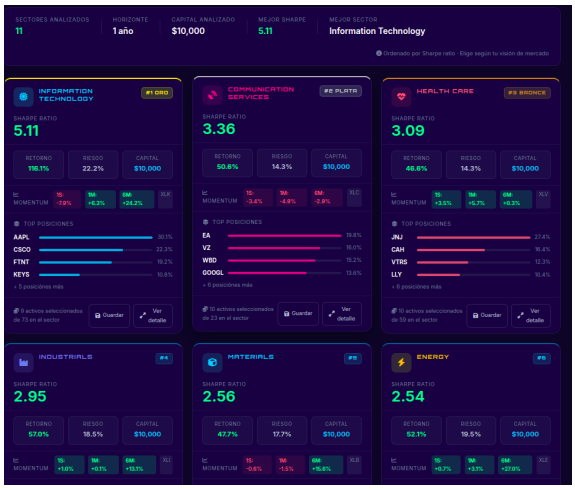


Figura 43: Carteras generadas por el algoritmo

El usuario podrá hacer clic en «Ver Detalle» para analizar las posiciones, al pulsar se muestra una ventana emergente donde se indica información acerca de la cartera: los pesos relativos de cada posición, el porcentaje de cada activo, el precio de cada activo, etc.

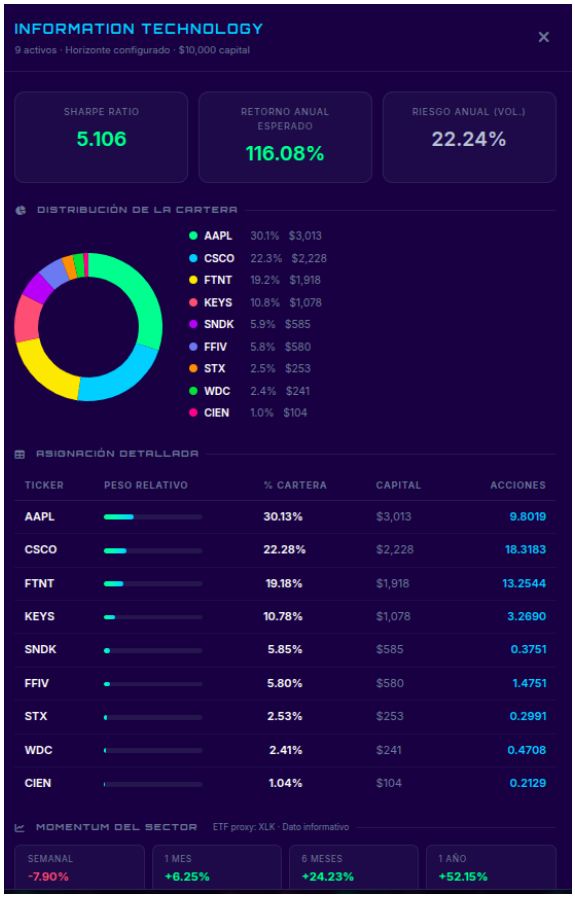


Figura 44: Interfaz de la cartera de inversión generada

6.3.10. Sección Mis Portafolios

En esta sección, se mostrarán los distintos portafolios que el usuario haya querido guardar.



Figura 45: Interfaz de los portafolios generados

Pulsando en una de las carteras, se presenta una ventana emergente con información sobre la evolución de la cartera a lo largo del tiempo dividido en tres apartados:

- **Rendimiento de la cartera:** Se muestra un gráfico de la evolución del valor total de la cartera desde la fecha de creación. Incluye también un gráfico con la ganancia acumulada en porcentaje a lo largo del tiempo, con el objetivo de mostrar si la cartera está generando rentabilidad positiva o negativa.

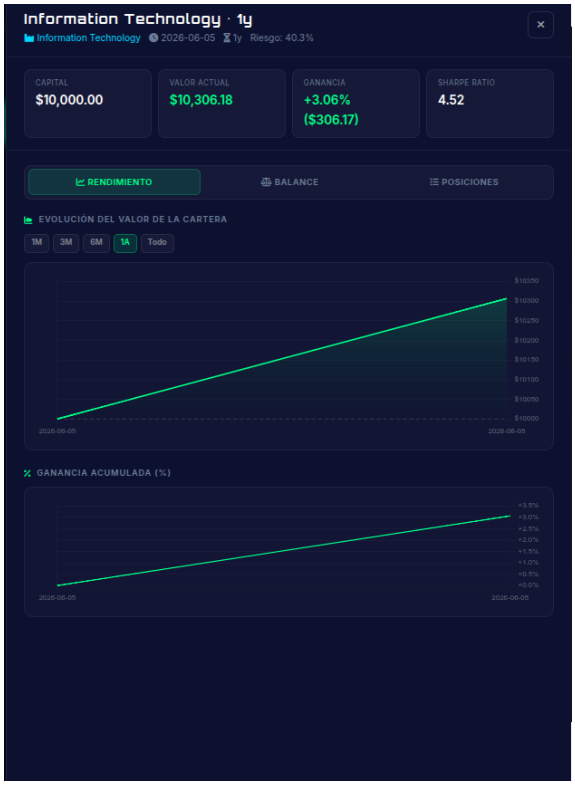


Figura 46: Rendimiento de la cartera

- **Balance de los activos:** Presenta un diagrama de barras de las ganancias o pérdidas de cada posición. Además, muestra un gráfico de rosquilla con la distribución en porcentaje del valor de los distintos activos.



Figura 47: Balance de los activos

- **Balance de las posiciones:** Muestra una lista con los precios de entrada frente al precio actual, el número de acciones, el valor actual de la posición y las ganancias o pérdidas.



Figura 48: Balance de las posiciones

Pulsando en uno de los activos también se muestra un gráfico del rendimiento de esa posición a lo largo del tiempo.



Figura 49: Rendimiento de la posición

6.3.11. Interfaces admin

En este apartado se muestra la interfaz de usuario administrador de la aplicación. En ella podrá verificar el funcionamiento de la aplicación, y si es rentable o no utilizarla para invertir.

Dentro de la interfaz, introduce los mismos parámetros de la aplicación pero añadiendo una fecha pasada, el algoritmo simula la ejecución en esa fecha introducida.

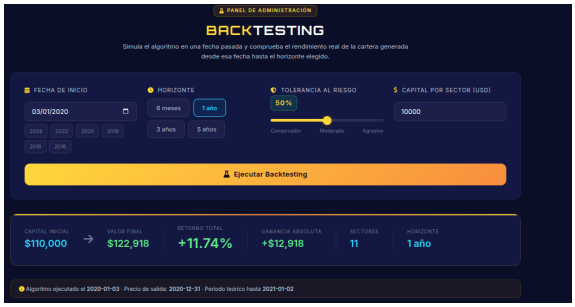


Figura 50: Interfaz de backtesting

Una vez introducidos los parámetros, al ejecutar se muestra la información de las distintas carteras de inversión y sus rendimientos en el tiempo establecido:

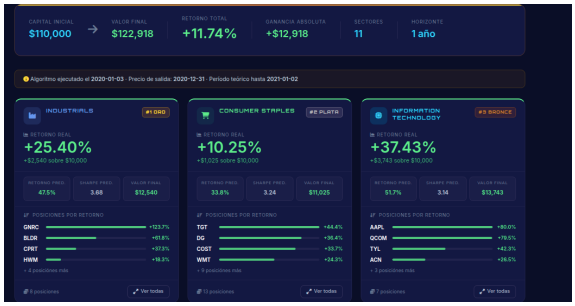


Figura 51: Resultados del backtesting

7. Desarrollo

En esta sección se describirá cómo se ha construido la aplicación de principio a fin. Para ello, se explicará qué tecnologías se han utilizado, por qué, cómo se ha estructurado el código, qué problemas se han ido encontrando a lo largo del proyecto y qué decisiones se han tomado.

7.1. Librerías y Dependencias

El proyecto está desarrollado en Python. Todas las dependencias se declaran en el archivo `app/requirements.txt` con las versiones adecuadas para el correcto funcionamiento dentro del entorno. Está probado en el sistema operativo Ubuntu 24.04 LTS.

7.1.1. Flask

Flask [40] es el *microframework* web utilizado para la aplicación. Se eligió frente a alternativas como Django porque permite crear de una manera muy sencilla aplicaciones web con Python. Además, en el caso de necesitar nuevas funcionalidades hay un conjunto grande de extensiones que se pueden instalar con Flask para mayor funcionalidad.

La versión utilizada es `Flask==3.1.2`, junto con las extensiones `Flask-Cors==5.0.0`cabeceras CORS para peticiones del frontend.

Cada página de la plataforma se define como una ruta:

```
@app.route('/market')
@login_required
def market():
  return render_template('market.html')
```

Listing 7.1: Definición de una ruta (app.py)

7.1.2. mysql-connector-python

`mysql-connector-python==9.1.0` es la biblioteca de MySQL para Python. Sirve para gestionar todas las conexiones que se realizan a la base de datos MySQL, permitiendo ejecutar consultas de forma segura.

7.1.3. yfinance

`yfinance==1.2.0` es la librería que sirve para descargar datos históricos de los precios de los activos de Yahoo Finance. Se utiliza para obtener el historial de todos los tickers, además de la información de las empresas.

Al principio se consideró utilizar una API de Financial Modeling Prep (FMP), pero se descartó ya que `yfinance` no requiere ni claves de autenticación, ni registros de usuarios y por su sencillez a la hora de realizar llamadas, devuelve tanto el histórico de precios como los datos fundamentales.

Así es como se descargan los precios de los tickers:

```
    raw = yf.download(
     group_tickers,
     start=from_date.strftime('%Y-%m-%d'),
     end=dl_end.strftime('%Y-%m-%d'),
     auto_adjust=True,
     progress=False,
     threads=True,
    )
```

Listing 7.2: Descarga de precios históricos (sp500_service.py)

7.1.4. NumPy y pandas

`numpy==2.2.1` y `pandas==2.2.3` son las dos librerías para realizar cálculos numéricos.

La librería **pandas** se encarga de organizar los datos históricos de los precios. La información se descarga y se almacena en un formato específico: una fila por cada combinación ticker y fecha, pero para calcular una cartera es necesario cambiar el formato: una columna por cada activo y una fila para cada fecha, con el precio de cierre correspondiente. Con la función `pivot` de pandas, se transforma y permite calcular fácilmente el rendimiento logarítmico de cada activo.

```
    prices = df.pivot(index='fecha', columns='ticker', values='cierre')
    log_returns = np.log(prices / prices.shift(1)).dropna()
```

Listing 7.3: Pivote de precios y retorno logarítmico (sp500_portfolio_generator.py)

La librería **NumPy** sirve para la parte matemática del optimizador. Calcula el rendimiento y el riesgo de una cartera mediante operaciones con matrices, es decir: multiplica matrices de forma muy rápida para conocer cómo los activos se mueven entre sí (su covarianza).

7.1.5. SciPy

`scipy==1.15.1` proporciona una función `scipy.optimize.minimize`, que es necesario para optimizar las carteras. Esta función permite encontrar entre todas las combinaciones posibles de pesos, la combinación que mejor cumple el objetivo, sin tener que probar de manera manual cada una de ellas.

A lo largo del proyecto, la función se utiliza para calcular cada punto de la frontera eficiente del algoritmo. Recibe un vector de pesos inicial que reparte el capital a partes

iguales a los activos. El objetivo que se minimiza/maximize puede ser el riesgo (volatilidad) o el ratio de Sharpe de la cartera, que se calculan a partir de los retornos esperados y la matriz de covarianzas.

Ningún activo puede tener un peso negativo ni superar el 30 % de la cartera. Las restricciones obligan a que todos los pesos sumen uno, es decir, que se invierta el 100 % del capital disponible.

El método elegido para resolver el problema es `SLSQP` (*Sequential Least Squares Programming*), que permite manejar las restricciones de igualdad (todos los pesos sumen uno) y de desigualdad (los límites que tiene cada peso).

```
    result = minimize(
      fun=self._negative_sharpe,
      x0=initial_weights,
      method='SLSQP',
      bounds=bounds,
      constraints=constraints,
      options={'maxiter': 1000, 'ftol': 1e-9}
    )
```

Listing 7.4: Optimización del Sharpe Ratio con SLSQP (efficient_frontier.py)

7.1.6. scikit-learn

`scikit-learn==1.7.2` se utiliza para calcular la matriz de covarianzas, que mide cómo se mueven los precios de los activos entre sí. El problema es que, si se calcula la matriz a partir de los datos históricos y hay pocos en comparación con el número de activos, el resultado sería poco fiable, ya que puede contener ruido.

Para evitarlo, se utiliza el estimador de `Ledoit-Wolf` que utiliza `shrinkage`. El estimador sirve para acercar la matriz de covarianzas a una versión más simple y suavizar los datos.

```
    lw = LedoitWolf()
    lw.fit(log_returns.values)
    cov_matrix = lw.covariance_ * 252
```

Listing 7.5: Estimación de covarianza con shrinkage (sp500_portfolio_generator.py)

El resultado es una estimación del riesgo más realista y menos sensible a los datos.

7.1.7. pandas-datareader

`pandas-datareader` se utiliza para descargar los *factores de Fama-French*, que son datos públicos que se actualizan periódicamente y sirven para explicar por qué unos activos rinden más que otros (por ejemplo, las empresas pequeñas frente a las grandes, o las que están baratas frente a las caras). Estos datos se descargan de la *Kenneth French Data Library* y se guardan en la tabla `ff5_factores`.

```
    import pandas_datareader.data as web
    raw = web.DataReader(
     'F-F_Research_Data_5_Factors_2x3_daily',
     'famafrench',
    )[0]
```

Listing 7.6: Descarga de factores Fama-French (sp500_service.py)

En el proyecto se utiliza para obtener un dato de referencia sobre la "tasa libre de riesgo"(lo que se podría ganar sin asumir ningún riesgo, invirtiendo en la deuda del Gobierno de EE. UU.), necesario para calcular el ratio de Sharpe. Por otro lado, se utiliza para detectar qué tipo de activos están de moda en el mercado en el último mes, y asumir más pesos a los activos que acompañan esa tendencia.

7.1.8. Bootstrap

Para poder adaptarse a un diseño *responsive*, se hace uso de Bootstrap. Esta librería permite que la aplicación se adapte a los distintos dispositivos utilizando su sistema de columnas y el menú del móvil (`navbar-toggler` / `collapse`).

7.2. Instalación y Preparación del Entorno

7.2.1. Preparación del entorno de desarrollo

Los requisitos previos para ejecutar la aplicación de forma local son:

- **Python 3.12** o superior.
- **MySQL 8.0** escuchando en el puerto `localhost:3306`.

```
git clone <repo-url> && cd TFG-EstrategiaInversion
python3 -m venv venv
source venv/bin/activate        # Linux / macOS
# venv\Scripts\activate.bat     # Windows
```

Listing 7.7: Creación del entorno virtual

7.2.2. Instalación de las librerías

```
pip install -r app/requirements.txt
```

Listing 7.8: Instalación de dependencias

7.2.3. Configuración de variables de entorno

La aplicación lee la configuración a través del fichero `.env` en la carpeta raíz, el repositorio incluye un fichero de ejemplo `.env.example` para documentar las variables del entorno.

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=tu_contrasena
DB_NAME=estrategia_inversion
SECRET_KEY=cambia-esto-por-una-clave-secreta
FLASK_PORT=5000
FLASK_DEBUG=True
```

Listing 7.9: .env.example

7.2.4. Inicialización de la base de datos

El esquema completo de la base de datos se encuentra en `database/crear_tablas.sql`. Para inicializarlo:

```
mysql -u root -p < database/crear_tablas.sql
```

Listing 7.10: Inicialización de la base de datos

Este script crea la base de datos junto con las tablas de la aplicación.

7.3. Implementación

7.3.1. Estructura de las carpetas

El proyecto se divide en carpetas que separan las rutas, los servicios, utilidades y recursos estáticos:

```
TFG-EstrategiaInversion/
└── app/
    ├── __init__.py
    ├── app.py
    ├── database.py
    ├── requirements.txt
    ├── routes/
    │   ├── usuarios.py
    │   ├── sp500_routes.py
    │   └── admin_routes.py
    ├── services/
    │   ├── sp500_service.py
    │   ├── sp500_portfolio_generator.py
    │   ├── efficient_frontier.py
    │   └── factor_service.py
    ├── utils/
    │   ├── auth.py
    │   └── email.py
    ├── templates/
    │   ├── index.html
    │   ├── auth.html
    │   ├── market.html
    │   ├── portfolios.html
    │   ├── sp500.html
    │   ├── aprende.html
    │   └── admin_backtest.html
    └── static/
        ├── css/style.css
        └── js/
            ├── main.js
            ├── sp500.js
            ├── market.js
            ├── profile.js
            └── admin_backtest.js
```

```
├── database/
│   └── crear_tablas.sql
├── .env
└── .env.example
```

- **app.py:** Aquí se crea la app de Flask, se conectan los Blueprints y se definen las páginas principales de la aplicación.
- **database.py:** Utiliza dos funciones (get_db_connection() y close_db_connection()) que se utiliza para abrir o cerrar la conexión con la base de datos.
- **routes/:** Las carpetas con lo que recibe las peticiones HTTP.
- **services/:** Donde se ejecuta la lógica de la aplicación, dentro del directorio están los ficheros que se utilizan para descargar datos, optimizar las carteras y calcular los factores.
- **utils/:** Funciones auxiliares para proteger rutas que requieren estar logueado (`login_required`) y la función de envío de correo electrónico.

7.3.2. Población e inicialización de datos

Una vez creado el esquema de la base de datos, es necesario introducir datos históricos en ella.

```
python3 scripts/init_db.py
```

Listing 7.11: Inicialización de datos (primera ejecución)

El script realiza las siguientes acciones de forma secuencial:

1. Descarga las empresas que componen el S&P 500 desde un *dataset* público de GitHub y la contrasta con los datos de Wikipedia para evitar fallos.
2. Descarga el historial de precios de las 503 empresas mediante `yfinance`, procesándolas en lotes de 50 tickers para evitar sobrecargar la API.
3. Descarga los factores Fama-French 5 desde la *Kenneth French Data Library* mediante `pandas-datareader` y los almacena en la tabla `ff5_factores`.

7.3.3. Mantenimiento automático de datos

Para mantener los datos actualizados tras la inicialización, existen dos scripts de actualización programados mediante `cron`:

- **update_daily.py**: actualiza los precios de cierre de todos los activos del universo.
- **update_monthly.py**: actualiza la composición del índice S&P 500 y los factores Fama-French 5.

```
# Diario: lunes a viernes a las 18:00h
0 18 * * 1-5  /ruta/venv/bin/python /ruta/proyecto/scripts/update_daily.py

# Mensual: dia 1 de cada mes a las 9:00h
0 9  1  * *   /r
```

Listing 7.12: Configuración de cron para las actualizaciones automáticas

Las fechas en las que se ejecuta y se actualizan dependerán de la configuración del cron.

7.3.4. Estructura Lógica: Blueprints de Flask

La aplicación organiza cada endpoint en un Blueprint independiente, en lugar de centralizar todas las rutas de la plataforma.

- **usuarios** (`/api/usuarios`): registro, login, logout, perfil, recuperación de contraseña.
- **sp500** (`/api/sp500`): generación de carteras, mercado, sincronización del universo, gestión de carteras guardadas.
- **admin** (`/api/admin`): backtesting histórico del algoritmo, protegido por token de administrador.

Es importante que se realicen de manera independiente para facilitar las modificaciones, ya que no dependen de ningún otro.

7.3.5. Código del Backend

A continuación se detalla la implementación de indicadores financieros necesarios para el optimizador.

7.3.5.1. Sharpe Ratio

El Sharpe Ratio mide la relación rentabilidad por riesgo asumido, y es la métrica que se maximiza para obtener la cartera óptima:

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p} \tag{7.1}$$

- R_p **(Rentabilidad de la cartera)**: es la rentabilidad media que se espera obtener con la combinación de activos elegida.
- R_f **(Tasa libre de riesgo)**: es la rentabilidad de una inversión prácticamente sin riesgo (como la deuda pública), y se utiliza como referencia para comparar.
- σ_p **(Volatilidad de la cartera)**: es la desviación típica de los rendimientos de la cartera, y representa el riesgo que se asume.

Por lo tanto, el numerador ($R_p - R_f$) representa el *exceso de rentabilidad*: lo que se gana por encima de una inversión sin riesgo.

```
def _portfolio_return(self, weights: np.ndarray) -> float:
    # Formula: R_p = sum(w_i * R_i)
    return np.dot(weights, self.expected_returns.values)

def _portfolio_volatility(self, weights: np.ndarray) -> float:
    # Formula: sigma_p = sqrt(w^T * Cov * w)
    variance = np.dot(weights, np.dot(self.cov_matrix.values, weights))
    return np.sqrt(variance)

def _sharpe_ratio(self, weights: np.ndarray) -> float:
    portfolio_return = self._portfolio_return(weights)
    portfolio_vol = self._portfolio_volatility(weights)

    if portfolio_vol == 0:
        return 0.0

    sharpe = (portfolio_return - self.risk_free_rate) / portfolio_vol
    return sharpe
```

Listing 7.13: Cálculo del Sharpe Ratio (efficient_frontier.py)

La tasa libre de riesgo `risk_free_rate` se obtiene del factor RF de Fama-French (Sección 2.2.9).

7.3.5.2. Estimación de la covarianza: Ledoit-Wolf

Con pocos datos en relación al número de activos, calcular la matriz de covarianza es poco fiable: confunde ruido con correlaciones reales. Ledoit-Wolf suaviza la matriz acercándola a una versión más simple. Para ello, utiliza una media ponderada entre dos matrices, la calculada a partir de los datos y otra ideal en la que se asume que ningún activo se correlaciona con otro, es decir, una matriz diagonal.

```
try:
    lw = LedoitWolf()
    lw.fit(log_returns.values)
    cov_matrix = pd.DataFrame(
        lw.covariance_ * 252, index=selected, columns=selected
    )
    logger.debug(f'[{sector}] LW shrinkage={lw.shrinkage_:.4f}')
except Exception as exc:
    logger.error(f'[{sector}] Ledoit-Wolf fallo: {exc}')
    return None
```

Listing 7.14: Estimación de covarianza con Ledoit-Wolf (sp500_portfolio_generator.py)

`LedoitWolf().fit()` hace todo el proceso internamente: calcula la matriz muestral, la matriz diagonal (sin correlaciones) y el δ óptimo, y las mezcla. El resultado ya combinado queda en `lw.covariance_`(Σ_{shrunk}), mientras que `lw.shrinkage_` es el valor de δ usado, que solo se registra para depuración. La multiplicación por 252 simplemente anualiza la covarianza diaria (252 días de mercado al año) y no forma parte del shrinkage.

7.3.5.3. Estimación de retornos esperados: James-Stein

El Markowitz clásico usa la media histórica de los rendimientos como retorno esperado, pero la estimación puede dar a error con pocos datos. Se aplica *shrinkage* de James-

Stein, contrayendo cada retorno hacia la media del sector:

$$\hat{\mu}_i = (1 - \alpha)\,\bar{\mu}_i + \alpha\,\bar{\mu}_{\text{global}} \tag{7.2}$$

```
# mu_shrunk = (1-alpha) * mu_hist + alpha * mu_grand
raw_returns     = log_returns.mean() * 252
grand_mean      = raw_returns.mean()
expected_returns = pd.Series(
    (1 - RETURN_SHRINKAGE) * raw_returns + RETURN_SHRINKAGE * grand_mean,
    index=selected
)
```

Listing 7.15: Shrinkage de retornos esperados (sp500_portfolio_generator.py)

Con $\alpha = 0{,}30$ (`RETURN_SHRINKAGE`), cada retorno esperado se acerca un 30 % a la media del sector, reduciendo el ruido.

7.3.5.4. Señal de momentum: risk-adjusted 12-1 y máximo de 52 semanas

Para seleccionar los activos más recomendados de cada sector se combina una señal de momentum y si el activo está cerca del máximo en las últimas 52 semanas:

```
# Senal 1: momentum risk-adjusted 12-1
raw_mom      = sector_px.iloc[-(mom_window + skip)]
end_mom      = sector_px.iloc[-skip]           # precio de hace 1 mes
mom_ret      = (end_mom / raw_mom - 1)

vol_series   = sector_px.pct_change().rolling(mom_window).std().iloc[-1]
vol_series   = vol_series.replace(0, np.nan).fillna(vol_series.mean())
risk_adj_mom = mom_ret / vol_series

# Senal 2: 52-week high proximity (George & Hwang, 2004)
window_52w   = min(252, len(sector_px) - 1)
high_52w     = sector_px.iloc[-window_52w:].max()
ratio_52w    = sector_px.iloc[-1] / high_52w.replace(0, np.nan)
ratio_52w    = ratio_52w.fillna(0.5)

# Senal combinada: 60% momentum risk-adj + 40% 52w-high
def normalize(s):
    mn, mx = s.min(), s.max()
    return (s - mn) / (mx - mn) if mx > mn else pd.Series(0.5, index=s.index)

combined_signal = (
    0.60 * normalize(risk_adj_mom) +
    0.40 * normalize(ratio_52w)
).sort_values(ascending=False)

selected = combined_signal.head(MAX_STOCKS_PER_SECTOR).index.tolist()
```

Listing 7.16: Señal combinada de momentum (sp500_portfolio_generator.py)

7.3.5.5. Factor Momentum Overlay

Una vez calculado los pesos, se realiza un ajuste según el momentum reciente de los factores Fama-French.

Para ello se calcula si para cada uno de los cinco factores, su tendencia de último mes es positiva o negativa:

```python
def get_factor_momentum_signal(ff5: pd.DataFrame) -> Dict[str, int]:
    # +1 si el retorno acumulado del ultimo mes es positivo, -1 si negativo
    if ff5.empty or len(ff5) < MOMENTUM_WINDOW + 5:
        return {}

    available = [f for f in FACTORS if f in ff5.columns]
    recent    = ff5[available].iloc[-MOMENTUM_WINDOW:]
    cum_ret   = (1 + recent).prod() - 1

    signal = {f: int(np.sign(cum_ret[f])) for f in available if not np.isnan(
        cum_ret[f])}
    return signal
```

Listing 7.17: Señal de momentum de los factores FF5 (factor_service.py)

Para cada factor se calcula su retorno acumulado de ese mes, $\prod_t(1 + r_t) - 1$, y `np.sign(...)` lo convierte en $+1$ (tendencia positiva) o -1 (tendencia negativa).

A continuación, se calcula cuánto está expuesta cada acción a esos factores *loadings*, mediante regresión Ordinary Least Squares (OLS) y se combina con esta señal: las acciones con mayor exposición a factores en tendencia positiva reciben un ajuste de hasta `MAX_OVERLAY = +8 %` sobre su peso, y las expuestas a factores en tendencia negativa, hasta $-8\,\%$.

7.3.6. Código del Frontend

El frontend de InvestFlow está implementado mediante *JavaScript*: el navegador pide los datos al backend con `fetch` y el propio JavaScript construye la interfaz. Para las páginas, se utilizan plantillas *Jinja2* de Flask.

7.3.6.1. Jinja2

Las páginas necesitan saber si el usuario está conectado, para mostrar nombres o botones. En el caso de la aplicación, este proceso se hace una sola vez para todas las páginas en vez de calcularlo en cada ruta:

```python
@app.context_processor
def inject_user():
    return {
        'usuario_logueado': 'usuario_id' in session,
        'usuario_nombre': session.get('usuario_nombre', ''),
    }
```

Listing 7.18: Inyección de datos de sesión (app.py)

Así, en cualquier plantilla HTML se puede usar directamente esa información para mostrar una cosa u otra:

```
{% if usuario_logueado %}
  <button class="nav-link user-nav-btn" onclick="openProfileModal()">
    {{ usuario_nombre }}
  </button>
  <button class="btn btn-outline-accent" onclick="confirmarLogout()">
    Cerrar Sesión
  </button>
{% endif %}
```

Listing 7.19: Navegación condicional (index.html)

7.3.6.2. Estilos

En vez de utilizar un framework para los colores y estilos, se definen al principio del CSS una única vez. En el caso de que se quiera cambiar el estilo de toda la aplicación solo habría que cambiarlo en un único lugar.

```
:root {
    --bg-primary: #0a0e27;
    --bg-secondary: #131842;
    --accent-primary: #00ff88;
    --accent-secondary: #00d4ff;
    --accent-gold: #ffd700;
    --text-primary: #ffffff;
    --text-secondary: #b8c5d6;
}
```

Listing 7.20: Paleta de colores (style.css)

7.3.6.3. Gráficos: Chart.js

Para mostrar la evolución de los precios de los activos o de las carteras generadas por el algoritmo, se utiliza una librería *Chart.js*, que realiza los gráficos a partir de los datos que devuelve el backend. Además, también utiliza gráficos de barras para mostrar las ganancias o pérdidas de cada posición en las carteras generadas.

```
tickerChart = new Chart(ctx, {
    type: 'line',
    data: {
        labels: xLabels,
        datasets: [{
            data: yData,
            borderColor: color,
            backgroundColor: grad,
            fill: true,
            tension: 0.1,
        }]
    },
    options: {
        responsive: true,
        maintainAspectRatio: false,
        plugins: { legend: { display: false } }
    }
});
```

Listing 7.21: Grafico de evolución de precio (market.js)

El color de la línea (`color`, `grad`) cambia automáticamente: verde si el precio ha subido y rojo si ha bajado.

7.3.6.4. Animaciones con GSAP

En la página `aprende` se hace uso de la librería *GSAP* para que cada apartado de texto o imagen vaya apareciendo a medida que el usuario baja con el scroll.

7.4. Aspectos de Seguridad

En esta sección se detallan las medidas de seguridad implementadas para proteger los datos de los usuarios y la integridad de la plataforma.

7.4.1. Inyección SQL

Todas las consultas a la base de datos están parametrizadas (`%s`) para evitar que se realice cualquier inyección que modifique la consulta SQL:

```
cursor.execute("""
    SELECT * FROM usuarios
    WHERE (email = %s OR username = %s) AND activo = 1
""", (identifier, identifier))
```

Listing 7.22: Consulta parametrizada (usuarios.py)

7.4.2. Cabeceras HTTP y cookies de sesión

Se configuran las cabeceras HTTP y los atributos de la cookie de sesión:

```
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['SESSION_COOKIE_SECURE'] = os.getenv('FLASK_DEBUG', 'True').lower
    () != 'true'

@app.after_request
def set_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    return response
```

Listing 7.23: Cabeceras de seguridad y configuración de cookies (app.py)

`HttpOnly` impide que la cookie de sesión se pueda leer desde JavaScript, y `SameSite=Lax` evita que se envíe en peticiones que provienen de otras webs. `SESSION_COOKIE_SECURE` hace que la cookie solo se transmita por HTTPS cuando la aplicación está en producción. Además, `X-Content-Type-Options: nosniff` impide que el navegador interprete un archivo como un tipo distinto al que se indica, y `X-Frame-Options: DENY` impide que la página se cargue dentro de un `iframe` de otra web (*clickjacking*).

7.4.3. Prevención de XSS

Las plantillas se renderizan con *Jinja2* para que no alteren las consultas mediante slash-es. El objetivo es que si un usuario modifica el código HTML o *JavaScript* en alguno de los campos, se mostrará como texto plano y no se ejecuta en el navegador de otros usuarios.

7.4.4. Almacenamiento de contraseñas de forma segura

Las contraseñas de los usuarios no se guardan en texto plano. Se utiliza `werkzeug.security` que utiliza el algoritmo `PBKDF2` para generar un *hash*:

```
generate_password_hash(data['password'])
...
check_password_hash(usuario['password_hash'], password)
```

Listing 7.24: Hash y verificación de contraseñas (usuarios.py)

7.4.5. Recuperación de contraseñas

Cuando un usuario no recuerda la contraseña, la plataforma genera un token único, de un solo uso y con caducidad:

```
token      = secrets.token_urlsafe(32)
expires_at = datetime.utcnow() + timedelta(hours=1)
cursor.execute(
    'INSERT INTO password_reset_tokens (id_usuario, token, expires_at, tipo)
        '
    'VALUES (%s, %s, %s, %s)',
    (user['id_usuario'], token, expires_at, 'reset')
)
```

Listing 7.25: Generación del token de recuperación (usuarios.py)

7.4.6. Protección de rutas

Cuando una ruta requiere estar autenticado se utiliza `login_required` para comprobar si ya existe en el sistema una sesión activa antes de ejecutar la orden:

```
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if 'usuario_id' not in session:
            if request.path.startswith('/api/') or request.is_json:
                return jsonify({'success': False, 'error': 'No autenticado'})
                    , 401
            return redirect(url_for('auth'))
        return f(*args, **kwargs)
    return decorated
```

Listing 7.26: Decorador de autenticación (auth.py)

8. Pruebas

A lo largo del proceso de desarrollo se han ido realizando diversas pruebas con el objetivo de que la aplicación cumpla las expectativas y detectar los errores que pueden surgir antes de poner la aplicación en producción.

En esta sección, se han dividido las pruebas en cinco apartados distintos: pruebas de flujo de trabajo, pruebas de funcionalidad, pruebas con usuarios reales, pruebas de seguridad y pruebas de los indicadores utilizados.

8.1. Pruebas de Flujo de Trabajo

Para el correcto desarrollo del proyecto se ha utilizado *Git* como sistema para gestionar las versiones, junto con el repositorio remoto enm *GitHub*. Durante el desarrollo, se han ido eliminando y añadiendo funcionalidades, corrigiendo errores y modificando líneas de código, estos cambios se muestran mediante *commits* con mensajes que describen el cambio realizado.

Este sistema de control de versiones es esencial para poder comparar versiones al añadir una nueva funcionalidad. Además, en caso de error al añadir una nueva funcionalidad, el sistema permite restaurar el proyecto a un estado anterior donde la aplicación funcionase correctamente.

8.2. Pruebas de funcionalidad

Durante el proyecto, se ha utilizado el modo de depuración (*debug mode*) de Flask, que muestra un log con los detalles de cualquier error que se pueda producir en el servidor. Es importante mantener un log a lo largo del proyecto para poder detectar el origen del error y corregir de manera rápida y sencilla.

A continuación se recogen distintas pruebas funcionales realizadas en la aplicación, organizadas según si el usuario ha sido autenticado o no.

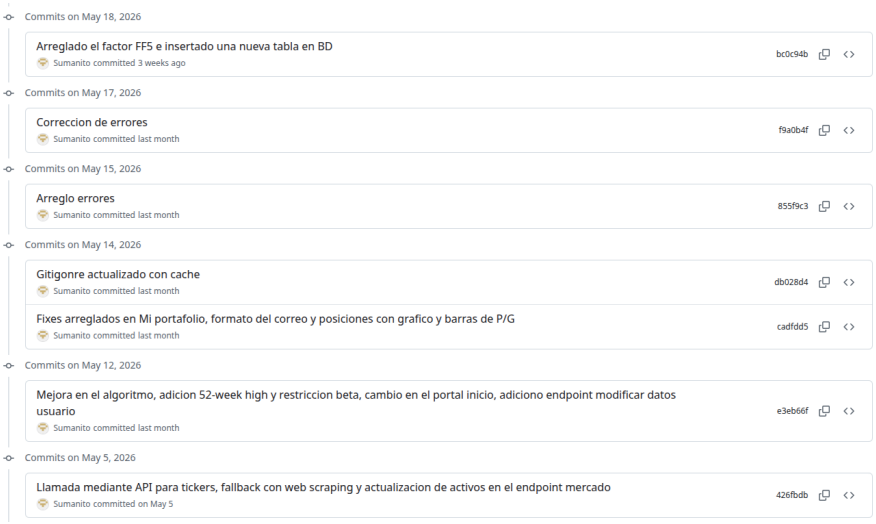


Figura 52: Historial de commits en GitHub

8.2.1. Pruebas para un usuario sin identificar

Prueba	Descripción	Resultado Esperado	Resultado Real
Prueba 1: Registro con algún campo obligatorio vacío	El usuario deja vacío algún campo obligatorio (nombre de usuario, email, contraseña...)	El sistema responde con el error *"Campo requerido"* y no se crea la cuenta.	Es correcto.
Prueba 2: Nombre de usuario demasiado corto	El usuario introduce un nombre de usuario de menos de 3 caracteres	El sistema indica que el nombre de usuario debe tener al menos 3 caracteres.	Es correcto.
Prueba 3: Email con formato inválido	El usuario introduce un email sin el formato correcto (sin @, sin dominio...)	El sistema indica que el correo electrónico no tiene un formato válido.	Es correcto.
Prueba 4: Contraseña demasiado corta	El usuario introduce una contraseña de menos de 6 caracteres	El sistema indica que la contraseña debe tener al menos 6 caracteres.	Es correcto.
Prueba 5: Registro con un email ya existente	El email introducido ya está registrado en la base de datos	El sistema indica que ya existe una cuenta con ese email y no se crea el registro.	Es correcto.
Prueba 6: Registro con un nombre de usuario ya existente	El nombre de usuario introducido ya está en uso por otra cuenta	El sistema indica que ese nombre de usuario ya está en uso.	Es correcto.
Prueba 7: Registro correcto	Todos los campos obligatorios se rellenan correctamente y de forma válida	Se crea la cuenta correctamente y se informa al usuario.	Es correcto.

Tabla 42: Pruebas de registro para un usuario sin identificar

8.2.2. Pruebas para un usuario identificado

Prueba	Descripción	Resultado Esperado	Resultado Real
Prueba 1: Generar cartera con un importe de inversión no válido	El usuario introduce un importe de inversión negativo, cero	El sistema indica que el importe de inversión debe ser mínimo 100.	Es correcto.
Prueba 2: Generar cartera con un importe de inversión no numérico	El usuario introduce un importe no numérico	El sistema lo bloquea dejando únicamente introducir números.	Es correcto.
Prueba 3: Generar cartera correctamente	El usuario selecciona un los parámetros necesarios para generar la cartera	Se genera la cartera optimizada mostrando los pesos, el retorno esperado, el riesgo y el ratio de Sharpe.	Es correcto.
Prueba 4: Guardar una cartera generada	El usuario pulsa el botón de guardar cartera	La cartera se guarda en la base de datos y aparece en la sección "Mis Portafolios".	Es correcto.

Tabla 43: Pruebas del generador de carteras del S&P 500

Prueba	Descripción	Resultado Esperado	Resultado Real
Prueba 5: Ver Mis Portafolios sin carteras guardadas	El usuario accede a la sección de portafolios sin haber guardado ninguna cartera	Muestra mensaje "Sin portafolios aún.eiinvita a crear uno.	Es correcto.
Prueba 6: Ver el balance de una cartera guardada	El usuario abre una cartera previamente guardada	Se muestra el rendimiento de la cartera, el gráfico de posiciones y el diagrama de barras de ganancias/pérdidas por activo.	Inicialmente no mostraba la información (error modificado en el desarrollo). Una vez corregido, es correcto el funcionamiento.
Prueba 7: Eliminar una cartera guardada previamente	El usuario pulsa el botón de eliminar sobre una de sus carteras	La cartera se elimina correctamente y desaparece de la lista.	Es correcto.
Prueba 8: Modificar datos del usuario con un nombre de usuario en formato de correo electrónico	El usuario modifica los datos introduciendo un nombre de usuario inválido	Muestra un mensaje indicando que el nombre de usuario no puede ser un correo electrónico.	Es correcto.
Prueba 9: Modificar datos del usuario con un nombre de usuario en formato de dominio	El usuario modifica los datos introduciendo un nombre de usuario inválido	Muestra un mensaje indicando que el nombre de usuario no puede ser un dominio web.	Es correcto.
Prueba 10: Modificar datos del usuario con un email ya en uso	El usuario intenta cambiar su email por uno que ya pertenece a otra cuenta	El sistema indica que el email ya está en uso por otra cuenta y no realiza el cambio.	Es correcto.
Prueba 11: Cerrar sesión	El usuario pulsa el botón de cerrar sesión	La sesión se cierra correctamente y se redirige al usuario a la página principal.	Es correcto.
Prueba 12: Consultar mercado de valores del índice.	El usuario pulsa en un activo para ver el rendimiento a lo largo del tiempo y buscar la información de la empresa.	El sistema muestra un gráfico indicando el rendimiento del activo e información empresarial.	Al pulsar sobre un activo se observaba un salto en los precios, debido a que la información no se actualizaba con los ajustes retroactivos. Una vez corregido, el gráfico muestra correctamente la evolución del rendimiento del activo.

Tabla 44: Pruebas de gestión de portafolios y cuenta de usuario

8.3. Pruebas con Usuarios Reales

Una vez realizado el proyecto y con la aplicación en condiciones de ponerse en producción, se ha solicitado a un grupo de personas (amigos y familiares sin conocimientos previos) que probaran la plataforma libremente y compartieran sus opiniones. El objetivo es conocer cómo se desenvuelve un usuario potencial con la aplicación e identificar si la experiencia ha sido satisfactoria o no.

Además, este tipo de pruebas permite detectar problemas de usabilidad y errores difíciles de identificar mediante distintas pruebas para el programador.

Otro de los objetivos es comprobar si la aplicación es intuitiva y de fácil uso para personas sin conocimientos de inversión. Gracias a los comentarios recibidos, se han realizado pequeños ajustes en algunas interfaces como la sección *Aprende*, que está pensada como un tutorial donde se explica de forma sencilla el funcionamiento del algoritmo utilizado.

8.4. Pruebas de Seguridad

Durante el proyecto, no se ha utilizado una herramienta para escanear y detectar fallos de seguridad. Se ha decidido realizar las siguientes pruebas manualmente:

1. **Inyección SQL**
   - **Prueba realizada:** En los campos de inicio de sesión y registro se han introducido cadenas típicas simulando un atacante, como `ÓR '1'='1 o '; DROP TABLE usuarios; --`.
   - **Resultado:** Como las consultas a la base de datos están parametrizadas, dichas cadenas se interpretan como texto plano. El acceso se rechaza y la base de datos permanece intacta.
2. **Cross-Site Scripting (XSS)**
   - **Prueba realizada:** Se ha intentado registrar un usuario y guardar datos del perfil utilizando como valor código como `<script>alert('XSS')</script>`.
   - **Resultado:** Jinja2 escapa automáticamente el contenido al renderizar, mostrándose como texto plano sin llegar a ejecutarse como script.
3. **Almacenamiento de contraseñas**
   - **Prueba realizada:** Se ha consultado directamente la tabla `usuarios` en la base de datos tras registrar una cuenta nueva.
   - **Resultado:** El campo `password_hash` no contiene la contraseña en texto plano, sino un hash con formato `scrypt:N:r:p$salt$hash` generado por Werkzeug, distinto en cada registro aunque se repita la misma contraseña.
4. **Recuperación de contraseña con email registrado**
   - **Prueba realizada:** Se ha solicitado la recuperación de contraseña con un email registrado.
   - **Resultado:** El sistema muestra un enlace de recuperación donde se podrá introducir la nueva contraseña.

5. **Recuperación de contraseña con email no registrado**
    - **Prueba realizada:** Se ha solicitado la recuperación de contraseña con un email no registrado.
    - **Resultado:** El sistema lanza un mensaje indicando que, si el correo estuviera registrado, podrá recuperar la contraseña en la bandeja de entrada del email. De esta manera, la respuesta no enseña si un correo electrónico está o no registrado en la aplicación.
6. **Protección de rutas privadas**
    - **Prueba realizada:** Sin haber iniciado sesión (eliminando la `cookie`, se ha intentado acceder directamente a distintos endpoints de la plataforma como `/market`, `/portfolios`, `/sp500`.
    - **Resultado:** Las páginas redirigen automáticamente a la pantalla de inicio de sesión.
7. **Cabeceras de seguridad HTTP**
    - **Prueba realizada:** Se han revisado las cabeceras de respuesta del servidor desde las herramientas de desarrollador del navegador.
    - **Resultado:** En todas las respuestas aparecen las cabeceras `X-Content-Type-Options: nosniff` y `X-Frame-Options: DENY`.
8. **Atributos de la cookie de sesión**
    - **Prueba realizada:** Se ha revisado la cookie de sesión generada al iniciar sesión.
    - **Resultado:** La cookie tiene activados los atributos `HttpOnly` y `SameSite=Lax`.

8.5. Pruebas de indicadores

Esta sección verifica que las funciones encargadas de calcular los indicadores fundamentales de la optimización de Markowitz —retorno esperado de la cartera, volatilidad (riesgo) y ratio de Sharpe—, implementadas en la clase `EfficientFrontier` (`app/services/efficient_frontier.py`), devuelven los valores correctos.

Para ello se ha creado el script `test_indicadores.py`, que genera datos controlados mediante `numpy.random.seed(42)` para 3 activos ficticios, define una matriz de covarianzas simétrica y semidefinida positiva, y establece unos pesos de cartera conocidos (`[0.5, 0.3, 0.2]`). A partir de estos datos se calculan a mano, con `numpy`, los tres indicadores:

- Retorno esperado: $R_p = \sum_i w_i R_i$
- Volatilidad: $\sigma_p = \sqrt{w^\top \Sigma w}$
- Ratio de Sharpe: $\text{Sharpe} = \dfrac{R_p - R_f}{\sigma_p}$, con $R_f = 0{,}02$

Estos valores calculados manualmente se comparan con los devueltos por los métodos `_portfolio_return`, `_portfolio_volatility` y `_sharpe_ratio` de `EfficientFrontier`, con una tolerancia de 10^{-6}.

Indicador	Valor esperado	Valor obtenido	¿Correcto?
Retorno esperado (R_p)	0.111888	0.111888	Sí
Volatilidad (σ_p)	0.014672	0.014672	Sí
Ratio de Sharpe	6.262709	6.262709	Sí

Tabla 45: Resultados de las pruebas de indicadores.

Los tres indicadores coinciden exactamente (diferencia inferior a 10^{-6}) con los valores calculados manualmente, lo que confirma que las fórmulas de retorno, riesgo y Sharpe están correctamente implementadas.

8.6. Pruebas del módulo de backtesting

Esta sección comprueba que el módulo de backtesting (función `_calcular_performance`, en `app/routes/admin_routes.py`, utilizada por el *endpoint* `/api/admin/backtest`) calcula correctamente el rendimiento real de una cartera entre dos fechas.

Para ello se ha creado el script `test_backtesting.py`, que define una cartera conocida compuesta por tres activos (AAPL, MSFT y JNJ) con una inversión inicial de 10.000 €repartida según los pesos 50 %, 30 % y 20 % respectivamente, y un periodo histórico concreto: del 03/01/2023 al 02/01/2024.

A partir de los precios de cierre reales almacenados en la tabla `sp500_precios` para esas dos fechas, se calcula manualmente, para cada activo, el número de acciones compradas (`importe / precio_entrada`), el valor final de la posición (`acciones × precio_salida`) y el retorno porcentual. Estos valores se comparan con los que devuelve `_calcular_performance` para la misma cartera y periodo.

Indicador	Valor esperado	Valor obtenido	¿Correcto?
AAPL – Valor final (€)	7462.93	7462.93	Sí
AAPL – Retorno (%)	49.26	49.26	Sí
MSFT – Valor final (€)	4685.18	4685.18	Sí
MSFT – Retorno (%)	56.17	56.17	Sí
JNJ – Valor final (€)	1849.98	1849.98	Sí
JNJ – Retorno (%)	-7.50	-7.50	Sí
Cartera – Valor final (€)	13998.09	13998.09	Sí
Cartera – Retorno absoluto (€)	3998.09	3998.09	Sí
Cartera – Retorno (%)	39.98	39.98	Sí

Tabla 46: Resultados de las pruebas del módulo de backtesting.

Todos los valores calculados manualmente coinciden con los devueltos por el módulo de backtesting, lo que confirma que el cálculo del rendimiento real de una cartera (precio de entrada, precio de salida, acciones compradas, valor final y retorno) se realiza correctamente.

9. Conclusiones

En este capítulo se resume el trabajo realizado durante el proyecto, se evalúan los cumplimientos realizados respecto a los objetivos planteados inicialmente, se analizan las desviaciones y los riesgos que se produjeron.

9.1. Objetivos

El objetivo principal era desarrollar una aplicación web funcional que permitiera generar carteras de inversión diversificadas del S&P 500 aplicando el algoritmo de Markowitz y ofreciendo una interfaz legible para el usuario.

Respecto al planteamiento inicial, se ha implementado el algoritmo de Markowitz cumpliendo el objetivo esperado. Además el resultado final ha superado el alcance del proyecto, ya que se ha mejorado el algoritmo a través de tres implementaciones:

- **Ledoit-Wolf** Se estima robustamente la matriz de covarianzas, lo que reduce el ruido estadístico en carteras con muchos activos.
- **James-Stein** Se contraen los retornos esperados mediante el estimador para generar carteras más estables.
- **Fama-French (FF5)** Se añaden los cinco factores para establecer y mejorar los pesos en los activos de más valor.

Además, se han añadido las señales de selección por máximo de 52 semanas y el filtro de momentum, consolidando la optimización para poder establecer un algoritmo más robusto.

Desde el punto de vista de la planificación temporal, el proyecto se ha finalizado en los plazos establecidos. Han existido pequeñas desviaciones causadas por el desarrollo e implementación del código, ya que es la parte más complicada del proyecto.

En cuanto a riesgos durante la planificación, el único que no llegó a resolverse como se esperaba fue la viabilidad de la API financiera, que requería planes de pago para poder funcionar en la aplicación. Por otro lado, funcionalidades como el rebalanceo automático de carteras se plantearon inicialmente pero se descartó para completar con los plazos correctamente.

El control de las versiones ha sido satisfactorio a lo largo del proyecto. Teniendo un historial en GitHub permite identificar exactamente dónde se aloja el error y poder resolverlo con rapidez.

9.2. Reflexión Personal

Terminar el trabajo que tenía previsto y realizarlo en el tiempo establecido me ha transmitido una sensación de compromiso y autoconciencia. La aplicación salió gracias a la inquietud de querer invertir y proteger el dinero frente a la inflación. Intentar realizar algo que de verdad valga la pena para ello me ha dado una tranquilidad como final para mi carrera.

Respecto al nivel técnico, me ha ayudado a recordar conceptos que he ido aprendiendo a lo largo de la carrera, como son el manejo de base de datos y programación con el modelo cliente-servidor. Además, para completar toda la formación a la hora de desarrollar el proyecto, es necesario realizar búsquedas en artículos académicos, repositorios y tiempo dedicado a aprender. Esa búsqueda será necesaria para el mundo laboral, ya que una frase que suelo decir yo:

"Un ingeniero informático no es el que más conceptos conoce, sino el que mejor sabe buscar la información."

En ocasiones te desesperas ya que no consigues avanzar, pero es necesario tener paciencia y resiliencia para seguir adelante y ante momentos de duda, superarlos. El ejemplo más claro es: la vida en general.

Por último, me gustaría agradecer a mis familiares y amigos que siempre me ayudan a mejorar y a sacar todo adelante, así como a mi tutor Kepa que sin su ayuda y consejos esto no habría salido adelante.

10. Trabajo Futuro

Los objetivos planteados al inicio del proyecto se han alcanzado con éxito. Sin embargo, una vez finalizado el proyecto, se pueden construir mejoras tanto a nivel de algoritmo financiero como de arquitectura. A continuación, se detallan mejoras de cara al trabajo futuro.

10.1. Envío Real de Correos y Autenticación

10.1.1. Envío de Correos

Actualmente, para la recuperación de contraseña se genera un token de un solo uso almacenado en la base de datos, pero el envío del correo está representado por una función que registra el enlace en el *log* y no se termina de enviar al usuario. El propio enlace se devuelve en la respuesta.

Como mejora futura, esta función debería sustituirse por un envío real, mediante una cuenta de Gmail con autenticación SMTP o un servicio de correo transaccional como Amazon SES.

10.1.2. Registro e Inicio de Sesión con Proveedores Externos

Incorporar el inicio de sesión mediante proveedores como Google mediante OAuth 2.0 permitiría delegar esa verificación con proveedor externo de confianza, reducir el tiempo que tarda el usuario en registrarse o iniciar sesión, además de reducir inconvenientes en los datos del usuario.

10.2. Rebalanceo Automático de Carteras

Una vez que el usuario genera una cartera optimizada, la aplicación registra su rendimiento futuro, pero no existe ningún mecanismo que detecte cuándo los pesos de los activos se han desviado y necesitan ser recalculados. Una mejora sería implementar un módulo de rebalanceo, partiendo de carteras ya generadas y registradas en la base de datos, que compare las posiciones actuales y sugiera al usuario operaciones de compra/venta para alinearse a los parámetros que introdujo el usuario al comienzo de la generación de la cartera.

10.3. Ampliación de activos

El sistema se centra exclusivamente en las empresas del S%P 500, lo que limita la diversificación de la cartera en distintos índices o activos. Como trabajo futuro se podría ampliar el universo a otros índices (IBEX 35, STOXX 600, NASDAQ, etc).

10.4. Conexión con Broker Real

Actualmente, la aplicación consta de un algoritmo informativo, es decir, no invierte automáticamente, sino que se limita a ofrecer oportunidades y recomendaciones de inversión en distintas carteras. Una de las mejoras futuras más ambiciosa sería conectar la aplicación con la API de un bróker real para que, una vez generada la cartera y aprobada la inversión, el sistema pueda realizar las transacciones para comprar las posiciones. Junto a esta funcionalidad, el sistema requeriría mayores aspectos de seguridad y normativas.

10.5. Comunidad de inversores

Por otro lado, la aplicación podría dar un salto si permitiera a los usuarios compartir carteras, opiniones o estrategias. Esto aportaría gran valor añadido a la comunidad de usuarios y ampliar el conocimiento de las inversiones a todo tipo de usuarios.

10.6. Aplicación Móvil

La aplicación está desarrollada como sitio web, accesible desde cualquier dispositivo a través del navegador. Una app dedicada permitiría, mantener al usuario informado con notificaciones y acciones relevantes para su cartera. La aplicación móvil, utilizando frameworks multiplataforma, sería una práctica imprescindible para brindarle un valor añadido profesional.

Bibliografía

[1] GitHub. *GitHub Copilot*. 2026. https://github.com/features/copilot

[2] Anthropic. *Claude*. 2026. https://claude.ai/

[3] OpenAI. *ChatGPT*. 2026. https://chatgpt.com/

[4] Fondo Monetario Internacional. *Fiscal Monitor: Putting a Lid on Public Debt*. FMI, 2024. https://www.imf.org/en/Publications/FM

[5] Federal Reserve Bank of St. Louis. *M2 Money Stock*. FRED, 2025. https://fred.stlouisfed.org/series/M2SL

[6] Banco Central Europeo. *Asset purchase programmes*. BCE, 2018. https://www.ecb.europa.eu/mopo/implement/app/html/index.en.html

[7] Instituto Nacional de Estadística. *Índice de Precios de Consumo. Julio 2022*. INE, 2022. https://www.ine.es/daco/daco42/daco421/ipc0722.pdf

[8] Banco de España. *Tipos de interés de nuevas operaciones de depósitos*. Banco de España, 2022. https://www.bde.es/webbde/es/estadis/infoest/tipos/tipos.html

[9] Funcas. *Encuesta de cultura financiera en España*. Funcas, 2023. https://www.funcas.es

[10] Banco Santander. *El valor de aprender: cultura financiera en España y el mundo*. Banco Santander, 2023. https://www.bancosantander.es

[11] Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77–91. https://doi.org/10.1111/j.1540-6261.1952.tb01525.x

[12] Hernández, L. Á. (2022). *¿Qué es y cómo funciona el modelo de Markowitz? Teoría de la cartera y frontera eficiente*. Rankia. https://www.rankia.com/blog/bolsa-desde-cero/3479118-que-como-funciona-modelo-markowitz-teoria-cartera-frontera-eficiente

[13] S&P Dow Jones Indices. *S&P 500*. 2026. https://www.spglobal.com/spdji/en/indices/equity/sp-500/

[14] S&P Dow Jones Indices. *S&P Sectors*. 2026. https://www.spglobal.com/spdji/en/index-family/equity/us-equity/sp-sectors/

[15] James, W., y Stein, C. (1961). Estimation with quadratic loss. *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 361–379. https://projecteuclid.org/proceedings/

berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fourth-Berkeley-Symposium-on-Mathematical-Statistics-and/Chapter/Estimation-with-Quadratic-Loss/bsmsp/1200512173

[16] Sharpe, W. F. (1964). *Capital asset prices: A theory of market equilibrium under conditions of risk*. The Journal of Finance, 19(3), 425–442. https://doi.org/10.2307/2977928

[17] Ledoit, O., y Wolf, M. (2003). Honey, I Shrunk the Sample Covariance Matrix. *The Journal of Portfolio Management*, 30(4), 110–119. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=433840

[18] Ledoit, O., y Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411. https://doi.org/10.1016/S0047-259X(03)00096-4

[19] Fama, E. F., y French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1), 1–22. https://doi.org/10.1016/j.jfineco.2014.10.010

[20] French, K. R. *Kenneth R. French Data Library*. Dartmouth College, 2026. https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html

[21] Naciones Unidas. *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*. ONU, 2015. https://www.un.org/sustainabledevelopment/es/

[22] Comisión Nacional del Mercado de Valores y Banco de España. *Encuesta de competencias financieras*. CNMV, 2022. https://www.cnmv.es/portal/Publicaciones/ECF.aspx

[23] Glassdoor. *Sueldo: Programador Junior en España*. 2026. https://www.glassdoor.es/Sueldos/programador-junior-sueldo-SRCH_KO0,18.htm

[24] Ministerio de Universidades. *Salario medio del profesorado universitario en España*. 2024. https://www.universidades.gob.es

[25] Red Eléctrica de España. *Precio medio del kWh en España 2025*. 2025. https://www.ree.es

[26] Indexa Capital. *Indexa Capital: Robo-advisor líder en España*. 2024. https://indexacapital.com

[27] Finizens. *Finizens: Gestión indexada automatizada*. 2024. https://finizens.com

[28] eToro. *eToro: The World's Leading Social Trading Platform*. 2024. https://etoro.com

[29] Aroussi, R. *yfinance: Yahoo! Finance market data downloader*. 2024. https://github.com/ranaroussi/yfinance

[30] Wikipedia. *List of S&P 500 companies*. Wikipedia, 2024. https://en.wikipedia.org/wiki/List_of_S%26P_500_companies

[31] DataHub. *S&P 500 Companies dataset*. GitHub, 2024. https://github.com/datasets/s-and-p-500-companies

[32] Financial Modeling Prep. *Financial Modeling Prep API Documentation*. 2024. https://financialmodelingprep.com/

[33] Alpha Vantage. *Alpha Vantage API Documentation*. 2024. https://www.alphavantage.co/

[34] Polygon.io. *Polygon.io API Documentation*. 2024. https://polygon.io/

[35] Reitz, K., y otros colaboradores. *Requests: HTTP for Humans*. 2026. https://requests.readthedocs.io/en/latest/

[36] Visual Paradigm. *Visual Paradigm Online Diagram Tool*. 2026. https://online.visual-paradigm.com/app/diagrams/

[37] GlooMaps. *GlooMaps: Visual Sitemap Tool*. 2026. https://www.gloomaps.com/

[38] JGraph Ltd. *draw.io / diagrams.net: Diagramming Application*. 2026. https://www.drawio.com/

[39] GanttProject. *GanttProject: Free Project Management App*. 2026. https://ganttproject.biz/

[40] Pallets Projects. *Flask Documentation*. 2024. https://flask.palletsprojects.com

[41] The Bootstrap Authors. *Bootstrap 5 Documentation*. 2024. https://getbootstrap.com

[42] Chart.js Contributors. *Chart.js Documentation*. 2024. https://www.chartjs.org

[43] Harris, C. R. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[44] Virtanen, P. et al. (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2

[45] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference. https://doi.org/10.25080/Majora-92bf1922-00a

[46] Oracle Corporation. *MySQL 8.0 Reference Manual*. 2024. https://dev.mysql.com/doc/refman/8.0/en/

[47] SciPy Developers. *minimize(method='SLSQP')*. SciPy Documentation, 2026. https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html

[48] scikit-learn developers. *LedoitWolf*. scikit-learn Documentation, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.covariance.LedoitWolf.html

[49] pandas-datareader developers. *Fama-French Data: Ken French's Data Library*. pandas-datareader Documentation, 2024. https://pandas-datareader.readthedocs.io/en/latest/readers/famafrench.html

[50] Pallets Projects. *Werkzeug Utilities: Security helpers*. Werkzeug Documentation, 2026. https://werkzeug.palletsprojects.com/en/stable/utils/

[51] Mozilla Developer Network. *Using HTTP cookies*. MDN Web Docs, 2026. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

[52] Limon Monte, L. *SweetAlert2 Documentation*. 2024. https://sweetalert2.github.io

[53] Michalsnik. *AOS: Animate On Scroll Library*. 2024. https://michalsnik.github.io/aos/

[54] GSAP. *GSAP Documentation*. 2026. https://gsap.com/docs/v3/

[55] Richardson, L. *Beautiful Soup Documentation*. 2024. https://www.crummy.com/software/BeautifulSoup/

[56] Microsoft. *Visual Studio Code Documentation*. 2024. https://code.visualstudio.com/docs

[57] Software Freedom Conservancy. *Git Documentation*. 2024. https://git-scm.com/doc

[58] GitHub Inc. *GitHub Documentation*. 2024. https://docs.github.com

[59] Canonical Ltd. *Ubuntu 24.04 LTS*. 2024. https://ubuntu.com

[60] Overleaf. *Overleaf Documentation*. 2024. https://www.overleaf.com